

שם התלמידה: דנה קוץ

שם העבודה: יעלוטרון

ת.ז. התלמידה: 217385798

מנחים: ליאת סגל, רום רפסון,

עודי רז, עידן פלדברג, קובי שוצמן.

שם החלופה: הגנת סייבר

תאריך הגשה: 21.6

תוכן עניינים

מבוא:	4
ייזום	4
פירוט תיאור המערכת	5
תיאור תחום הידע- פרק מילולי	7
פירוט מעמיק של היכולות שהוצגו בשלב הקודם ומעבר	7
היכולות בצד שרת:	7
היכולות בצד לקוח:	8
מבנה / ארכיטקטורה של הפרויקט	11
תיאור הארכיטקטורה של המערכת המוצעת	11
פירוט ההחלטות שנלקחו למימוש המערכת:	11
תיאור הטכנולוגיה הרלוונטית:	11
תיאור זרימת המידע במערכת:	12
תיאור האלגוריתמים המרכזיים בפרויקט:	14
תיאור סביבת הפיתוח:	16
תיאור פרוטוקול התקשורת:	17
תיאור מסכי המערכת:	19
תיאור מבני הנתונים:	25
סקירת חולשות ואיומים:	27
מימוש הפרויקט	28
פירוט מימוש המערכת- חלק א'	28
סקירת המודולים המיובאים:	28
מודולים/מחלקות בפיתוח עצמי:	29
חלק ב'	42
חלק ג'	48
מדריך למשתמש	50
פירוט על כלל קבצי המערכת:	50
התקנת המערכת:	51
משתמשי המערכת:	51
רפלקציה	57
ביבליוגרפיה	58
נעזרתי במקורות הבאים:	58

58.....	קטעי קוד ממקורות חיצוניים:
59.....	נספחים
59.....	הקובץ main:
64.....	הקובץ client:
66.....	הקובץ staff:
70.....	הקובץ build:
73.....	הקובץ creator:
79.....	הקובץ rsa_encryption:
81.....	הקובץ log:
85.....	הקובץ reshape:
86.....	הקובץ screens:
91.....	הקובץ view_data:
97.....	הקובץ server:
105.....	הקובץ db_worker:

מבוא:

ייוזם:

- היעלוטרון היא אפליקציה המאפשרת לעובדי מעבדה ליעל את תהליך עבודתם דרך דיגיטציה של חלק מעבודתם. האפליקציה מספקת ממשק דיגיטלי דרכו עובדים יכולים לעקוב אחר הפרוטוקולים לבדיקות שהם מבצעים, ושומרת תיעוד לכל בדיקה. הפרוטוקולים הינם רשימות של הוראות והנחיות לביצוע בדיקות מעבדה. הם מתפקדים כ- "צ'קליסט". בחרתי בפרויקט זה לאחר ששמעתי על תהליך העבודה הלא יעיל במעבדה המטבולית בבית החולים שיבא תל השומר בה עובדת אמי, יעל. מכאן שם הפרויקט- יעלוטרון. בחרתי אותו כיוון והאפליקציה היא מתנה עבורה. בהתאם להנחיות משרד הבריאות, כאשר אחד העובדים במעבדה מבצע בדיקה חובה עליו להדפיס את פרוטוקול העבודה שלה, ולסמן ווי לאחר ביצוע כל שלב. לאחר מכן על העובד לשמור את אותו טופס עם שמו ותאריך ביצוע הבדיקה בתיקיה. בעקבות כך המשרדים במעבדה מלאים במדפים רבים ומסורבלים עם עשרות תיקיות מלאות בדפים, למרות שבצעם ישנם עשרות דפים כמעט זהים. בנוסף לכך, בעקבות רדיפת בית החולים אחר סטטוס של "בית חולים ירוק" העובדים מוגבלים בכמות הדפים שמוקצים להם מידי חודש. העובדים לעתים נאלצים לבזבז זמן יקר בחיפוש אחר דפים אצל מעבדות אחרות. במעבדה ישנו מחסור חמור בכוח אדם, העובדים עמוסים והעבודה דחופה וחיונית. כל ההתעסקות בדפי הפרוטוקולים מבזבזת לעובדים זמן יקר. האפליקציה שלי אמורה לעזור לבעיה זו. אחד האתגרים שאני צופה לפרויקט זה הוא אתגר האבטחה, למרות שהאפליקציה לא תנהל מידע רגיש על חולים, אבטחת ושלמות המידע היא חיונית.
- המערכת מיועדת לעובדי המעבדה המטבולית בשיבא תל השומר, אך היא עשויה להיות שימושית גם עבור מעבדות אחרות אם תוכיח את עצמה במעבדה הראשונה.
- מטרת הפרויקט היא לספק ממשק דיגיטלי נוח, מאובטח ומסודר עבור עובדי מעבדה לפרוטוקולים בהם הם משתמשים.
- הבעיה הקיימת היום היא שתהליך העבודה במעבדה אינו דיגיטלי, מה שהופך אותו לאיטי, לא מסודר, לא יעיל ולא אמין. הדיגיטציה של התהליך דרך היעלוטרון תוביל להסדר שלו, ולחסכון משמעותי בזמן העובדים מה שיאפשר להם להתמקד בעבודתם החשובה.
- הפתרון שקיים היום הוא הדפסת הפרוטוקולים, ששמורים על המחשבים במעבדה באופן לא מסודר ובפורמטים שונים על דפים כדי לעבוד איתם, כתיבת פרטים (תאריך, שם העובד) על הדף באופן ידני, והכנסתו לתיקיה המתאימה לו.

- כיום אין פתרון טכנולוגי ספציפי לבעיה אותה אני פותרת, אך ישנם פתרונות לבעיות דומות- אפליקציות ארגוניות שונות שמספקות ממשק לעבוד עם רשימת מטלות, אך אפליקציות אלה לא עונות על הדרישות של המעבדה ולכן הם לא יכולים להשתמש בהן. ישנו סייג בהגדרת המערכת כיוון ועל מנת להתקין תכנה על המחשבים שבמעבדה דרושים אישורים מיוחדים ממשרד הבריאות הכרוכים בבירוקרטיה שמבטיחה שהאפליקציה לא תאושר לעולם ולא תנוסה בפועל במעבדות.
- הפרויקט עוסק בתחום הרשתות, באבטחה, ממסד נתונים. Python, pycharm, sqlite3, tkinter, hashing, client server networking, threading.

פירוט תיאור המערכת:

- המערכת אמורה לספר ממשק לעבודה עם פרוטוקולים, ממשק להוספת ועריכת פרוטוקולים, ממשק מעקב אחר התקדמות העובדים, ממשק לעריכת העובדים ולצפייה בהיסטורית הפעילות במערכת.
- לסוג משתמש "עובד" המערכת תספק ממשק לעבודה עם פרוטוקולים וגישה לצפייה בנתונים השמורים בממסד הנתונים. למשתמש "מנהל" המערכת תספק את אותם דברים ובנוסף תאפשר מעקב אחר פעילות העובדים במערכת, עריכת נתוני עובדים, הוספת פרוטוקולים ועריכת פרוטוקולים קיימים.
- הבדיקות שיבוצעו על הפרויקט הן בדיקות רגילות של תקינות המערכת ה-GUI התקשורת והבסיס מידע.
- תכננתי את הזמן שלי לפי אילוצים שונים, וסידרתי את חלקי המערכת בדר עבודה הגיוני כך שאזכור את כל חלקי המערכת ולא אלך לאיבוד.
חלקי המערכת והסדר בו עבדתי עליהם:
 - יצירת שלד ל-database
 - התחברות למערכת
 - דף בית
 - עריכת משתמשים
 - ביצוע בדיקה
 - עריכת בדיקה
 - צפיה בהיסטוריה
 - צפיה בהתקדמות העובדים
 - הצטרפות למערכת

- תקשורת מרובת משתמשים
- ניהול ונרמול database
- הצפנת RSA
- הסיכונים בפרויקט:
 - משתמש האקר שמוריד את הקוד של האפליקציה יכול לנסות לשחק עם הקוד ולבקש מהשרת מידע שאין לו הרשאה לראות. על מנת להתמודד עם זאת, בקוד של השרת ישנה שמירה של הרשאות לכל socket מחובר כך שsocket שלא הזדהה באמצעות שם וסיסמה או שם וקוד כניסה נכונים לא יוכל לקבל שום מידע מהdatabase.
 - האקר שמסניף את הפקטות שמועברות בין השרת ללקוחות שלו עשוי לגלות את מפתחות ההצפנה ובכך לקבל גישה למידע שאין לו הרשאה לראות. על מנת למנוע את זה, בעת העלאת התוכנה וברגע 2 הלחיצות הראשונות של המשתמש על המסך התכנה שומרת חותמת זמן (חייבות להיות לפחות 3 לחיצות לפני שהוא יוכל לפנות לשרת) ומשתמשת בכמות הזמן שלקח למשתמש עד כל לחיצה כדי ליצור 2 משתנים אקראיים בלתי ניתנים לחיזוי, הקטנים ממיליון, ואז בוחרת שני מספרים מתוך קובץ של מיליון מספרים ראשוניים הגדולים ממיליון ואיתם מייצרת את מפתחות ההצפנת RSA. במקביל, השרת שומר את כמות הזמן שעברה מאז התחברות הלקוח האחרון והתחברות אותו לקוח איתו ישנה תקשורת כעט בפעם האחרונה ומייצר בעזרתם מפתחות הצפנה באותה דרך.
 - האקר עשוי לעבור על כל הרשימה של מיליון המספרים הראשוניים ולנסות כל זוג אפשרי מתוכה ובכך לגלות את מפתחות ההצפנה. כדי להתמודד עם כך, בכל חזרה לדף הבית בתכנה ישנה החלפה של מפתחות ההצפנה בלקוח לפי זמני הלחיצות של המשתמש ובשרת לפי התוכן של ההודעות הקודמות שקיבל. כך, עד שיגיע ההאקר למפתחות הנכונים, הם לא יהיו רלוונטיים יותר.
 - האקר עשוי להתחבר לתכנה ללא הרשאה לכך. על מנת להתמודד עם זה, מנהל המעבדה צריך להוסיף מראש כל עובד חדש במעבדה לרשימת העובדים ולכתוב את כתובת המייל שלו, כך שכאשר המשתמש ינסה להצטרף לתכנה הוא יצטרך לספק את שמו ואז את קוד הכניסה שישלך אליו למייל.
 - האקר עשוי לפנות למערכת בעזרת אלפי בקשות על מנת לגרום לשרת לקרוס. על מנת להתמודד עם כך, השרת דוחה ניסיונות לחיבור אליו כאשר יש לו מעל 1000 בקשות. בהינתן שבמעבדה המטבולית לשמה האפליקציה מיועדת ישנם עובדים אחדים, דבר זה

יוודא שהשרת לא יקרוס וימשיך לרוץ כרגיל גם בעת מתקפה, ובסיומה יוכלו העובדים להתחבר ללא בעיה.

תיאור תחום הידע- פרק מילולי

פירוט מעמיק של היכולות שהוצגו בשלב הקודם ומעבר

היכולות בצד שרת:

קליטת משתמשים רבים במקביל, אימות של זהות המשתמשים, קבלת הודעות מהמשתמשים, קריאה וכתובה מבסיס הנתונים עבור משתמשים מאומתים, שליחת הודעות למשתמשים, שליחת מייל עם קוד אימות למשתמשים שמצטרפים למערכת, אימות הקוד והחלפת מפתחות הצפנת RSA עם כל לקוח.

- קליטת משתמשים ואימות זהותם:

השרת קולט משתמשים, ומאמת שהם נמצאים במערכת עם שם וסיסמה. אם אינם במערכת הוא מאמת משתמשים חדשים בעזרת קוד אימות שנשלח למייל. תוקפו של הקוד פג לאחר 2 דקות. הסיסמה של המשתמש או קוד ההתחברות שלו נשלחים עם האש 5MD. ברגע שמשתמש מזדהה באחת הדרכים הללו מוענקת לו הרשאת גישה מתאימה לפי תפקידו- עובד/מנהל. אוסף יכולות נדרשות: בסיס נתונים, בדיקת תקינות, הצפנה, פענוח.

- הצפנה:

השרת מצפין באופן מלא את התקשורת בינו לבין הלקוח. ההצפנה היא הצפנת RSA. עבור כל לקוח השרת יוצר זוג מפתחות ייעודי. יצירת המפתחות מתבצעת בעזרת אלגוריתם RSA שמקבל 2 מספרים ראשוניים הגדולים ממיליון שנבחרים באופן בלתי ניתן לצפייה מראש- בעזרת משתנים מספריים בלתי צפויים שנבנים מזמני ההתחברות של משתמשים. השרת מעביר לכל לקוח את המפתח הציבורי שנוצר עבורו ומקבל ממנו את המפתח הציבורי שלו. אוסף יכולות נדרשות: הצפנה, פענוח, תקשורת מרובת משתמשים.

- ניהול לקוחות מרובים:

-השרת משתמש בספריית select על מנת לנהל בו זמנית משתמשים מרובים.
 בכל ריצה של השרת הוא בודק האם לקוח מנסה להתחבר, להתנתק, לשלוח הודעה או לקבל הודעה ופועל בהתאם. השרת מגביל את כמות המשתמשים לקצת יותר מכמות הלקוחות הייעודיים שיכולים להיות למערכת על מנת למנוע קריסה של השרת עקב מתקפה.

אוסף היכולות הנדרשות:

שרת מרובה משתמשים.

אובייקטים:

לקוחות, socket

- ניהול בסיס הנתונים:

השרת מריץ שאילות על בסיס הנתונים, הוא יכול לעדכן, להוסיף להסיר ולקרוא את בסיס הנתונים לבקשת הלקוחות המורשים, ולשלוח להם את המידע שביקשו/ אישור על כך שהפעולה שביקשו לבצע התבצעה.

אוסף יכולות נדרשות:

קריאה וכתיבה מבסיס נתונים, ניהול שרת מרובה משתמשים, שמירת הרשאות של לקוחות.

אובייקטים: curser, שרת

- סידור פרוטוקול והפיכתו לקריא על ידי המערכת:

השרת מקבל תוכן של קובץ אקסל של פרוטוקול קיים, שולח אותו למודל שפה ומבקש סידור שלו לפי סוג השורות ופורמט אחיד, ומחזיר ללקוח.

אוסף יכולות נדרשות:

ניהול לקוחות מרובים, גישה ל־api של מודל שפה.

אובייקטים: שרת, thread

היכולות בצד לקוח:

- הכנסות למערכת:

כניסה למערכת כמשתמש קיים בעזרת שם וסיסמה שנשלחות לשרת המאמת את זהות המשתמש. הסיסמה לאחר מעבר פונקציית האש וההודעה כולה מוצפנת.

אוסף יכולות נדרשות:

תקשורת עם שרת מרכזי, ממשק משתמש, הצפנה, פענוח, תקשורת, בסיס נתונים.

אובייקטים:

- לקוח, שדות טקסט, תוויות, אובייקט תקשורת עם השרת, שרת, בסיס נתונים

• הצטרפות למערכת (לאחר רישום על ידי מנהל)

כניסה למערכת דרך הכנסת שם משתמש רשום ולאחר מכן קוד כניסה שנשלח למייל, ויצירת סיסמה קבועה.

אוסף יכולות נדרשות:

תקשורת עם שרת מרכזי, בדיקת תקינות, הצפנה, פענוח, ממשק משתמש, שליחת מייל ללקוח.

אובייקטים:

לקוח, שדות טקסט, אובייקט תקשורת עם השרת, שרת, בסיס נתונים

• שימוש בפרוטוקול:

כניסה לפרוטוקול קיים במערכת ושמירה של תיעוד ביצוע הבדיקה.

אוסף יכולות נדרשות:

ממשק משתמש, תקשורת עם שרת, הצפנה, פענוח, שליחת וקבלת מידע.

אובייקטים:

פרוטוקול, checkbox, תוויות, כפתורים, אובייקט תקשורת עם השרת, שרת, בסיס נתונים

• עריכת עובדים:

משתמש מנהל יכול לערוך את רשימת עובדי המעבדה והרשאותיהם.

אוסף יכולות נדרשות:

ממשק משתמש, תקשורת עם שרת, הצפנה, פענוח, הרשאות משתמש.

אובייקטים:

עובדים, כפתורים, תפריטים, שדות טקסט, אובייקט תקשורת עם השרת, שרת, בסיס נתונים

• צפייה בסטטוס העבודה של עובדים:

משתמש מנהל יכול לצפות בזמן אמת על איזה פרוטוקול כל אחד מהעובדים עובד.

אוסף יכולות נדרשות:

ממשק משתמש, תקשורת עם שרת, הצפנה, פענוח, הרשאות משתמש.

אובייקטים:

טבלה, עובדים, אובייקט תקשורת עם השרת, שרת, בסיס נתונים

• צפייה בהיסטוריה:

צפייה בתיעוד של איזו בדיקה כל עובד ביצע מתי ומה היו ההערות שלו. ניתן לסנן את היסטוריה בעזרת עד 3 מסננים לפי שם העובד, שם הבדיקה, והתאריך. על ידי לחיצה על אחת השורות ניתן להתבונן בטופס הבדיקה, הכולל את ההערות שנרשמו עליה. ממשיך משתמש, תקשורת עם שרת, הצפנה, פענוח.

אובייקטים:

טבלאות, תפריטים, שדות טקסט, מסכים, פרוטוקולים, אובייקט תקשורת עם השרת, שרת, בסיס נתונים

- עריכת פרוטוקול קיים:

שינוי של מבנה הפרוטוקולים הקיימים במערכת ושמירת גרסה חדשה שלהם.

אוסף יכולות נדרשות:

ממשק משתמש, תקשורת עם שרת, הצפנה, פענוח, איסוף מידע מממשק המשתמש. אובייקטים:

כפתורים, שדות טקסט, תפריטים, שורות, פרוטוקול, אובייקט תקשורת עם השרת

- מחיקת פרוטוקולים:

הסרת פרוטוקולים ממסד הנתונים ומממשק המשתמש.

אוסף יכולות נדרשות:

ממשק משתמש, תקשורת עם שרת, הצפנה, פענוח, הרשאות משתמש.

אובייקטים:

פרוטוקולים, כפתורים, מסכי הודעה (win32ui MessageBox), אובייקט תקשורת עם השרת

- הוספת פרוטוקולים חדשים:

ניתן ליצור פרוטוקולים חדשים מאפס, או להעלות פרוטוקולים ישנים לתוכנה מקובץ אקסל/ וורד קיים. הקבצים נקראים והמידע נשלח לול אשר מסדר אותו, מקטלג כל שורה לפי משמעותה ומחזיר רשימה בפורמט אחיד.

אוסף יכולות נדרשות:

ממשק משתמש, תקשורת עם שרת, הצפנה, פענוח, הרשאות משתמש, קריאה מקובץ

אקסל/ וורד, שימוש ב- api של בינה מלאכותית.

אובייקטים:

קבצים, פרוטוקול, כפתורים, מסך העריכה, אובייקט תקשורת עם השרת, שרת, בסיס

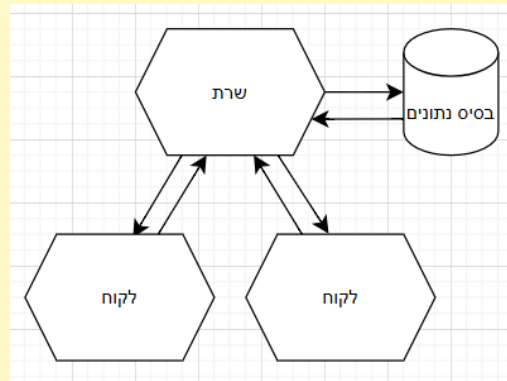
נתונים

מבנה / ארכיטקטורה של הפרויקט

תיאור הארכיטקטורה של המערכת המוצעת

פירוט ההחלטות שנלקחו למימוש המערכת:

- החומרה במערכת: שרת, אשר ישמש הלפטופ שלי, ולקוחות, שישמשו המחשבים בבית הספר. הלקוחות יתחברו לשרת אשר יקשיב באותו פורט.

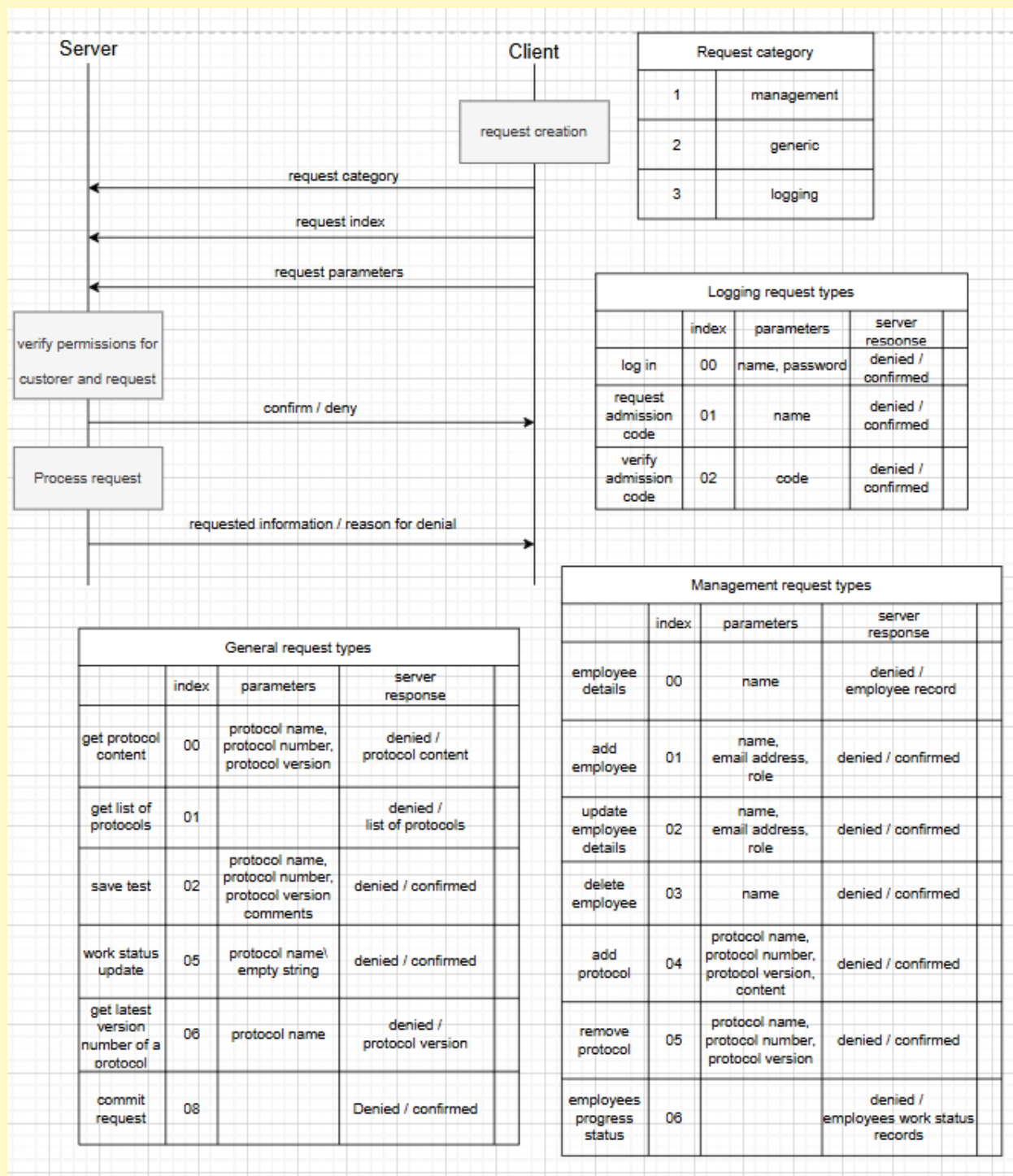


תיאור הטכנולוגיה הרלוונטית:

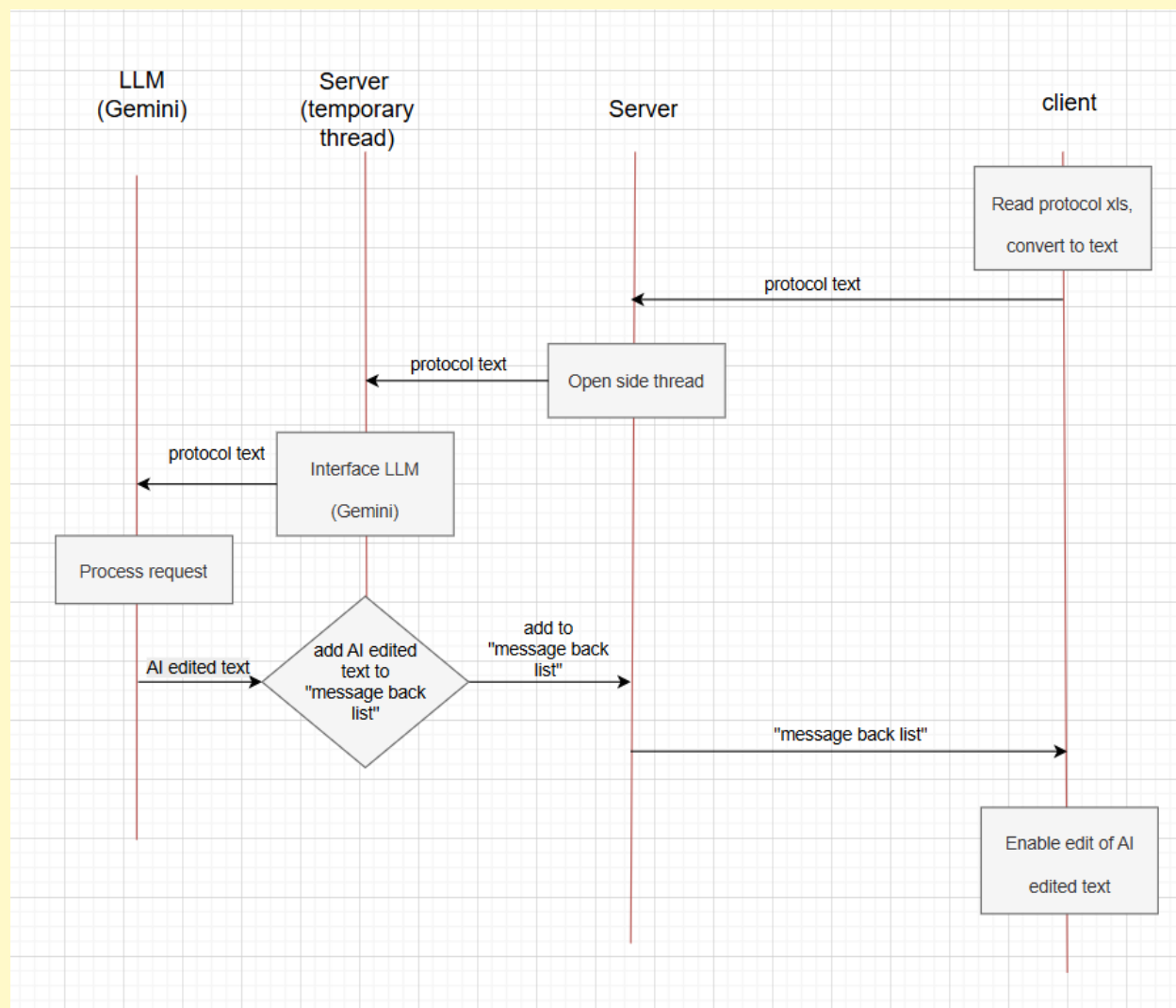
- שפת פיתון, תקשורת, ניהול ממסד נתונים גדול, הצפנה, ממשק משתמש, ממשק בינה מלאכותית, מערכת הפעלה windows.
- בחרתי לתכנת בשפת פיתון, כיוון והיא מאפשרת לי לעבוד בצורה נוחה ולהיעזר בספריות חיצוניות.
- בפרויקט השתמשתי בתקשורת שרת לקוח בעזרת socket בה ישנו מחשב אחד המשמש כשרת ומחשבי הלקוחות מתקשרים אתו. השרת רץ בלופ אינסופי ומגביל חיבור של עד 20 משתמשים- כיוון שיכולים להיות עד כ-10 עובדים במעבדה זה לא יהווה בעיה. הלקוחות שולחים לשרת בקשות והוא עונה להם לפי בסיס הנתונים שלו וההרשאות שבידם.
- ניהול ממסד נתונים גדול: השתמשתי ב-sqlite3 על מנת לנהל את ממסד הנתונים בפרויקט אשר שמור על מחשב השרת. בממסד הנתונים שלי ישנן 3 טבלאות, אשר כולן מקושרות אחת לשנייה. אחד עבור רשימת העובדים, אחד עבור רשימת הפרוטוקולים ואחד עבור היסטורית העבודה.

תיאור זרימת המידע במערכת:

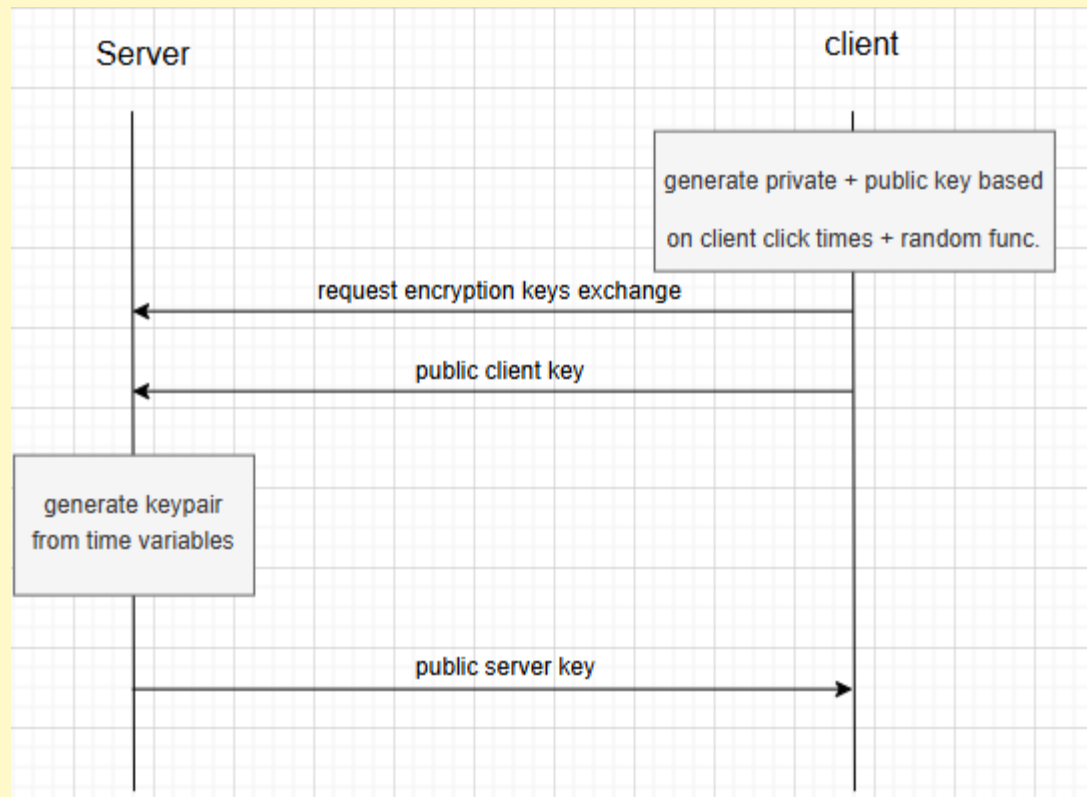
- זרימת המידע במערכת פועלת בדרך הבאה:



עבור היכולת של הפיכת תוכן פרוטוקול מקובץ לקריא:



עבור היכולת של החלפת מפתחות:



תיאור האלגוריתמים המרכזיים בפרויקט:

- אלגוריתם מרכזי בפרויקט הוא אלגוריתם ריצת השרת.
 - השרת צריך לנהל מספר משתמשים במקביל, תוך שירות כל אחד מהם באופן מאובטח ומוצפן, כאשר ישנן לפעמים בקשות מסובכות, ויש צורך לאמת את זהות המשתמשים
 - ישנם אלגוריתמים רבים שמטפלים בבעיה זו. אני הסתמכתי על האלגוריתם הבסיסי של ספר גבהים רשתות לבניית ריצת השרת שלי.
 - הפתרון בו בחרתי עבור האפליקציה שלי, הוא פתרון בו האבטחה מקבלת עדיפות גבוהה יותר מהמהירות, מכיוון ובמערכת יכולים להיות רק משתמשים מועטים מאוד, ואבטחת המידע היא חשובה מאוד כאשר האפליקציה מיועדת לגוף ממשלתי. אלא אם ישנה בקשה עבור מודל השפה, שזה תהליך שעלול לקחת מספר שניות, השרת אינו משתמש בthreading אלא עובד בתהליך אחד עבור כל הבקשות. השרת עובר בעזרת מודול select על כל sockets, מחבר את אלה שמבקשים להתחבר אם יש מקום (אם לא, הוא שולח הודעה מתאימה), מקבל הודעות ממי שיש ושולח הודעות למי שצריך בכל ריצה של הלולאה. בתוך פעולות אלו השרת מצפין ומפענח הודעות, יש לו זוג מפתחות

עבור כל אחד מהלקוחות שלו על מנת למקסם את האבטחה. השרת ישרת רק לפי הרשאות הגישה השמורות במילון עבור כל socket.

- אלגוריתם מרכזי בפרויקט הוא אלגוריתם הצגת הפרוטוקולים. הפרוטוקולים במערכת נשמרים כך שבתחילת כל שורה ישנו header מסוים- "!!", "..", "!", "??". כל אחד מהם מסמל על איזה סוג של שורה מדובר. בעת הצגת פרוטוקול התוכנה עוברת על כל שורה ומציגה אותה לפי header שלה.
- אלגוריתם מרכזי בפרויקט הוא אלגוריתם הצפייה בהיסטורית הפעילות במערכת. אלגוריתם זה מחולק לשלוש מחלקות- ViewData, Data, ו-Filter. המחלקה פילטר מגדירה אובייקט המכיל שני תפריטי אפשרויות, אחד עבור קטגורית הסינון ואחד עבור הערך. המחלקה Data מגדירה אובייקט מסוג CTkTabView, עליו ישנן כל הטבלאות המציגות את ההיסטוריה. בכל רגע נתון ישנם 6 כפתורים על אובייקט זה, כפתור "previous", כפתור "next" ו-4 כפתורי עמודים. כאשר לוחצים על הבא העמודים יוחלפו לארבעת העמודים הבאים ולהיפך. לאחר שעמוד מסוים נבנה, הטבלה שעליו נשמרת כך שאם תצא מעמוד מסוים או תלחץ על הבא או קודם, ואז תחזור אליו היא לא תצטרך להטען מחדש. המחלקה ViewData מגשרת בין השתיים הקודמות, היא מגדירה את העמוד עצמו של הצפייה בהיסטוריה ובה יש את האלגוריתם אשר אוסף את המסננים מכל האובייקטים של הפילטר, שולחת בקשה לשרת לקבל את המידע, ונותנת אותו לData לבנות את העמודים.
- אלגוריתם מרכזי בפרויקט הוא אלגוריתם עריכת הפרוטוקולים. אלגוריתם זה מאפשר ללקוח גם לערוך את תוכן כל שורה בפרוטוקול, דרך שימוש באובייקט אשר הוא כפתור שכשלווחצים עליו הופך לשדה טקסט, וכשיוצאים ממנו חוזר להיות כפתור. ניתן לשנות את סוג השורה בעזרת תפריט אפשרויות שמציג את כל סוגי השורות הנתונים. ניתן להוסיף או להסיר שורות. ניתן לשנות בחופשיות את המקום של כל אחד מהשורות.
- אלגוריתם מרכזי נוסף בפרויקט הוא אלגוריתם ה-RSA.
 - יש לבחור 2 מספרים ראשוניים
 - יש להכפיל אותם וזה יקרא המודולו-N.
 - יש למצוא את כמות המספרים הראשוניים ביחס ל-N לפי מתמטיקה מודולרית- יקרא פי של N.

- יש להסיר מרשימת המספרים הראשוניים ביחס ל-N את אלה שיש להם מחלק משותף עם N, והם יקראו הקו-פריים של N.
- הנוסחא: $(p-1)*(q-1)$. כאשר ק ו- q הם הראשוניים.
- המפתח הציבורי יהיה מספר גדול מאחד וקטן מפי של N, ראשוני ביחס ל- N ולפי של N.
- הנוסחא לחישוב המפתח הפרטי, d, תהיה $d * e \pmod{f} = 1$ כאשר f הוא פי של N.
- על מנת להצפין ישנו האלגוריתם הבא:
- מוצאים את ערך הascii של כל תו בהודעה.
- מעלים אותו בחזקת המפתח הציבורי מודולו N.
- על מנת לפענח, ישנו האלגוריתם הבא:
- מעלים כל מספר בהודעה בחזקת מפתח הפיענוח, מודולו N.

תיאור סביבת הפיתוח:

- סביבת הפיתוח איתה בחרתי לעבוד היא סביבת Pycharm.
- כלי הפיתוח איתם עבדתי הם השפה python, והספריות הבאות:

- socket
- threading
- json
- time
- random
- select
- smtplib
- hashlib
- customtkinter
- PIL
- win32ui
- functools
- pandas
- datetime
- re

- sqlite3
- google
- dotenv
- email.message

תיאור פרוטוקול התקשורת:

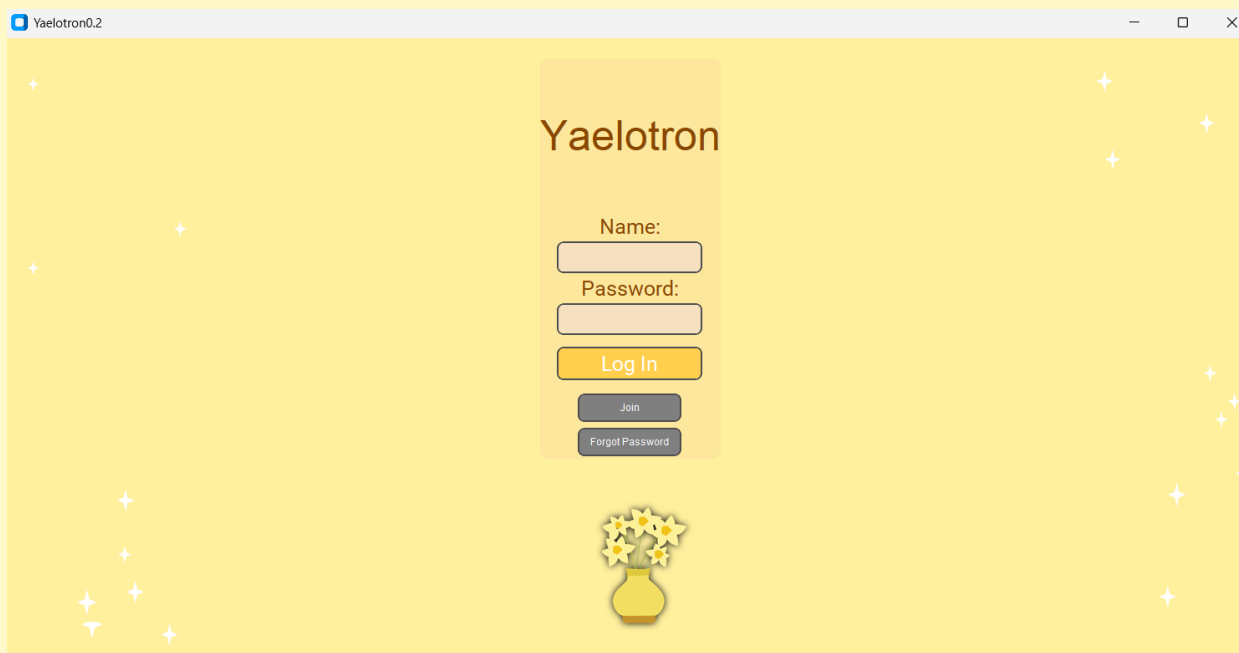
- מבנה ההודעות אשר נשלחות מהלקוח לשרת הוא:
 - שם הקטגוריה של הבקשה (בקשה גנרית, בקשה מנהלית או בקשת התחברות)
 - "||"
 - האינדקס של הבקשה מתוך רשימת הבקשות מאותה קטגוריה (מספר דו ספרתי)
 - "||"
 - כל הפרמטרים אשר יש להעביר לאותה בקשה, מופרדים בעזרת התו "|"
 - ההודעות אשר נשלחות מהשרת ללקוח עבור בקשות של נתינת מידע הן או "Confirmed" או "Denied" או הודעה המפרטת למה הפעולה לא מתבצעת.
 - ההודעות אשר נשלחות מהשרת ללקוח עבור בקשות של לקיחת מידע הן או "Denied" אם ללקוח אין הרשאה לגשת למידע הזה, או המידע כטקסט, או אם המידע הוא רשימה אז את הרשימה בפורמט json.
- ההודעות הנשלחות במערכת:
 - (1) החלפת מפתחות הצפנה:
 - נשלחת מהלקוח לשרת
 - ההודעה מתחילה במילה |key ומלווה במפתח rsa הציבורי של הלקוח
 - השרת עונה ללקוח את המפתח הציבורי שלו.
 - (2) התחברות למערכת:
 - נשלחת מהלקוח לשרת
 - log|00|name|password
 - השרת עונה Confirmed או Denied
 - (3) הצטרפות למערכת:
 - נשלחת מהלקוח לשרת
 - log|01|name
 - השרת עונה Confirmed או Denied
 - (4) אימות קוד ההצטרפות למערכת
 - נשלחת מהלקוח לשרת

- log|02|code
- השרת עונה Confirmed או Denied
- (5 בקשת נתוני העובדים
- נשלחת מהלקוח לשרת
- man|00
- השרת מחזיר את הרשימה או Denied
- (6 בקשה להוסיף עובד לרשימה
- נשלחת מהלקוח לשרת
- man|01|name|email_address|role
- השרת עונה Confirmed או Denied
- (7 בקשה לעדכן פרטים של עובד
- נשלחת מהלקוח לשרת
- man|02|name|email_address|role
- השרת עונה Confirmed או Denied
- (8 בקשה להסיר עובד מהרשימה
- נשלחת מהלקוח לשרת
- man|03|name
- השרת עונה Confirmed או Denied
- (9 בקשה להוסיף פרוטוקול למערכת
- נשלחת מהלקוח לשרת
- man|04|protocol_name|protocol_number|protocol_version|content_of_protocol
- השרת עונה Confirmed או Denied
- (10 בקשה להסיר פרוטוקול מהמערכת
- נשלחת מהלקוח לשרת
- man|05|protocol_name|protocol_number|protocol_version
- השרת עונה Confirmed או Denied
- (11 בקשה לצפות בסטטוס העבודה של העובדים
- נשלחת מהלקוח לשרת
- man|06
- השרת מחזיר רשימה שמפרטת את סטטוס העבודה של העובדים או Denied
- (12 בקשה לקבלת תוכן של פרוטוקול

- נשלחת מהלקוח לשרת
- gen|00|protocol_name|protocol_number|protocol_version
- השרת מחזיר את התוכן או Denied
- (13) בקשה לקבל את הרשימה של הפרוטוקולים (שם וגרסה)
- נשלחת מהלקוח לשרת
- gen|01
- השרת מחזיר את הרשימה או Denied
- (14) בקשה לשמור ביצוע של בדיקה
- נשלחת מהלקוח לשרת
- gen|02|protocol_name|protocol_number|protocol_version
- השרת עונה Confirmed או Denied
- (15) בקשה לעדכן את סטטוס העבודה
- נשלחת מהלקוח לשרת
- gen|05|protocol_name/empty_string
- השרת עונה Confirmed או Denied
- (16) בקשה לקבל את מספר הגרסה האחרונה של פרוטוקול מסויים
- נשלחת מהלקוח לשרת
- gen|06|protocol_name
- השרת מחזיר את המספר או Denied
- (17) בקשה לקבל את ההערות שנכתבו על בדיקה מסוימת שבוצעה.
- נשלחת מהלקוח לשרת
- gen|07|line_id
- השרת מחזיר את ההערות או Denied
- (18) בקשה לבצע commit
- נשלחת מהלקוח לשרת
- gen|08
- השרת עונה Confirmed או Denied

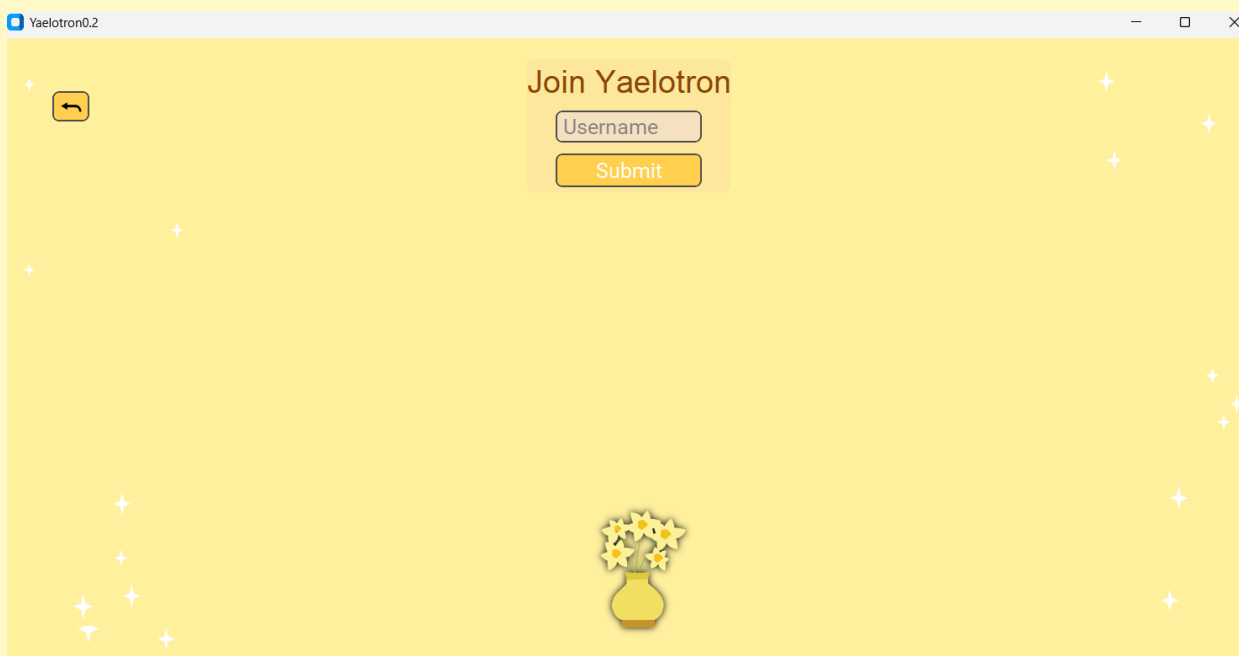
תיאור מסכי המערכת:

- מסך התחברות ראשוני:
- מכיל אפשרות להכניס שם וסיסמה, או לעבור למסך ההרשמה.



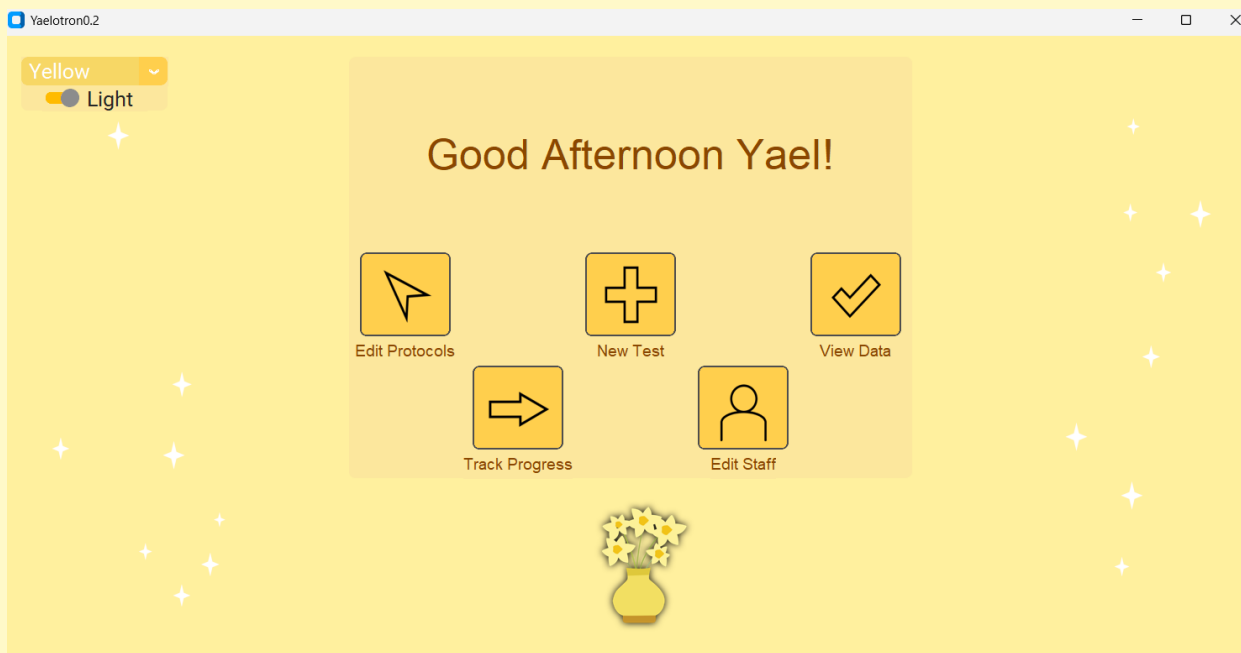
- מסך הרשמה:

מכיל הכנסת שם משתמש, לאחר מכן קוד כניסה ולאחר מכן יצירת סיסמה.



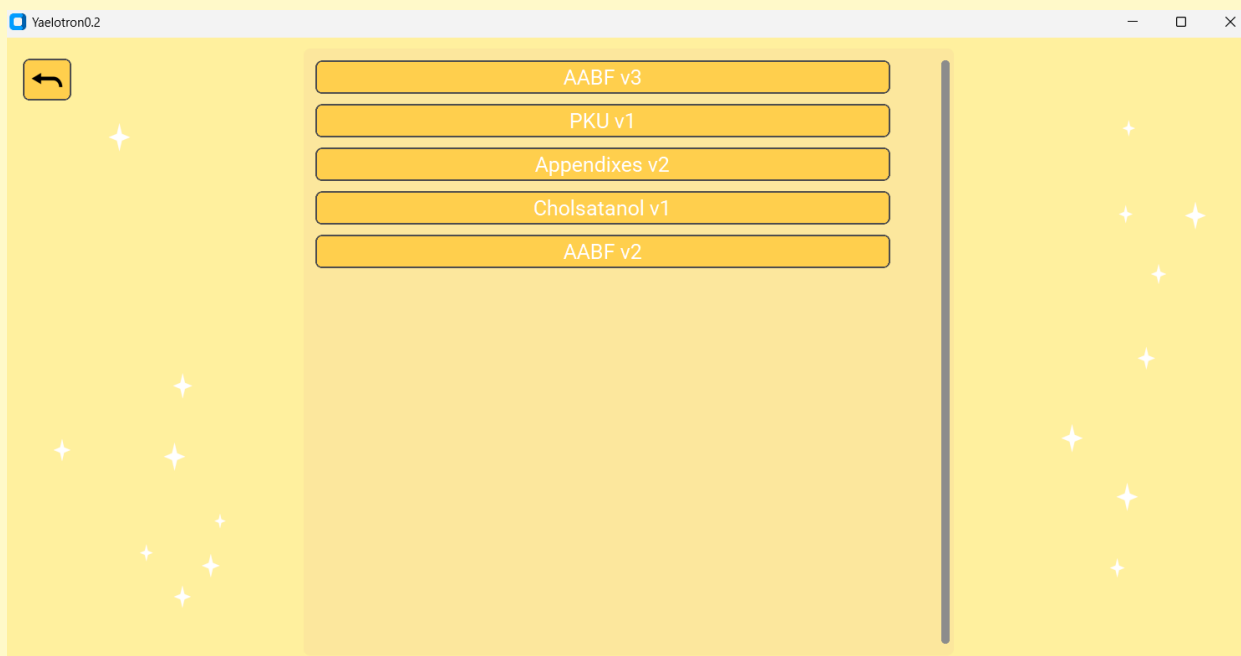
- מסך הבית:

מכיל כפתור מעבר לעמוד ההיסטוריה וכפתור מעבר לעמוד העבודה על הפרוטוקולים אם המשתמש הוא עובד, ואם לא מסך זה מכיל גם כפתורי מעבר לעמודי עריכת עובדים, עריכת פרוטוקולים וצפייה בסטטוס העבודה של העובדים.



- מסך בחירת פרוטוקול לעבודה:

מסך בו נמצאת רשימת הפרוטוקולים המוצאים.



- מסך העבודה על פרוטוקול:

תוכן הפרוטוקול הנבחר מוצג לעבודה נוחה.

PKU טופס הכנת דגימות

1 ☐ מהמקפיא בחדר QC 122 הוצאת כרטיסי

2 ☐ LOT:

שלב 1 פלטה מספר 1

3 ☐ בדיקת הכרטיסים וסידורם 5-92

* על כל שתי כרטיס שתי מדבקות ברקוד

4 ☐ ניקוב הכרטיסים במנקב לפי הסדר בשלב 2

5 ☐ פלטת ניקוב בחדר 119 במגירות ליד המקרר הפנימי

6 ☐ בריכוז IS 1:1000 הכנת

בשני שלבים IS הכנת תמיסת

7 ☐ 100ul and 4.9ml 100% Methanol

- מסך הצפייה בסטטוס העבודה של העובדים:

מוצגת טבלה של כל העובדים ושם הפרוטוקול אשר כל עובד עובד עליו באותו רגע.

Employee:	Working On:
Yael	
Eyal	
z	PKU
Tamar	
test	AABF
nufar	
Idan	
Yuli	
Ben	
Liya	
self	
t	

Reload

- מסך עריכת המשתמשים:

מסך בו מוצגת טבלה ובה כל עובד, כתובת המייל שלו והתפקיד שלו (עובד/מנהל), בו ניתן להסיר/להוסיף עובדים ולשנות את כתובות המייל/התפקיד של עובדים.

The screenshot shows a web application window titled "Yaelotron0.2". It features a table with three columns: "Name:", "Gmail:", and "Title:". The table contains 12 rows of user data. Each row has a trash icon to its right. Below the table is a "+" button and a "Save" button. The background is yellow with white star-like sparkles.

Name:	Gmail:	Title:
Yael	dana.kutz19@gmail.com	Unit Manager
Eyal	eyal.kutz@gmail.com	Worker
z		Unit Manager
Tamar	tamarihof@gmail.com	Unit Manager
test	dana.kutz19@gmail.com	Worker
nufar	amiaznuf@gmail.com	Worker
Idan	idanfeldberg@gmail.com	Worker
Yuli	yoliadini@gmail.com	Lab Manager
Ben	ben@bishko.net	Worker
Liya	liyaalmog28@gmail.com	Unit Manager
self	dnhqwrg@gmail.com	Worker

- מסך בחירת פרוטוקול לעריכה:

מסך בו מוצגת רשימת הפרוטוקולים, וניתן לבחור למחוק או לערוך את אחד מהם, או להוסיף פרוטוקול חדש. אם נבחר פרוטוקול חדש מוצגות שתי אפשרויות- יצירת פרוטוקול מאפס או מקובץ קיים. אם קובץ קיים נבחר ניתן לבחור מאיזה גיליון ליצור את הפרוטוקול.

The screenshot shows the same "Yaelotron0.2" application window. It displays a list of protocols in a table-like format. Each entry has a trash icon to its right. Below the list is a "+" button. The background is yellow with white star-like sparkles.

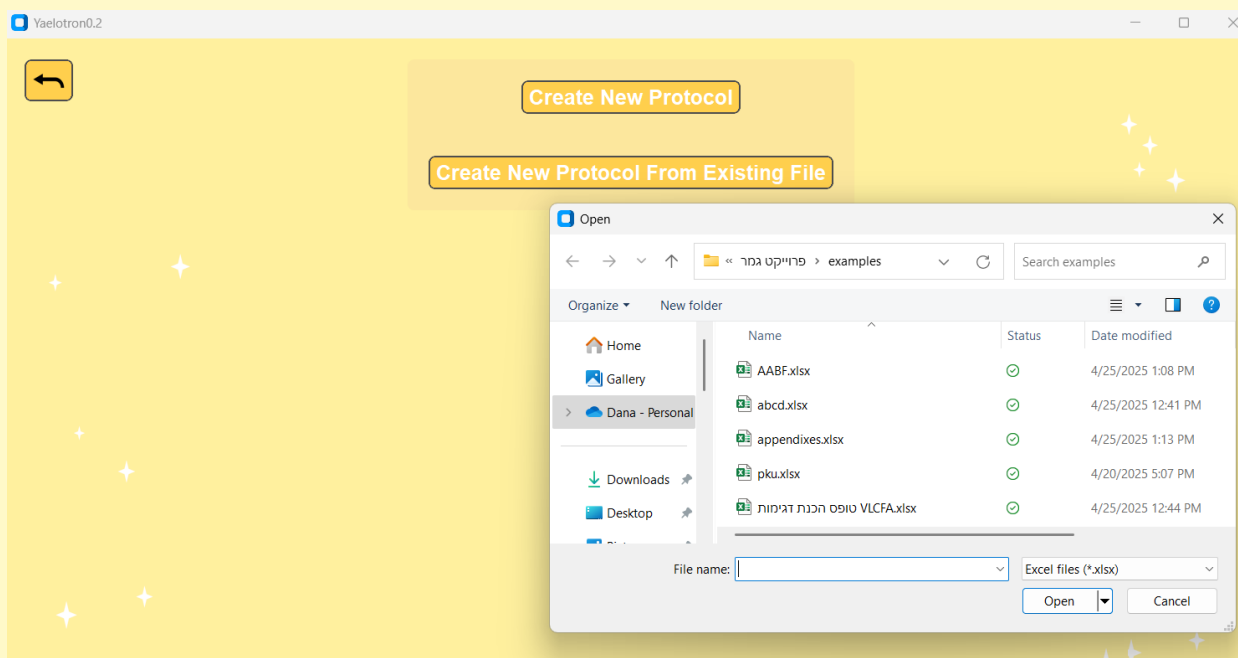
AABF v3
PKU v1
Appendixes v2
Cholsatanol v1
AABF v2

- מסך עריכת פרוטוקול:

מסך בו מוצג פרוטוקול קיים כך שניתן לשנות את תוכן השורות, סוג השורות, סדר השורות, להוסיף שורות או להסיר שורות. בנוסף ניתן לערוך את פרטי הפרוטוקול.

- מסך יצירת פרוטוקול חדש:

אותו דבר כמו עריכת פרוטוקול רק מתחיל ריק.



- מסך צפייה בהיסטוריה:

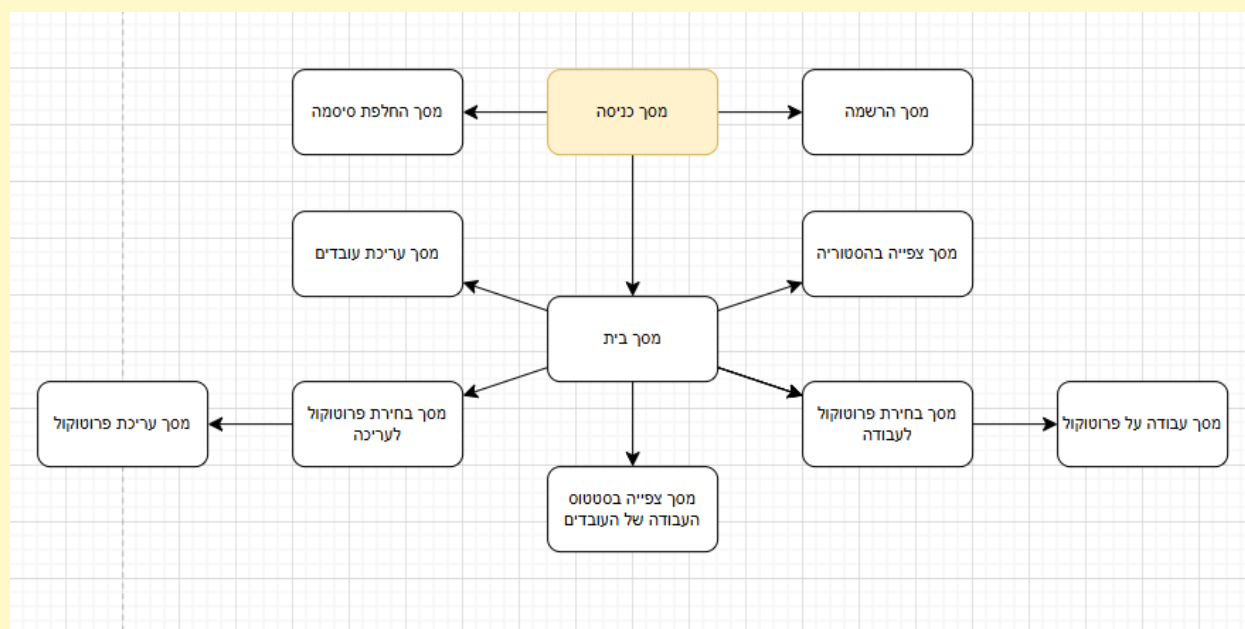
המסך מאפשר לצפות בכל היסטוריית הפעילות במערכת, או לחפש בהיסטוריה לפי מסננים ספציפיים. בנוסף כאשר לוחצים על שורה מסוימת בטבלה, יעלה על מסך נפרד תוכן הפרוטוקול הנבחר עם ההערות שנרשמו עליו בעת ביצועו.

Yaelotron0.2

Previous page1 **page2** page3 page4 Next

50	F-CHM00120501-13	AABF	3	z	2025-04-15 12:07:43
46	F-CHM00120501-13	AABF	3	z	2025-04-07 20:05:38
42	F-CHM00120501-13	AABF	4	z	2025-03-21 07:02:04
37	F-CHM00120501-13	AABF	3	z	2025-03-20 08:03:48
33	F-CHM00120501-13	AABF	3	z	2025-03-02 18:34:49
31	F-CHM00120501-13	AABF	3	z	2025-02-28 12:05:24
30	F-CHM00120501-13	AABF	3	z	2025-02-28 11:59:46
23	F-CHM00120501-13	AABF	3	z	2025-02-26 09:58:24
18	F-CHM00120501-13	AABF	3	z	2025-02-24 12:58:20
12	F-CHM00120501-13	AABF	4	z	2025-02-23 13:03:40
11	F-CHM00120501-13	AABF	4	z	2025-02-23 12:58:54
10	F-CHM00120501-13	AABF	4	z	2025-02-23 12:40:41

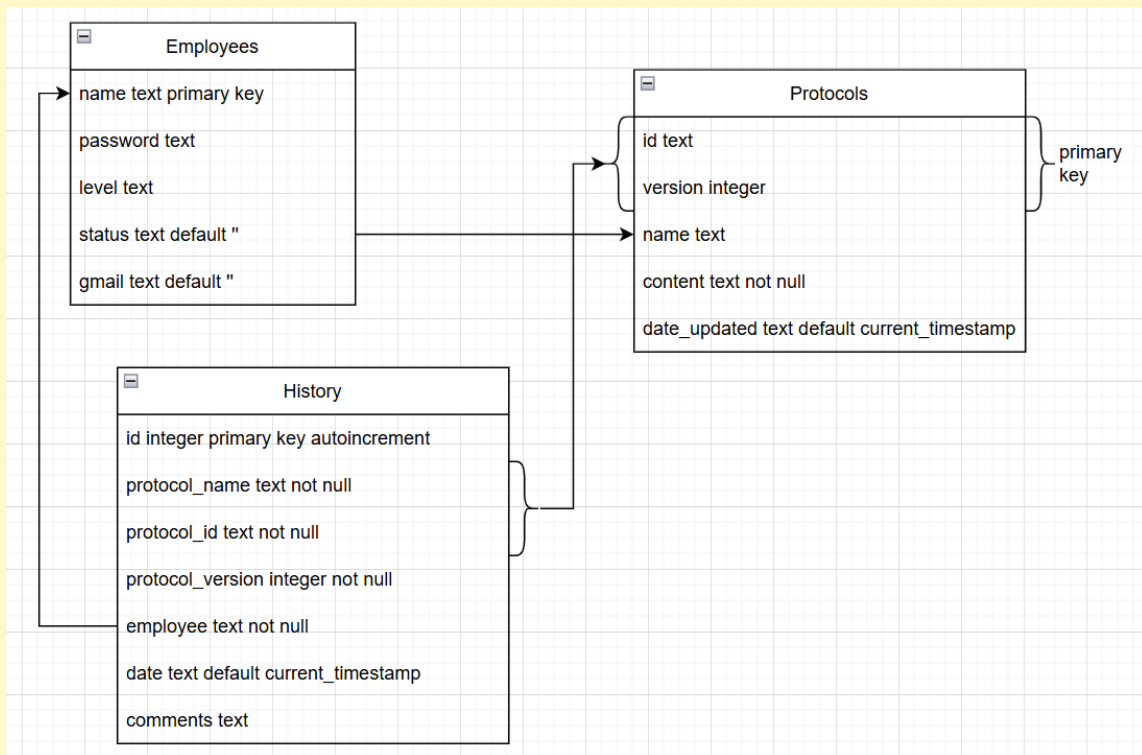
Add Filter Employee Protocol
apply z AABF



תיאור מבני הנתונים:

- הפרויקט מכיל ממסד נתונים, קובץ טקסט המכיל מספרים ראשוניים, קבצי תמונות וקובץ טקסט של הנחיות הפורמט עבור בקשות למודל השפה.
- בבסיס הנתונים ישנן 3 טבלאות: טבלה השומרת פרוטוקולים, טבלה לעובדים וטבלה להיסטורית הפעילות במערכת. השדות והקשרים בין הטבלאות מפורטים בתרשים.
- בטבלת העובדים 5 שדות:
- שם העובד

- סיסמה
- רמה- תפקיד, מנהל\ עובד
- סטטוס- על איזה פרוטוקול הוא עובד כרגע, אם הוא עובד על משהו.
- כתובת מייל
- בטבלת הפרוטוקולים 5 שורות:
 - id- המספר הסידורי שלו
 - גרסה- מספר הגרסה
 - שם
 - תוכן- כל תוכן הפרוטוקול, כטקסט
 - תאריך יצירה
- בטבלת ההיסטוריה 7 שדות:
 - id- מספר השורה בטבלה
 - שם- שם הפרוטוקול שנעשה
 - protocol_id- המספר הסידורי של הפרוטוקול שנעשה
 - גרסת הפרוטוקול
 - שם העובד שביצע אותה
 - תאריך ושעת הביצוע
 - הערות שנכתבו עליו



סקירת חולשות ואיומים:

- שכבת האפליקציה:

- SQL injection - בפרויקט ישנה הגנה מפני מתקפה זו, המידע שמתקבל מהלקוח לא נכנס ישירות לשאילתה אלא עובר כפרמטר כצורה מסודרת שמונעת ממנו להיות מסוגל להוות קוד. בנוסף, כאשר יש הכנסה של פרמטר מהמשתמש לשאילתה, יש קודם כל בדיקה של תוכן הפרמטר.
- Hashing - במסד הנתונים הסיסמאות של המשתמשים שמורות בhash על מנת למנוע גנבה של סיסמאות של משתמשים אם ישנה פריצה לשרת.
- DDoS - על מנת למנוע קריסה של השרת במקרה של מתקפת DDoS ישנה הגבלת משתמשים במערכת של 20 משתמשים, כאשר אם משתמשים נוספים מנסים להתחבר השרת דוחה את בקשתם וסוגר אותם, כך שבצד לקוח יעלה מסך כשל התחברות.
- תהליך ההזדהות- עבור להיכנס למערכת ולקבל הרשאות גישה למידע במערכת יש קודם להזדהות באמצעות שם וסיסמה או שם וקוד כניסה. גם אז, המשתמש יוכל לקבל הרשאות בהתאם לתפקידו, ולא יוכל לבצע כל פעולה במערכת אם אין לו את כולן.

- RSA - כל הודעה הנשלחת במערכת (למעט שליחת מפתחות ההצפנה) נשלחת לאחר שעוברת הצפנה בעזרת המפתח הציבורי של הצד השני, ומפוענחת באותו צד בעזרת המפתח הפרטי. כך ההודעות נשמרות מאובטחות ככל הניתן.
- כל לקוח יוצר לעצמו זוג מפתחות, והשרת יוצר זוג מפתחות משלו עבור כל לקוח על מנת ליצר אבטחה מקסימלית. כדי לבחור מספרים ראשוניים גדולים מאוד בלתי צפויים בשביל מפתחות ההצפנה המערכת משתמשת בהפרשים בין זמני הלחיצות של המשתמש על המסך משולב עם פונקציית random רגילה.
- שכבת התעבורה:
- פרוטוקול TCP - הפרוטוקול הסטנדרטי עבור תקשורת מאובטחת, משתמש בלחיצת יד משולשת.
- שכבת הרשת: חלוקה לפקטות - חלק מההודעות שנשלחות בין הלקוח והשרת ארוכות, ולכן בעת שליחת הודעה המערכת קודם כל שולחת הודעה שמציינת את כמות הפעמים שיש על הצד השני לקבל 1024 ביטים, ואז את ההודעה בחלקים.

מימוש הפרויקט

פירוט מימוש המערכת - חלק א'

סקירת המודולים המיובאים:

- socket
- ליצור ולנהל חיבור ותקשורת בין הלקוחות לשרת.
- threading
- ביצוע פעולות במקביל וביעילות.
- json
- ייצוג רשימה בפורמט טקסט והיפוך התהליך כדי להעביר רשימות בתקשורת באופן חלק.
- time
- על מנת לייצר מספרים בלתי צפויים וליצור תוקף לקודי הכניסה למערכת.
- random
- לייצר מספרים רנדומליים מטווחים מסוימים וקביעת מיקומים רנדומליים.
- select
- ניהול לקוחות מרובים.
- smtplib

שליחת מיילים ללקוחות בשביל לוודא את זהותם.

- hashlib
- לשמור את סיסמאות המשתמשים בצורה מאובטחת בבסיס הנתונים.
- customtkinter
- ממשק משתמש גרפי נוח.
- PIL
- להעלות תמונות לממשק המשתמש.
- win32ui
- להעלות מסכי הודעה מסודרים וברורים למשתמשים.
- functools
- לתת לפונקציה פרמטרים קבועים בעת הגדרת פעולות לכפתורים בלולאה.
- pandas
- קריאת קובץ אקסל.
- datetime
- קבלת השעה הנוכחית ביום על מנת להציג הודעה מתאימה למשתמש.
- re
- לוודא שהסיסמה שהלקוח יצר מספיק מסובכת.
- sqlite3
- ניהול ממסד הנתונים.
- google
- שימוש ב-API של מודל שפה של גוגל - gemini
- dotenv
- שימוש בסיסמאות השמורות בקבצים סביבתיים על מנת שהם ישארו מחוץ ל-github.
- email.message
- סידור המייל ללקוח באופן אסטטי.

מודולים/מחלקות בפיתוח עצמי:

- App
- יורשת מ- customtkinter.CTk
- מנהלת את המסך הראשי של האפליקציה, את המשתנים שלה, המסכים והמעברים בין מסכים.

- **תכונות במחלקה:**

HEIGHT (1)

מגדיר את הגובה של חלון האפליקציה (600 פיקסלים).

WIDTH (2)

מגדיר את הרוחב של חלון האפליקציה (1200 פיקסלים).

client (3)

אובייקט מסוג Client המטפל בתקשורת עם השרת — שליחת וקבלת בקשות.

json (4)

טוען את הגדרות ערכת העיצוב (theme) מתוך קובץ JSON כדי להחיל עיצוב מותאם אישית.

light_mode (5)

מציין האם מצב התצוגה הנוכחי הוא מצב בהיר (True) או כהה (False).

vase (6)

אלמנט גרפי מסוג Label המציג תמונת אגרטל בעיצוב הממשק.

glitter (7)

רשימה של אלמנטים גרפיים של "ניצנוצים" שמפוזרים ברקע ליצירת אפקט עיצובי. הרשימה לא מנוהלת ישירות בקוד, אך משמשת לאחסון האלמנטים שנוצרים ב־

place_glitter.

username (8)

שומר את שם המשתמש המחובר כרגע.

user_permission (9)

מציין את רמת ההרשאות של המשתמש (למשל admin, viewer: וכו').

gif(10)

אלמנט המציג אנימציה טעינה (GIF) במהלך פעולות ארוכות כמו יצירת פרטוקול חדש.

primary_buttons(11)

רשימה של פונקציות שמייצגות את הכפתורים העיקריים בתפריט הראשי: עריכת פרטוקולים, מעקב התקדמות, בדיקה חדשה, עריכת צוות, צפייה בנתונים.

keep_main(12)

שומר את המסך הראשי של המשתמש כדי שאפשר יהיה לחזור אליו לאחר יציאה למסכים אחרים.

times(13

רשימת זמנים של לחיצות משתמש – משמשת ליצירת מפתחות הצפנה (כלחלק מ-

exchange_key).

last_window(14

שומר את החלון הקודם כדי לאפשר חזרה אחורה (למשל במסך הרשמה או עריכה).

settings_page(15

מצביע למסך ההגדרות של המשתמש, כדי שניתן יהיה להסתיר או להציג אותו

בהתאם למעבר בין מסכים.

back_button(16

כפתור לחזרה למסך הראשי, מוצג במסכים משניים.

current_window(17

שומר את המסך הפעיל כעת באפליקציה (למשל: מסך כניסה, עמוד בית, עריכת

פרוטוקול וכו').

• פעולות במחלקה:

retry_connection (1

טענת כניסה: אין

טענת יציאה: מחזירה את המסך הראשי לאחר יציאה ממסך משני, ומבצעת החלפת

מפתחות הצפנה אם צריך.

back_to_main (2

טענת כניסה: אין

טענת יציאה: מציג מחדש את המסך הקודם (התחברות או הצטרפות)

back_to_log (3

טענת כניסה: אין

טענת יציאה: משנה את ערכת הצבעים של הממשק ומעדכנת את המסך הראשי

בהתאם

change_color_theme (4

טענת כניסה: bool מצב תאורה (True = אור, False = כהה)

טענת יציאה: משנה את מצב הממשק לתאורה או כהה

change_color_mode (5

טענת כניסה: אין

טענת יציאה: מציב את אובייקט האגרטל במקום המתאים בממשק

place_vase (6

טענת כניסה: אין

טענת יציאה: מציב את אובייקט האגרטל במקום המתאים בממשק

place_glitter (7

טענת כניסה: אין

טענת יציאה: ענת יציאה: מציב אלמנטים עיצוביים של נצנצים במסך

log_in (8

טענת כניסה: string שם משתמש, string סיסמה

טענת יציאה: שולח את פרטי ההתחברות לשרת, ואם יש אישור - עובר למסך

הראשי

open_main_window (9

טענת כניסה: string שם משתמש, string סוג הרשאה

טענת יציאה: מציג את המסך הראשי של המשתמש לפי ההרשאות

exit_main_window(10

טענת כניסה: אין

טענת יציאה: שומר את המסך הראשי ומעלים אותו כהכנה למעבר למסך משני

click(11

טענת כניסה: אין

טענת יציאה: שומר את זמן הלחיצה של המשתמש

new_test(12

טענת כניסה: אין

טענת יציאה: עובר למסך יצירת בדיקה חדשה.

edit_staff(13

טענת כניסה: אין

טענת יציאה: עובר למסך עריכת צוות

track_progress(14

טענת כניסה: אין

טענת יציאה: עובר למסך מעקב אחר התקדמות

edit_protocols(15

טענת כניסה: אין

טעגט יציאה: עובר למסך עריכת פרוטוקולים

edt_protocol_window(16

טעגט כניסה: string שם פרוטוקול (אופציונלי), string מספר (אופציונלי), string גרסה (אופציונלי)

טעגט יציאה: מציג מסך עריכת פרוטוקול או יצירת פרוטוקול חדש

creator(17

טעגט כניסה: string שם פרוטוקול, string מספר, string גרסה, string טקסט (אופציונלי)

טעגט יציאה: יוצר את ממשק העריכה הרלוונטי ומציג אותו.

new_protocol_window(18

טעגט כניסה: אין

טעגט יציאה: עובר למסך יצירת פרוטוקול חדש.

ai_edit_protocol_load(19

טעגט כניסה: string טקסט

טעגט יציאה: מציג אנימציות טעינה ומתחיל תהליך שליחה ל-AI לצורך עריכת פרוטוקול.

ai_edit_protocol(20

טעגט כניסה: string טקסט

טעגט יציאה: שולח טקסט לשרת לקבלת עריכה מ-AI ומפעיל את מסך העריכה.

view_data(21

טעגט כניסה: אין

טעגט יציאה: עובר למסך צפייה בנתונים

view_protocol(22

טעגט כניסה: string שם פרוטוקול, string מספר, string גרסה

טעגט יציאה: מציג את מסך הצפייה בפרוטוקול ומעדכן את השרת.

join(23

טעגט כניסה: list זמני לחיצות, string כותרת

טעגט יציאה: עובר למסך ההצטרפות למערכת.

on_closing(24

טעגט כניסה: אין

טענת יציאה: סוגר את התוכנית בצורה מסודרת – שומר נתונים, מעדכן סטטוס, וסוגר את החיבור לשרת.

• Client

- יורשת מ- socket.socket
- מנהלת את התקשורת עם השרת, את בניית ההודעות הצפנתן ושליחתן, קבלת הודעות ופיענוחן והחלפת מפתחות הצפנה.
- **תכונות במחלקה:**

- (1) MSG_LENGTH - לקבוע את אורך ההודעה המרבי שניתן לשלוח או לקבל בבת אחת.
- (2) PORT - לציין את מספר הפורט שאליו הלקוח יתחבר.
- (3) IP - להגדיר את כתובת ה IP של השרת שאליו מתחברים.
- (4) lock - לאפשר סנכרון בין תהליכי עבודה מקבילים כדי למנוע גישה מקבילה למידע קריטי.
- (5) indexes - למפות שמות של פעולות לקודים ייחודיים שנשלחים לשרת.
- (6) public - לאחסן את המפתח הציבורי של הלקוח להצפנת מידע.
- (7) private - לאחסן את המפתח הפרטי של הלקוח לצורך פיענוח מידע.
- (8) server_public - לאחסן את המפתח הציבורי של השרת לצורך הצפנת הודעות שנשלחות אליו.

• **פעולות במחלקה:**

- (1) confirm_connection
טענת כניסה: אין
טענת יציאה: הפעולה מנסה להתחבר לשרת ומחזירה סטרינג Denied או Confirmed בהתאם להאם ההתחברות צלחה.
- (2) yell
טענת כניסה: string שם הבקשה לשרת ו tuple הפרמטרים לבקשה.
טענת יציאה: הפונקציה שולחת את הבקשה לשרת ומחזירה את התשובה שלו.
- (3) exchange_key
טענת כניסה: רשימה של זמנים של לחיצות המשתמש
טענת יציאה: הפונקציה יוצרת זוג מפתחות הצפנה, שולחת לשרת את המפתח הציבורי שלה ומקבלת ממנו את המפתח הציבורי שלו ומחזירה אותו.

• LoginWindow

- יורשת מ- CTkFrame
- מסך כניסה למערכת.
- **תכונות במחלקה:**

times (1

שומר רשימת חותמות זמן של פעולות המשתמש לצורך יצירת מפתחות הצפנה מבוססי תזמון.

client (2

אובייקט מסוג Client שאחראי על התקשורת עם השרת, כולל שליחת נתוני התחברות וביצוע פעולות מאובטחות.

name (3

שדה קלט גרפי להזנת שם המשתמש.

password (4

שדה קלט גרפי להזנת סיסמה, עם הסתרת הטקסט לצורכי אבטחה.

- **פעולות במחלקה:**

user_click (1

טענת כניסה: אין

טענת יציאה: הפעולה שומרת את הזמן שבו נלחץ כפתור. אין החזרה.

next_entry (2

טענת כניסה: אין

טענת יציאה: הפעולה מעבירה את הפוקוס לשדה הבא. אין החזרה.

submit (3

טענת כניסה: אין

טענת יציאה: הפעולה מוודאת שהשדות לא ריקים וקוראת לפונקציית ההתחברות למערכת מהמחלקה App. אין החזרה.

join (4

טענת כניסה: אין

טענת יציאה: הפונקציה קוראת לפונקציה join של המחלקה App

change_password (5

טענת כניסה: אין

טענת יציאה: הפונקציה קוראת לפונקציה join של המחלקה App

- JoinPage

- יורשת מ- CTkFrame
- מסך ההצטרפות למערכת.
- **תכונות במחלקה:**

client (1)

אובייקט מסוג Client, המשמש לתקשורת עם השרת ושליחת בקשות רלוונטיות לתהליך ההרשמה.

times (2)

רשימת חותמות זמן שנאספות לצורך יצירת מפתחות הצפנה על בסיס תזמון פעולות המשתמש.

permission (3)

מחרוזת המציינת את רמת ההרשאה של המשתמש החדש) ברירת מחדל : "Worker").

title (4)

תווית גרפית המציגה את הכותרת העליונה של העמוד בהתאם להקשר) למשל "Join" או "Change Password").

entry (5)

שדה קלט עיקרי להזנת שם משתמש, קוד אימות או סיסמה – משתנה לפי שלב התהליך.

button (6)

כפתור ראשי המשמש לשליחה ואימות של הקלט (למשל בדיקת שם משתמש או קוד).

error_lbl (7)

תווית להצגת הודעות שגיאה או מידע למשתמש במהלך תהליך ההרשמה.

confirm_entry (8)

שדה קלט נוסף להזנת סיסמה לאימות (בעת יצירת סיסמה חדשה), עם הצגת תווים מוסתרים.

name (9

משתנה לשמירת שם המשתמש שנבחר, לשימוש בשלבים שונים של ההרשמה.

code(10

משתנה מספרי לשמירת קוד אימות, אם נדרש בתהליך.

back_button(11

כפתור גרפי עם אייקון חץ לחזרה למסך ההתחברות.

resend_button(12

כפתור המאפשר למשתמש לבקש שליחה חוזרת של קוד אימות במקרה שלא התקבל.

• פעולות במחלקה:

back (1

טענת כניסה: אין

טענת יציאה: הפונקציה קוראת לפונקציה back_to_log של App

user_click (2

טענת כניסה: אין

טענת יציאה: הפונקציה שומרת את זמן הלחיצה ואם יש מספיק לחיצות קוראת ל-

exchange key של Client

click (3

טענת כניסה: אין

טענת יציאה: הפונקציה מפעילה את הפעולה של כפתור מסוים (על מנת שלחיצה על מקש enter תהיה כמו לחיצה על הכפתור)

next (4

טענת כניסה: אין

טענת יציאה: הפונקציה מפקסת את השדה הבא.

send_code (5

טענת כניסה: אין

טענת יציאה: הפונקציה מבקשת שליחה של בקשת מייל עם קוד כניסה מהשרת ומחזירה את התשובה שלו.

check_username (6

טענת כניסה: אין

טענת יציאה: הפונקציה קוראת לבדיקה של השם משתמש והסיסמה בשרת ועוברת לעמוד הבית אם יש אישור.

confirm_code (7)

טענת כניסה: אין

טענת יציאה: הפונקציה מבקשת מהשרת לאשר את קוד הכניסה ואם יש אישור עוברת למסך הבית.

create_password (8)

טענת כניסה: אין

טענת יציאה: הפונקציה מסדרת את מסך יצירת הסיסמה.

save (9)

טענת כניסה: אין

טענת יציאה: מבקשת מהשרת להוסיף את הסיסמה החדשה של המשתמש לבסיס הנתונים.

• Server

- יורש מsocket.socket
- מנהל את התקשורת עם הלקוחות, מתקשר עם בסיס הנתונים.
- **תכונות במחלקה:**

difference_between_last_joins (1)

שומר את ההפרש בשניות בין שתי התחברויות לקוח אחרונות לשרת.

last_time_when_client_joined (2)

זמן התחברות הלקוח האחרון – חותמת זמן של time.time().

when_each_ip_was_last_connected (3)

מילון שמכיל לכל כתובת IP את הזמן האחרון שבו התחברה לשרת.

each_ip_difference_from_last_join (4)

מילון של הפרשי זמנים בין התחברות נוכחית של IP מסוים לבין התחברות קודמת שלו.

client_addresses (5)

מילון המקשר בין סוקט של לקוח לבין כתובת ה IP-שלו.

client_permissions (6

מילון שמכיל עבור כל לקוח את רמת ההרשאה שלו (למשל: עובד, מנהל).

client_codes (7

מילון עם קודי אימות שנשלחו למשתמשים, כולל זמן השליחה ותפקיד המשתמש.

manager_requests (8

רשימת פעולות שזמינות למשתמשים בעלי הרשאות ניהול.

general_requests (9

רשימת פעולות זמינות לכל המשתמשים המורשים.

complex_requests(10

רשימת פעולות מיוחדות שדורשות טיפול נוסף (כגון קריאה למודל בינה מלאכותית).

log_options(11

רשימת פעולות הקשורות להתחברות ואימות המשתמש (כגון שליחת קוד אימות, התחברות).

rabbit_hole(12

מילון שממין את הבקשות לפי קטגוריה (לדוגמה: ניהול, כלליות, התחברות).

clients(13

רשימה של כל הלקוחות שמחוברים כרגע לשרת.

messages_out(14

רשימת הודעות שממתינות להישלח ללקוחות (יוצא מהשרת ללקוח).

client_keys(15

מילון שמכיל עבור כל לקוח את מפתח ההצפנה הציבורי שלו והמפתח הפרטי של השרת עבורו.

• פעולות במחלקה:

run (1

טענת כניסה: אין

טענת יציאה: הפונקציה רצה לנצח ולכן אין טענת יציאה. הפונקציה מכילה את האלגוריתם של התנהלות השרת.

close_connection (2)

טענת כניסה: ה- socket שאותו רוצים לסגור

טענת יציאה: הפונקציה סוגרת חיבור ל- socket מסוים וממוציאה אותו מהרשימות שלה, ומדפיסה הודעת סגירה.

log_in (3)

טענת כניסה: פרטי כניסה למערכת מהמשתמש בtuple ואת אובייקט הsocket אשר שלח אותם.

טענת יציאה: אם המשתמש שלח שם וסיסמה נכונים, תוחזר רמת ההרשאה שלו במערכת. אם הוא שלח שם וסיסמה שגויים הבקשה תדחה. אם הוא שלח שם נכון ובקשה לקבל קוד כניסה יוחזר אישור.

send_mail (4)

טענת כניסה: אובייקט socket, כתובת המייל של המשתמש, הזמן בו התקבלה הבקשה והרשאת המשתמש.

טענת יציאה: הפונקציה שולחת מייל עם קוד כניסה לאפליקציה למשתמש.

verify_admission_code (5)

טענת כניסה: המידע מהמשתמש ואובייקט הsocket שלו.

טענת יציאה: הפונקציה מאמתת את קוד הכניסה של הלקוח ומוודאת שהוא נשלח בזמן, ומחזירה הודעה מתאימה.

ask_ (6)

טענת כניסה: אובייקט socket והתוכן הלא מסודר של קובץ פרוטוקול מהלקוח.

טענת יציאה: הפונקציה מריצה thread ששולח בקשה לgemini לסדר את התוכן באופן מסודר.

ask_gemini (7)

טענת כניסה: אובייקט socket והתוכן הלא מסודר של קובץ פרוטוקול מהלקוח.

טענת יציאה: הפונקציה שולחת בקשה לgemini שיסדר את תוכן ההודעה לפי פורמט מסוים, ולבסוף מוסיפה את התשובה לרשימת ההודעות חזרה ללקוח.

• EditableBidirectionalLabel

- יורשת מ CTKFrame
- אובייקט גרפי שמציג כפתור שניתן לעריכה על ידי הקלקה, ולאחר מכן חוזר לכפתור עם הטקסט המעודכן. על מנת לאפשר שינוי הפורמט של הטקסט בעת הצגתו כדי לאפשר כתיבה דו כיוונית ולאפשר עריכה.

- **תכונות המחלקה:**

enter_command (1)

פונקציה שמופעלת כאשר המשתמש מסיים לערוך את הטקסט) בלחיצה על Enter או ביציאה מהשדה.

button (2)

כפתור שמציג את הטקסט הנוכחי. כאשר לוחצים עליו, הוא מוחלף בשדה טקסט (Entry) לצורך עריכה.

entry (3)

שדה טקסט שבו ניתן להזין טקסט חדש. מופיע במקום הכפתור כשרוצים לערוך.

- **פעולות במחלקה:**

click (1)

טענת כניסה: אין.

טענת יציאה: מחליפה את הכפתור בשדה טקסט ומפקסת אותו לעריכה.

enter (2)

טענת כניסה: אין.

טענת יציאה: מחליפה את שדה הטקסט חזרה לכפתור, מעדכנת את הטקסט על הכפתור ומפעילה את הפונקציה שהוגדרה בעת האתחול עם הטקסט החדש.

set_font (3)

טענת כניסה: גופן חדש tuples

טענת יציאה: מעדכנת את הגופן של גם הכפתור וגם שדה הטקסט.

set (4)

טענת כניסה: מחרוזת חדשה לעריכה.

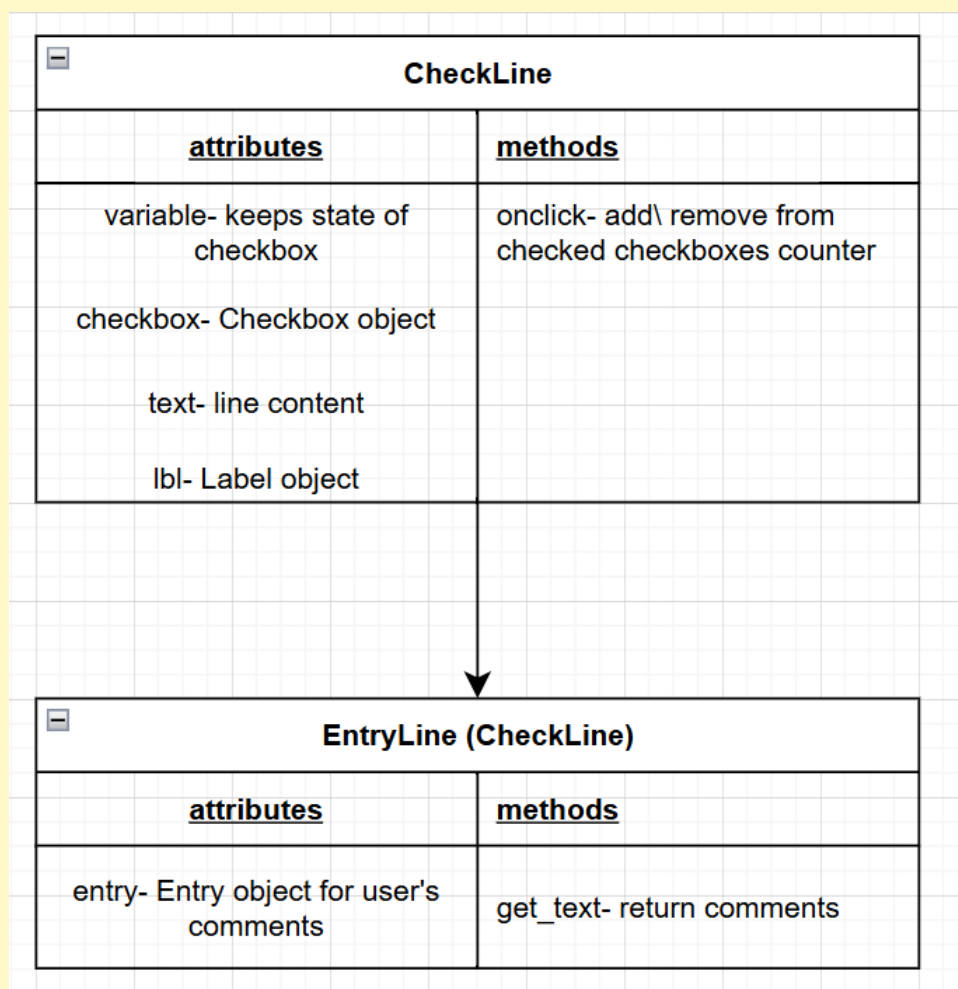
טענת יציאה: משנה את ערך הטקסט בשדה ובכפתור לטקסט שנמסר.

get (5)

טענת כניסה: אין.

טענת יציאה: מחזירה את הטקסט הנוכחי שבשדה הטקסט.

מחלקות עם יחס ירושה:



חלק ב'

- קטע קוד יפה בעיניי הוא הקוד של עמוד הצפייה בהיסטוריית הפעילות במערכת.

קטע זה משלב 3 מחלקות בפיתוח עצמי, אשר עובדות יחד כאשר לכל אחת יש תפקיד בשרשרת. יחד הן מאפשרות ללקוח לבחור עד 3 קטגוריות לסנן בעזרתן את המידע שהוא מחפש, באופן מובן, נוח ויעיל. הן מאפשרות ללקוח לנווט את היסטוריית הפעילות בקלות גם אילו הייתה בעלת אלפי ערכים- הרי לזה המערכת נועדה. המידע מופרד לטבלאות, טבלה בכל עמוד, 12 שורות בכל טבלה, ובכל רגע נתון ישנם 4 עמודים נגשים על המסך, וכפתורי "אחורה" ו- "קדימה" על מנת לנווט לעמודים אחרים. ללקוח ישנה היכולת לסנן לפי טווח תאריכים, שם עובד ושם פרטוקול. השאליות כתובות ביעילות, והטבלאות שנוצרו נשמרות כך שאותה טבלה לא תטען פעמיים. היכולת הזו של המערכת מיידית הופכת אותה לנעלה לחלוטין על פני השיטה הקיימת במעבדה כפי שהיא היום, כיוון וכאשר העובדים רוצים למצוא תיעוד עבודה מסוים הם נאלצים לחפש מבין תיקיות וטפסים רבים בחוסר יעילות.

```

# The View History page.
class ViewData(Frame):
    def __init__(self, master, client):
        super().__init__(master)
        self.client = client # communication object
        self.has_next = True # are there currently more pages available
to load
        self.data_pages = Data(self, self.get_data) # TabView object that
contains the tables of data
        self.data_pages.grid(row=0, column=0, columnspan=4)
        self.filters = [] # current active filter objects
        self.last_id = json.loads(client.yell("get_last_history_id")) #
last id number in the history table
        self.apply_button = Button(self, text="apply", command=self.apply)
# button to apply filters
        self.add_button = Button(self, text="Add Filter",
command=self.add_filter) # button to add filter
        self.add_button.grid(row=1, column=0)
        self.apply_button.grid(row=2, column=0)
        self.build_tables()

        # make initial pages show
    def build_tables(self):
        data, n = self.get_data(True)
        self.data_pages.build_pages(data, n)

        # create new Filter object
    def add_filter(self):
        if len(self.filters) < 3:
            f = Filter(self, self.client)
            self.filters.append(f)
            f.grid(row=1, rowspan=2, column=len(self.filters))

        # is_new: True if the tables being built are because the screen just
opened or a new filter was applied
        # False if they're being built due to "Next" button
        # gets all info from filters and gets data from server
        # returns data and number of pages
    def get_data(self, is_new):
        if self.has_next or is_new:
            filters = []
            values = []
            for i in range(len(self.filters)):
                filter_name = self.filters[i].get_filter()
                filter_value = self.filters[i].get_value()
                if filter_name != "No Filter" and filter_value is not
None:
                    filters.append(filter_name)
                    values.append(filter_value)
            # data = db_worker.get_4_pages_filtered(self.last_id, filters,
values)
            data = json.loads(
                self.client.yell("get_4_pages_filtered", (self.last_id,
json.dumps(filters), json.dumps(values))))

```

```

        if len(data) == 0:
            return None, 0
        self.last_id = data[-1][0]
        n = len(data) // 12
        if len(data) % 12 != 0:
            n += 1
        if n < 4:
            self.has_next = False
        return data, n
    return None, 0

# reload the Data object according to new filters
def apply(self):
    self.last_id = json.loads(self.client.yell("get_last_history_id"))
    self.has_next = True
    self.data_pages.clear()
    self.build_tables()

# TabView object that always has on 4 current pages, with "Previous" and
"Next" buttons to navigate
class Data(Tabview):
    def __init__(self, master, get_data_command):
        super().__init__(master, command=self.click)
        self.NUM = 4 # number of pages available on screen at any moment
        self.get_data_command = get_data_command # get_data command from
ViewData class
        self.built_pages = [] # list of CTKTable objects built for each
page in order
        self.current_first = 1 # keeps the current first page with a
button available on the screen
        self.current_num_of_pages = self.NUM # how many pages on screen
currently have a table on them (are not empty)
        # create all buttons for the TabView object
        self.add("Previous")
        for i in range(self.NUM):
            self.add(f"page{i + 1}")
            Label(self.tab(f"page{i + 1}"))
        self.add("Next")
        self.set("page1")

        # data: 2d list of history values from database. num_of_pages: amount
of pages to create with data
        # creates all tables and places them on their pages
        def build_pages(self, data, num_of_pages):
            self.current_num_of_pages = num_of_pages
            for t in self.built_pages[self.current_first - self.NUM -
1:self.current_first]:
                t.pack_forget()
            if self.current_num_of_pages != 0:
                for i in range(self.current_first, self.current_first +
num_of_pages - 1):
                    t = Table(self.tab(f"page{i}"),
                                values=data[(i - self.current_first) * 12: (i -

```

```

self.current_first + 1) * 12],
                                command=self.open_protocol_view)
        t.pack()
        self.built_pages.append(t)
        t = Table(self.tab(f"page{self.current_first + num_of_pages -
1}"), values=data[(num_of_pages - 1) * 12:],
                                command=self.open_protocol_view)
        t.pack()
        self.built_pages.append(t)

# cell: information about the cell that was pressed in the table
# opens TopLevel screen with the protocol that was clicked, with the
comments that were written on it.
def open_protocol_view(self, cell):
    # table = self.tab(self.get()).wininfo_children()[0]
    table = self.built_pages[int(self.get()[4:]) - 1]
    row = cell["row"]
    row_id = table.get(row=row, column=0)
    protocol_id = table.get(row=row, column=1)
    protocol = table.get(row=row, column=2)
    version = table.get(row=row, column=3)
    comments = self.master.client.yell("get_comments", (row_id,))
    if comments != "None":
        comments = json.loads(comments)
    else:
        comments = []
    w1 = tk.CTkToplevel(self)
    w1.geometry("600x600")
    w1.after(100, w1.lift)
    print(type(comments))
    if protocol_exists(self.master.client, [protocol, protocol_id,
version]):
        p = build.ProtocolView(w1, protocol, protocol_id, version,
self.master.client, comments=comments)
        p.pack()
    else:
        Label(w1, text=f"This Protocol Has Been
Deleted.\n\nComments:\n{comments}").pack()

# when TabView segmented button is clicked
def click(self):
    if self.get() == "Next":
        self.next()
    elif self.get() == "Previous":
        self.previous()

# if the tables have already been built, place them
# if they haven't, build them
def next(self):
    if len(self.built_pages) > self.current_first + self.NUM - 1:
        self.place_next()
    else:
        self.build_next()
    self.set(f"page{self.current_first}")

```

```

# get the data and build pages if there's data
def build_next(self):
    data, n = self.get_data_command(False)
    if data is not None:
        for i in range(self.NUM):
            self.rename(f"page{self.current_first + i}",
f"page{self.current_first + i + 4}")
            self.current_first = self.current_first + self.NUM
            self.build_pages(data, n)

# place the next built tables
def place_next(self):
    for i in range(self.NUM):
        self.built_pages[i + self.current_first - 1].pack_forget()
    for i in range(self.current_first + self.NUM - 1,
self.current_first + 2 * self.NUM - 1):
        self.rename(f"page{i - self.NUM + 1}", f"page{i + 1}")
        i = self.current_first + self.NUM - 1
        while i < self.current_first + 2 * self.NUM - 1 and i <
len(self.built_pages):
            self.built_pages[i].pack()
            i += 1
        self.current_num_of_pages = i - self.current_first - self.NUM + 1
        self.current_first = self.current_first + self.NUM

# place the previous tables if there are any
def previous(self):
    n = self.current_num_of_pages
    if self.current_first > 1:
        for i in range(n):
            self.built_pages[i + self.current_first - 1].pack_forget()
        for i in range(self.NUM):
            self.rename(f"page{self.current_first + i}",
f"page{self.current_first + i - self.NUM}")
            self.built_pages[self.current_first + i - 1 -
self.NUM].pack()
            self.current_first = self.current_first - self.NUM
            self.move(self.NUM + 1, "Next")
            self.current_num_of_pages = self.NUM
            self.set(f"page{self.current_first}")

# delete all tables, to prepare for new ones with new applied filters
def clear(self):
    print("first", self.current_first, "num",
self.current_num_of_pages)
    i = 0
    while i < self.NUM and len(self.built_pages) >= self.current_first
+ i:
        self.built_pages[self.current_first + i - 1].pack_forget()
        i += 1
    # for i in range(self.current_first, self.current_first +
self.current_num_of_pages):
        if self.current_first != 1:

```

```

        for i in range(self.current_first, self.current_first +
self.NUM):
            self.rename(f"page{i}", f"page{i - self.current_first +
1}")

        self.current_first = 1
        self.set("page1")
        self.built_pages = []

# a single filter object has a category and value
class Filter(Frame):
    def __init__(self, master, client):
        super().__init__(master)
        self.client = client
        self.values = {"Employee": []}
        self.date = False
        self.filter_menu = OptionMenu(self, values=["No Filter",
"Employee", "Protocol", "Date Range"],
                                     command=self.first_filter_by)

        self.filter_menu.pack(pady=5)
        self.secondary_filter = OptionMenu(self)
        self.from_lbl = Label(self, text="From:", font=("Roberto", 15),
height=15)
        self.to_lbl = Label(self, text="To:", font=("Roberto", 15),
height=15)
        self.start_date_entry = Entry(self, placeholder_text="YYYY-MM-DD")
        self.end_date_entry = Entry(self, placeholder_text="YYYY-MM-DD")

        # arg: value chosen in the OptionMenu
        # what to do when the filter's category is chosen
        # loads possible values into the next OptionMenu or places Entries
    def first_filter_by(self, arg):
        if arg == "No Filter":
            self.secondary_filter.pack_forget()
        if self.date:
            self.exit_date_range()
        if arg == "Employee":
            employees = self.client.yell("get_employee_names")
            employees = json.loads(employees)
            employees = [x[0] for x in employees]
            self.secondary_filter.configure(values=employees)
            self.secondary_filter.set(employees[0])
            self.secondary_filter.pack(pady=5)
        elif arg == "Protocol":
            protocols = self.client.yell("get_protocol_names")
            protocols = json.loads(protocols)
            protocols = [x[0] for x in protocols]
            self.secondary_filter.configure(values=protocols)
            self.secondary_filter.set(protocols[0])
            self.secondary_filter.pack(pady=5)
        elif arg == "Date Range":
            self.date = True
            self.from_lbl.pack()
            self.start_date_entry.pack()

```

```

        self.to_lbl.pack()
        self.end_date_entry.pack()

# remove date range entries and labels
def exit_date_range(self):
    self.date = False
    self.from_lbl.pack_forget()
    self.start_date_entry.pack_forget()
    self.to_lbl.pack_forget()
    self.end_date_entry.pack_forget()

# returns the filter category name
def get_filter(self):
    return self.filter_menu.get()

# return the value of the filter, if it is valid, in the way it should
go in the sqlite query
# note that the server verifies the data to prevent sql injection
def get_value(self):
    if not self.date:
        return f'{self.secondary_filter.get()}'
    date_from = self.start_date_entry.get()
    date_to = self.end_date_entry.get()
    time_format = "[0-9]{4}(-[0-1][0-9](-[0-3][0-9])?)?" # 'YYYY-MM-
DD' or 'YYYY-MM' or 'YYYY'
    if re.fullmatch(time_format, date_from) and
re.fullmatch(time_format, date_to):
        ans = f'{date_from}' + " and " + f'{date_to}'
        print("CONFIRM")
        return ans
    win32ui.MessageBox("Make sure you're using YYYY-MM-DD format!",
"ok", 1)
    return None

# check if a certain protocol currently exists
def protocol_exists(client, values):
    protocols = json.loads(client.yell("get_protocols"))
    for p in protocols:
        print(p)
        if p == values:
            return True
    return False

```

חלק ג'

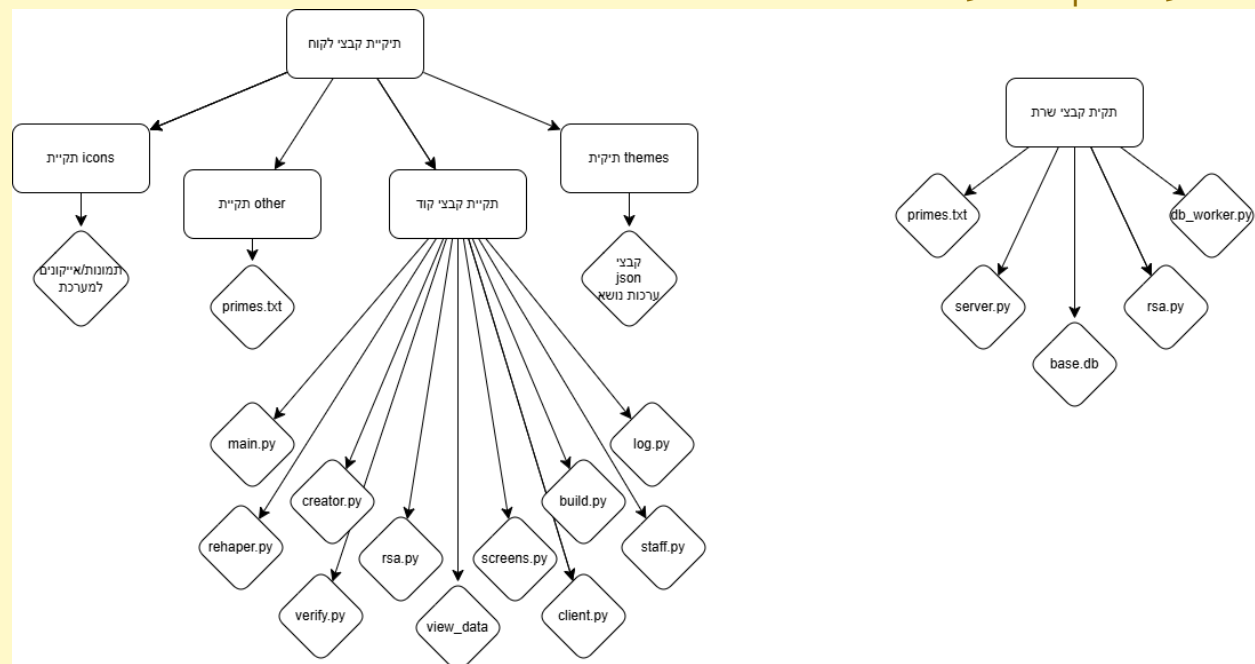
מטרת הבדיקה	מה בוצע בפועל	תוצאות הבדיקה	פתרונות
שליחת הודעה ארוכה	הרצת התוכנה עם שליחה של פרוטוקול ארוך בתקשורת	הפרוטוקול הגיע חלקי	חלוקה של ההודעות שעוברות בתקשורת לפקטות.

יצירת משתמש	הרשמת משתמש חדש	בעיה בשמירה	Commit לא נקרא כראוי.
הוספת משתמש במקביל	שני משתמשים פתחו את עריכת המשתמשים יחד וניסו להוסיף משתמש עם אותו שם למערכת באותו זמן	שגיאה	אימות של המידע מהלקוח גם בצד של השרת.
עריכת פרוטוקולים חופשית	שיחקתי עם הממשק לעריכת פרוטוקולים בכמה שיותר בלגאן	חלה שגיאה	עברתי על האלגוריתם ותיקנתי את השגיאה.
מעקב אחר התקדמות העובדים	נכנסתי מכמה משתמשים לפרוטוקולים שונים ויצאתי מהם בדרכים לא מסודרות	חלק מהעובדים הופיעו כעדיין עובדים על פרוטוקולים גם כאשר כבר לא היו מחוברים	הוספתי הסרה של סטטוס העבודה עם סגירת חלון האפליקציה
הצפנת rsa במספרים גדולים	הרצתי את הקוד שעבד בלי הצפנה עם הצפנת rsa	התוכנה נכנסה ללופ אינסופי	עברתי כל קוד ההצפנה ושיניתי את פעולת החזקה שתעשה בעזרת pow עם מודולו מובנה במקום חזקה רגילה, כדי שהתוצאה לא תהיה גדולה מידי.
בדיקת ביטחון מידע במערכת	ניסיתי להשיג מידע מהשרת דרך מניפולציה של הקוד בצד לקוח בלי שם וסיסמה.	השרת החזיר את המידע.	הוספתי הרשאות גישה השמורות בשרת ומבוססות על ההתחברות.
בדיקת תקינות הוספת הפרוטוקול בעזרת ai	העליתי מהאפליקציה קובץ של פרוטוקול אמיתי.	עלה על המסך "פרוטוקול" שהוא בעצם מתכון לעוגת שוקולד.	שכחתי להוסיף את הטקסט מהפרוטוקול לבקשה שנשלחה למודל השפה.

בדיקת קריסת לקוח	הקרסתי קוד לקוח מחובר	השרת קרס	הוספתי try לקבלת המידע מהלקוח
בדיקת תקינות שליחת הקודים במייל	נתתי למספר חברים להירשם למערכת	לעתים רחוקות קרה שמייל לא הגיע עד לאחר מספר דקות.	הוספתי כפתור של שליחה חוזרת של הקוד.
בדיקת תקינות העמוד לצפייה בהיסטוריה	הלכתי הלך וחזור בין העמודים שוב ושוב	התכנה קרסה	פספסתי איפוס של אחד המשתנים.
בדיקת תקינות המסננים של ההיסטוריה	הפעלתי מסננים	התוכנית קרסה	פספסתי json.loads
בדיקת קריסת שרת	הקרסתי את השרת בזמן שהיה מחובר אליו לקוח	התכנה קרסה	הוספתי try בכל קבלה/שליחה של הודעה, שאם כושל התוכנה נסגרת באופן מסודר.

מדריך למשתמש

פירוט על כלל קבצי המערכת:



- main - הקובץ אשר מריצים ממנו את התכנה מהצד של הלקוח

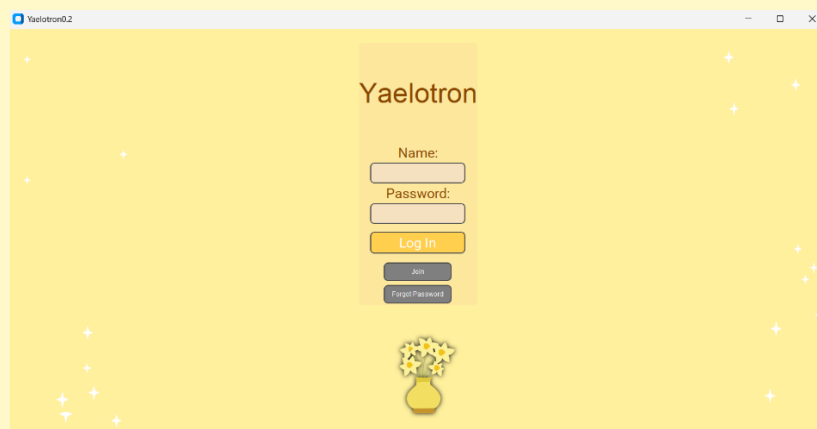
- reshaper - אוסף פעולות המשמשות להפוך טקסט רגיל לטקסט המסודר כך שכאשר הוא יעלה על המסך הוא יראה רגיל למרות שהוא מכיל עברית ואנגלית.
- creator - מחלקות שיחד בונות את הממשק לעריכת פרוטוקולים
- rsa - קובץ שמכיל את כל האלגוריתמים הקשורים לrsa.
- view_data - קובץ המכיל את כל המחלקות שיחד בונות את עמוד הצפייה בפרוטוקולים.
- client - קובץ המכיל את המחלקה של אובייקט התקשורת עם השרת.
- staff - קובץ המכיל את המחלקה של עמוד עריכת המשתמשים.
- log - קובץ המכיל את המחלקות של עמודי ההתחברות וההרשמה למערכת.
- build - הקובץ מכיל את המחלקות שבונות יחד את ההצגה של תוכן הפרוטוקולים.
- screens - הקובץ מכיל מספר מחלקות שוליות קצרות, כגון המחלקה שמאפשרת הצגה של gif, המחלקה של עמוד הבית, המחלקה של מסגרת ההגדרות, של עמוד בחירת הפרוטוקולים, ושל הצפייה בהתקדמות העובדים.
- server - הקובץ מכיל את המחלקה של התקשורת של השרת, ואת כל הפעולות שהשרת יכול לבצע.
- db_worker - הקובץ מכיל פעולות על בסיס הנתונים והוא היחיד שיכול לכתוב ולקרוא ממנו. השרת משתמש בו.

התקנת המערכת:

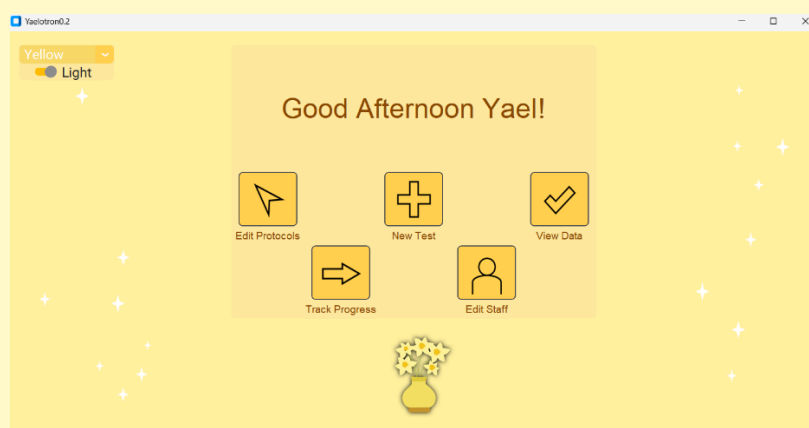
- על מנת להתקין את המערכת דרושים שפת פיתון pipi.
- בממסד הנתונים צריך להיות לפחות מנהל אחד שמור עם שם וסיסמה.
- ללקוח חייב להיות חיבור לאינטרנט, מערכת הפעלה windows/Linux/macOS, מעבד מודרני, זיכרון ואחסון.
- באופן תיאורטי, אילו הפרויקט היה מיועד לשימוש, הייתה נקנית כתובת dns קבועה עבור השרת אשר הייתה כתובה בקוד הלקוח. מכיוון וזה לא המצב, על הלקוח לכתוב את כתובת ה-ip המדויקת של השרת בקוד של הלקוח.

משתמשי המערכת:

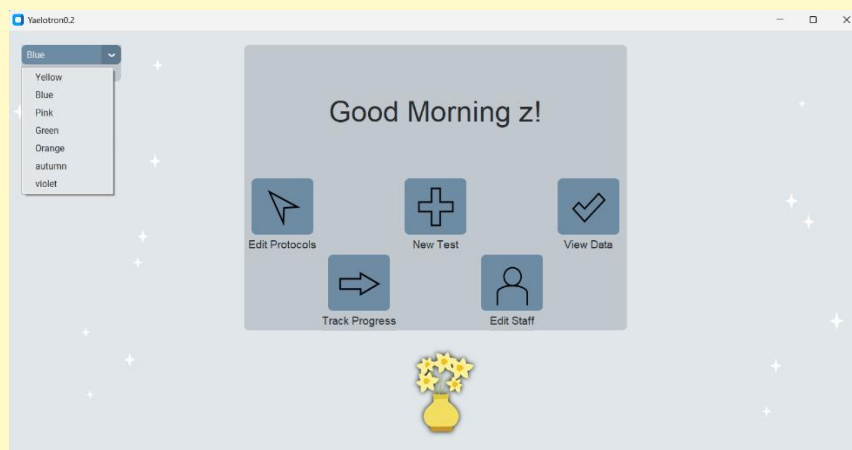
- עבור מנהלי המעבדה:
- יש להריץ את main על מנת לפתוח את האפליקציה.



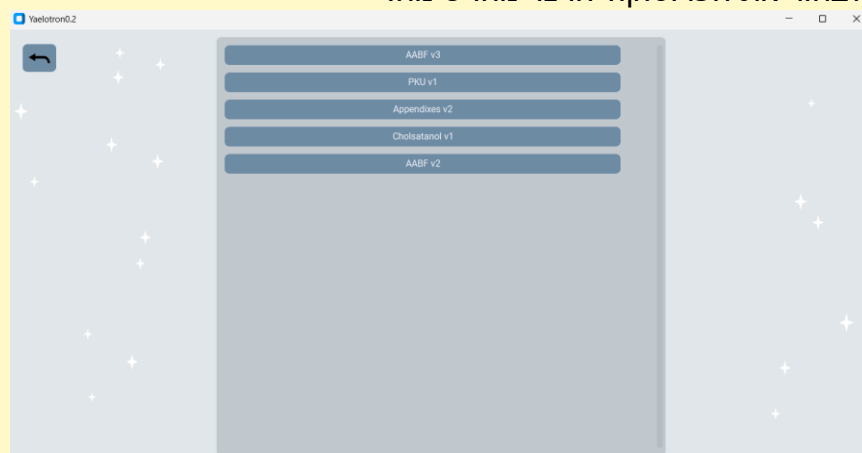
- יש להכניס שם משתמש וסיסמה וללחוץ Log In או על מקש Enter.



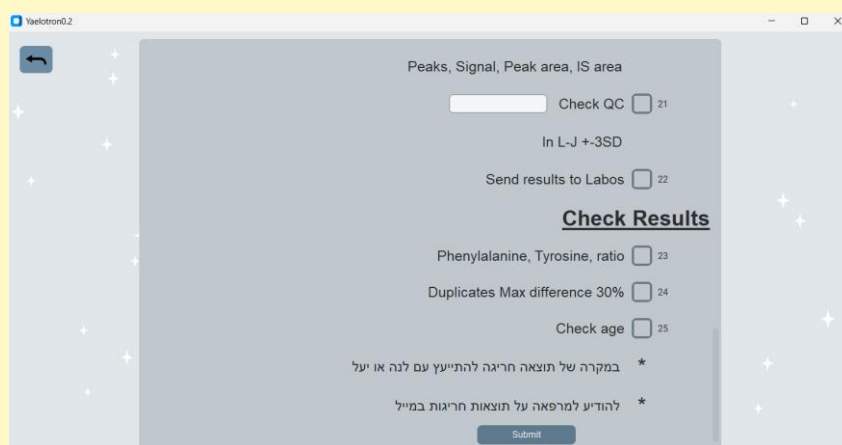
- על מנת לשנות את ערכת הנושא של האפליקציה יש לפתוח את התפריט למעלה-שמאלה על המסך ולבחור באופציה הרצויה. על מנת לעבור ממצב בהיר לכהה יש לשנות את ה"סליידר" שמתחת לתפריט.



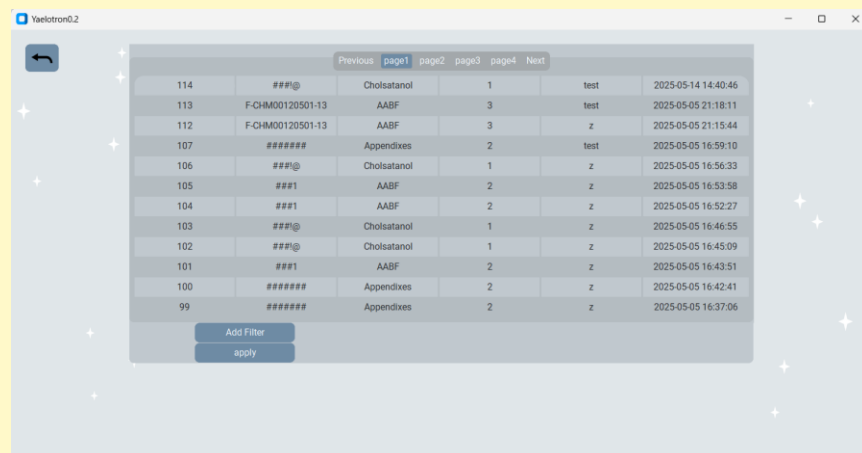
- אם ברצונך להשתמש בפרוטוקול מסוים יש ללחוץ על כפתור New Test במסך הבית, ולבחור את הפרוטוקול הרצוי מהרשימה.



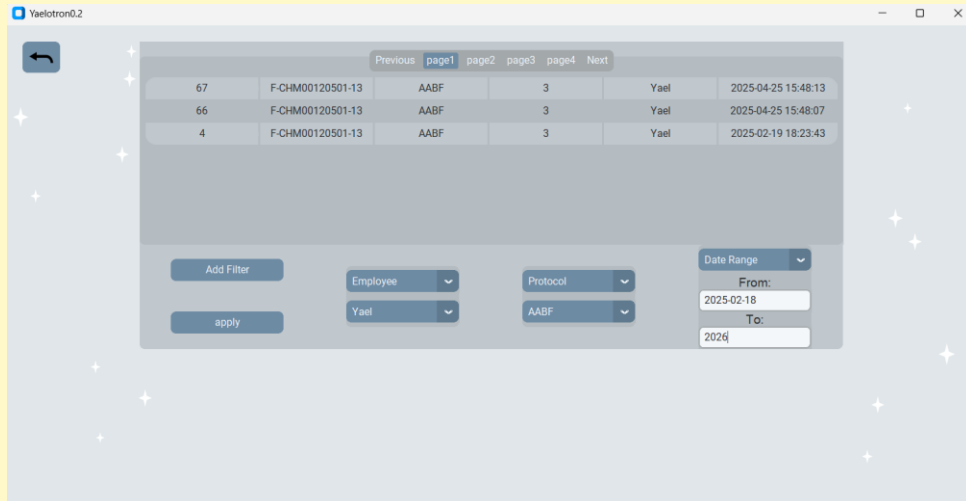
- ניתן לעבוד עם הפרוטוקול כרגיל. בעט סיום העבודה יש ללחוץ על Submit.
- לאחר לחיצה על "PKU" וגלילה למטה:



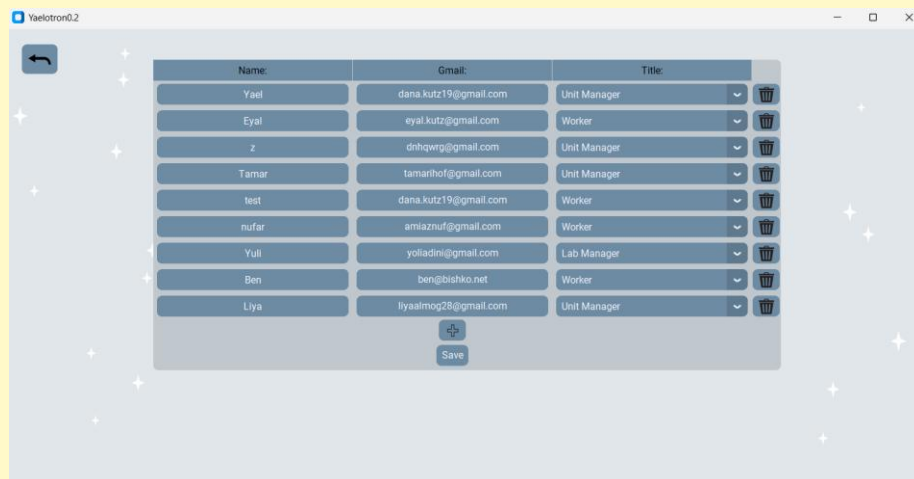
- על מנת לצפות בהיסטורית העבודה, יש ללחוץ על כפתור View Data במסך הבית.



- על מנת לחפש תוצאות מסוימות מתוך ההיסטוריה יש ללחוץ על Add Filter, ולבחור את המסנן הרצוי. ניתן להוסיף עד 3 מסננים- לפי עובד, לפי פרוטוקול ולפי תאריך. ניתן לשלב ולהשתמש בשלושתם.
- לאחר הוספת 3 פילטרים לדוגמה:

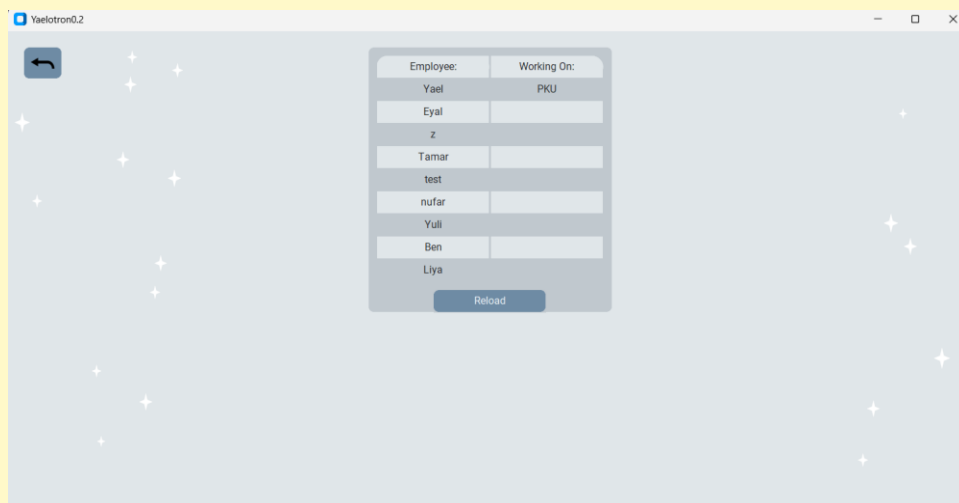


- על מנת לערוך את רשימת העובדים יש ללחוץ על הכפתור Edit Staff במסך הבית.



- על מנת להוסיף עובד, יש ללחוץ כל הכפתור +, לכתוב את שם העובד, לעבור לשדה הבא (או ללחוץ Enter), ולכתוב את כתובת המייל של אותו עובד. ניתן להוסיף/ לערוך את המייל בזמן אחר, אך השם הוא קבוע. על מנת לערוך את המייל יש ללחוץ עליו. על מנת להסיר עובד יש ללחוץ על האייקון של הפח הנמצא ליד שמו, ואז לאשר בחלונית האישור שקופצת. על מנת לשנות את התפקיד של אחד העובדים יש ללחוץ על התפריט שעליו כתוב תפקידו כרגע, ולבחור את האופציה הרצויה. בסיום העריכה יש ללחוץ על save, אלה אם רוצים לבטל את השינויים. במקרה כזה יש לצאת מהעמוד והשינויים לא ישמרו.

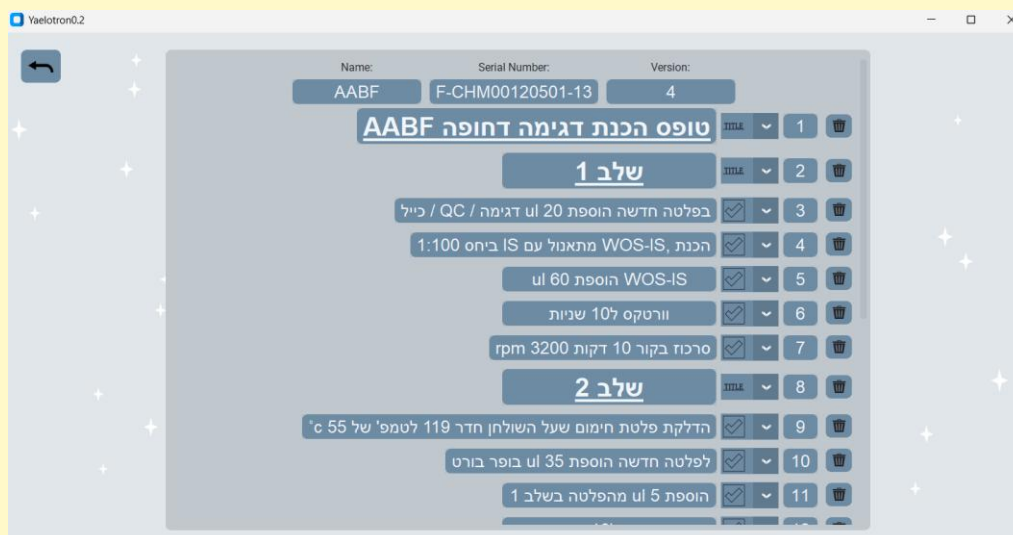
- על מנת לצפות בסטטוס העבודה של יתר העובדים יש ללחוץ על הכפתור Track Progress במסך הבית, ועל מנת לרענן את העמוד יש ללחוץ על הכפתור Reload.



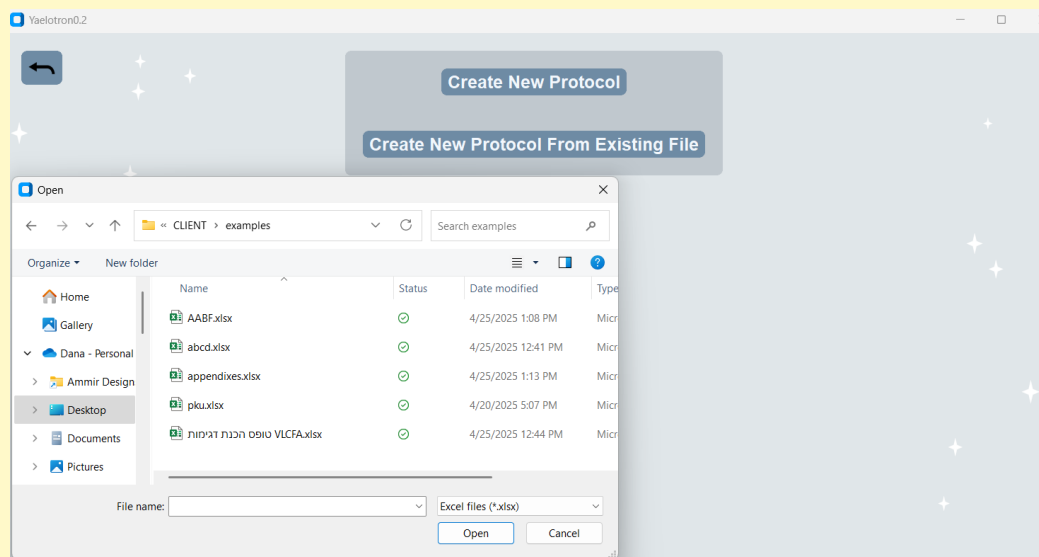
- על מנת לערוך את הפרוטוקולים יש ללחוץ על הכפתור Edit Protocols.



- על מנת להוסיף פרוטוקול יש ללחוץ על +, על מנת למחוק פרוטוקול יש ללחוץ על האייקון של הפח שלידו, ועל מנת לערוך פרוטוקול קיים יש ללחוץ עליו.
- לאחר לחיצה על "AABF":



- כאשר עורכים פרוטוקול, הוא עולה על המסך כך שניתן לשנות את התוכן של כל שורה על ידי לחיצה עליה, את הסוג של כל שורה על ידי בחירה מתוך התפריט שמיימנה, את סדר השורות על ידי לחיצה על מספר השורה, להוסיף שורות על ידי לחיצה על + או למחוק שורות על ידי לחיצה על הפח. הכותרות מציינות את שם הפרוטוקול והמספר הסידורי שלו, והגרסה היא אוטומטית אחת אחרי הגרסה האחרונה הקיימת של הפרוטוקול עליו לחצתם. ניתן לערוך את כולם.
- לאחר לחיצה על + במסך בחירת הפרוטוקולים לעריכה:



- על מנת להוסיף פרוטוקול לפי קובץ קיים, יש ללחוץ על 'Create New Protocol From Existing File'. אז נפתח חלון בחירת קבצים- בו יש לבחור את הקובץ הרצוי. אילו זה קובץ אקסל, יעלו עוד כפתורים על המסך המציינים כל גיליון בקובץ. יש לבחור את הגיליון הרצוי, והפרוטוקול יעלה על המסך במצב עריכה.
- על מנת להוסיף פרוטוקול חדש מאפס יש ללחוץ על הכפתור 'Create New Protocol', ויפתח פרוטוקול ריק במצב עריכה.

רפלקציה

- במהלך העבודה על הפרויקט נתקלתי בקשיים רבים, תחילה כי לא היה לי מספיק ידע בנושאי רשתות, ניהול שרת מרובה משתמשים, sql, הצפנה, ואבטחה. מתוך העבודה על הפרויקט למדתי את הנושאים האלה ובעזרת ספר גבהים רשתות ומחקר רב באינטרנט. התקשיתי במיוחד כאשר גיליתי על כשל במודול customtkinter אשר לא מאפשר כתיבה דו כיוונית, אשר היא הכרחית לפרויקט שלי. חוויתי הרבה מאוד כשלים כאשר ניסיתי לסדר את העניין הזה. בנוסף, אני מרגישה שמאוד העמקתי את הידע שלי בנושא של ניהול ממסד נתונים גדול, כאשר חשבתי על דרכים לבצע שאילתות מסובכות על בסיס נתונים שמיועד להיות באורך אלפי שורות ואף יותר.
- למדתי איך עובדת תקשורת מרובת משתמשים, הצפנה, הגנה מפני מספר מתקפות אפשריות, ניהול חכם של בסיס נתונים, ניהול לקוחות ועוד.
- מבין הנושאים שלמדתי לבד, נושאים שלא נלמדו כלל בכיתה הם שימוש בשליחת מייל, שימוש בapi של מודל שפה, הגנה מפני DDoS.
- מתוך העבודה אני אקח תובנות רבות על איך עובדות מערכות בעולם האמיתי, איך מתנהלים שרתים ואיך להגן מפני מתקפות.
- בהסתכלות אחורה בהחלט הייתי מיישמת חלקים מהפרויקט אחרת. הייתי משלבת threading בכל בקשה שהשרת מטפל בה שדרשת קריאה או כתיבה מבסיס הנתונים, הייתי בונה את העמוד של הצפייה בהיסטוריה אחרת- מסודר יותר, הייתי מעצבת את פרוטוקול התקשורת אחרת, אולי הייתי בונה אתר במקום אפליקציה בcustomtkinter.
- אילו היו ברשותי משאבים נוספים הייתי יכולה להשיג כתובת dns לשרת שלי על מנת שהלקוחות לא יצטרכו להכניס את כתובת הip של השרת לפני הריצה. במידה והיה לי עוד זמן הייתי משפרת את הפרויקט ומוסיפה לו דברים נוספים ומשפרת את יעילות ביצועיו. אילו היה לי מההתחלה הרבה יותר זמן לבנות את הפרויקט הזה, הייתי בונה אתר במקום אפליקציה בCTK, כדי להקל על עובדי למעבדה להיות מסוגלים להעלות את האפליקציה על המחשבים שלהם.
- **תודות:**
 - ליאת סגל- מורה מסורה, שאכפת לה לא רק מכל התלמידים שלה, אלא גם תלמידים שלא למדו איתה אף פעם, מספיק כדי להציע עזרה בכל הזדמנות, להיות זמינה תמיד, ולשאול לשלומם ולהתקדמותם. מוכנה לשבת ולהשקיע מזמנה עבור כל מי שצריך.
 - רום רפסון- מורה מצוין בכל המובנים. מלמד בעניין וביעילות, באופן הברור ביותר. מעניק יחס אישי לכל מי שצריך, קשוב ומתאים את רמת הלימוד לכל תלמיד. רואה את כל התלמידים שלו מתוך אכפתיות אמיתית.

- עידן פלדברג- מורה חיובי ותומך, שמעלה את המורל בכיתה בהתלהבות מדבקת מהמקצוע, שתמיד מוכן לבוא לעזרת תלמידים, לשבת איתם על כל בעיה ולפתוח עבורם את המעבדה שעות נוספות.
- אודי רז- בתקופה הקצרה שלמדתי בכיתתו הוא לימד ביעילות, העביר חומר טוב, ותמיד היה מוכן לייעץ ולעזר לתלמידים.
- קובי שוצמן- מורה מיוחד שהיה יכול לצאת לפנסיה אך בחר להמשיך ללמד מתוך אכפתיות למערכת החינוך, בעל ניסיון רב בתחום, ושחולק חומרי לימוד מהאתר שלו עם כל תלמידיו.
- יעל קוץ- אימי, שיחדיו חשבנו על הרעיון של האפליקציה הזו. היא עזרה לי לכל אורך הדרך, הסבירה לי את המצב ואת דרך ההתנהלות שלהם, את צרכיהם והעדפותיהם, כך שאוכל לבנות את האפליקציה הטובה ביותר עבור המשתמשים.
- גדעון קוץ- אבי, שתמך בי לאורך הדרך ונתן לי לדבר אליו ולהסביר לו מה אני מתכננת ולנסות להבין על מה אני מדברת מתוך הדיבור אליו. הוא ייעץ לי ותמך בי.
- אייל קוץ- אחי, שעזר לי בדרך, הסביר וייעץ
- חבריי למגמה- שהפכו את הלמידה לכיפית, שבזכותם ציפיתי לשיעורים, שנרשמו לאפליקציה שלי כדי לעזור לי לבדוק אותה, או סתם כדי לתת לי להשוויץ בהתקדמותה, והיו ליצידי לאורך כל הדרך.

ביבליוגרפיה

נעזרתי במקורות הבאים:

- ספר גבהים רשתות
- [Stack Overflow](#)
- [w3schools](#)
- [geeksforgeeks](#)
- [gemini](#)
- [draw.io](#)

קטעי קוד ממקורות חיצוניים:

- קוד המחלקה [GifLabel](#)
- הבסיס של קוד הקובץ [rsa.py](#)

נספחים

הקובץ main:

```

from customtkinter import CTkLabel as Label, CTkButton as Button, CTkImage
as CTkImage
import program_files.view_data3 as view_data
import program_files.creator2 as creator2
import program_files.screens as screens
import program_files.client as client
import program_files.build as build
import program_files.staff as staff
import program_files.log as log
import customtkinter as tk
from PIL import Image
import threading
import hashlib
import random
import json
import time

# האפליקציה של הראשית המחלקה הגדרת
class App(tk.CTk):
    def __init__(self):
        # מחדל כברירת צבעים ערכת הגדרת
        tk.set_default_color_theme("designs/Yellow.json")
        # החלון ממדי הגדרת
        self.HEIGHT = 600
        self.WIDTH = 1200
        # האב המחלקה אתחול
        super().__init__()
        # לתקשורת אובייקט אתחול
        self.client = client.Client()
        self.json = json.load(open("designs/Yellow.json"))
        self.geometry(f"{self.WIDTH}x{self.HEIGHT}")
        self.title("Yaelotron0.2")
        self.light_mode = True
        # ויזואליים אובייקטים
        self.vase = Label(self,
image=CTkImage(light_image=Image.open("icons/vase2.png"), size=(150,
150)), text="")
        self.glitter = []
        self.username = ""
        self.user_permission = ""
        # טעינה אנימציות
        self.gif = screens.GIFLabel(self, "icons/loading.gif", width=150,
height=150)
        # גרפיים אלמנטים הצגת
        self.place_glitter()
        self.place_vase()
        # עיקריים לחצנים
        self.primary_buttons = [self.edit_protocols, self.track_progress,
self.new_test,
                                self.edit_staff, self.view_data]

```

```

# מצבים לשמירת נוספים משתנים
self.keep_main = None
self.times = [time.time()]
self.last_window = None
self.settings_page = None
# חזר לחצן
self.back_button = Button(self, text="",
command=self.back_to_main,

image=CTkImage(light_image=Image.open("icons/back.png"), size=(30, 30)),
width=20)
# לשרת חיבור בדיקת
ans = self.client.confirm_connection()
if ans == "Deny":
    self.current_window = screens.ErrorWindow(self,
self.retry_connection)
else:
    self.current_window = log.LogInWindow(self, self.client)
    self.current_window.pack(pady=20)

# לשרת להתחברות חוזר נסיון
def retry_connection(self):
    self.client = client.Client()
    if self.client.confirm_connection() == "Confirmed":
        self.current_window.pack_forget()
        self.current_window = log.LogInWindow(self, self.client)
        self.current_window.pack(pady=20)

# הראשי למסך חזרה
def back_to_main(self):
    if len(self.times) % 2 == 0:
        var1, var2 = int(self.times[random.randrange(len(self.times) -
1)] // 1), int(self.times[-1] // 1)
        t = threading.Thread(target=self.client.exchange_key,
args=(var1, var2))
        t.start()
    if type(self.current_window) == build.ProtocolView:
        self.client.yell("update_status", ("", self.username))
    last = self.current_window
    self.current_window.pack_forget()
    self.current_window = self.keep_main
    self.last_window = last
    self.current_window.pack(pady=20)
    self.back_button.place_forget()
    self.settings_page.place(x=20, y=20)
    self.place_vase()

# לוגין) הקודם למסך חזרה
def back_to_log(self):
    self.current_window.pack_forget()
    self.current_window = self.last_window
    self.current_window.pack()

# צבעים ערכת שינוי

```

```

def change_color_theme(self, color):
    self.current_window.pack_forget()
    tk.set_default_color_theme(f"designs/{color}.json")
    self.current_window = screens.PrimaryWindow(self, self.username,
self.primary_buttons, self.user_permission)
    self.settings_page.place_forget()
    self.settings_page = screens.Settings(self)
    self.settings_page.place(x=20, y=20)
    self.current_window.pack(pady=20)
    self.json = json.load(open(f"designs/{color}.json"))
    self.configure(fg_color=self.json['CTk']['fg_color'])
    self.back_button = Button(self, text="",
command=self.back_to_main,

image=CTkImage(light_image=Image.open("icons/back.png"),
size=(30, 30)), width=20)

# כהה/בהיר) תאורה מצב שינוי
@staticmethod
def change_color_mode(light):
    tk.set_appearance_mode(("light", "dark")[light])

# אגרטל אובייקט הצבת
def place_vase(self):
    self.vase.place(relx=0.51, y=500, anchor=tk.CENTER)

# נצנצים הצגת
def place_glitter(self):
    for i in range(20):
        s = random.randrange(20, 40)
        r = random.randrange(0, 2) * (self.WIDTH - 200)
        Label(self,
image=CTkImage(light_image=Image.open("icons/sparkle.png"), size=(s, s)),
text="").place(
x=random.randrange(1 + r, 200 + r), y=random.randrange(0,
self.HEIGHT))

# משתמש התחברות
def log_in(self, name, password):
    password = hashlib.md5(password.encode()).hexdigest()
    ans = self.client.yell("log_in", (name, password))
    print("deny??", ans)
    if ans == "Not In System" or ans == "Denied":
        self.current_window.wininfo_children()[7].pack()
    else:
        self.open_main_window(name, ans)

# התחברות לאחר הראשי המסך פתיחת
def open_main_window(self, name, permission):
    self.username = name
    self.user_permission = permission
    self.current_window.destroy()
    self.current_window = screens.PrimaryWindow(self, self.username,
self.primary_buttons, self.user_permission)

```

```

self.current_window.pack(pady=20)
self.settings_page = screens.Settings(self)
self.settings_page.place(x=20, y=20)

# משניים למסכים מעבר
def exit_main_window(self):
    self.keep_main = self.current_window
    self.current_window.pack_forget()
    self.vase.place_forget()
    self.settings_page.place_forget()
    self.back_button.place(x=20, y=20)

# לחיצה תיעוד
def click(self):
    self.times.append(time.time())

# חדשה בדיקה מסך פתיחת
def new_test(self):
    self.click()
    self.exit_main_window()
    self.current_window = screens.BrowseProtocolsWindow(self, False,
self.client, self.view_protocol)
    self.current_window.pack(pady=10)

# צוות עריכת מסך פתיחת
def edit_staff(self):
    self.click()
    self.exit_main_window()
    self.current_window = staff.EditStaff(self, self.json)
    self.current_window.pack(pady=40)

# התקדמות מעקב מסך פתיחת
def track_progress(self):
    self.click()
    self.exit_main_window()
    self.current_window = screens.ViewProgress(self, self.client,
self.json)
    self.current_window.pack(pady=20)

# פרוטוקולים עריכת מסך פתיחת
def edit_protocols(self):
    self.click()
    self.exit_main_window()
    self.current_window = screens.BrowseProtocolsWindow(self, True,
self.client, self.edit_protocol_window,
self.new_protocol_window)
    self.current_window.pack(pady=20)

# מסוים פרוטוקול עריכת
def edit_protocol_window(self, name=None, number=None, version=None):
    self.click()
    self.current_window.pack_forget()
    self.gif.pack(pady=100)

```

```

        b = threading.Thread(target=self.creator_help, args=(name, number,
version))
        b.start()

# קלט לפי - לעריכה עוזר
def creator_help(self, name, number, version, text=""):
    if not number:
        self.current_window = creator2.NewProtocol(self,
self.back_to_main, self.json, self.client)
    else:
        self.current_window = creator2.BuildEditor(self, name, number,
version, self.back_to_main, self.json, text,
                                                    self.client)

        self.current_window.pack(pady=20)
        self.gif.pack_forget()

# חדש פרוטוקול פתיחת
def new_protocol_window(self):
    self.click()
    self.last_window = self.current_window
    self.current_window.pack_forget()
    self.current_window = screens.NewProtocol(self,
self.edit_protocol_window, self.ai_edit_protocol_load)
    self.current_window.pack(pady=20)

# n-AI לפרוטוקול טקסט טעינת
def ai_edit_protocol_load(self, text):
    self.current_window.pack_forget()
    self.gif.pack()
    threading.Thread(target=self.ai_edit_protocol,
args=(text,)).start()

# AI בסיס על פרוטוקול עריכת
def ai_edit_protocol(self, text):
    ans = self.client.yell("ask_ai", (text,))
    print(ans)
    self.creator_help("a", "1", "a", ans)

# נתונים הצגת מסך פתיחת
def view_data(self):
    self.click()
    self.exit_main_window()
    self.current_window = view_data.ViewData(self, self.client)
    self.current_window.pack(pady=20)

# מסוים פרוטוקול הצגת
def view_protocol(self, name, number, version):
    self.click()
    self.current_window.pack_forget()
    self.last_window = self.current_window
    self.client.yell("update_status", (name, self.username))
    self.current_window = build.ProtocolView(self, name, number,
version, self.client, self.username, self.back_to_main)
    self.current_window.pack(pady=10)

```

```

# למערכת ההצטרפות עמוד פתיחת
def join(self, times, title):
    self.last_window = self.current_window
    self.last_window.pack_forget()
    self.current_window = log.JoinPage(self, times, self.client,
title)
    self.current_window.pack(pady=20)

# בטוחה בצורה האפליקציה סגירת
def on_closing(self):
    self.destroy()
    if self.client.private != ():
        if type(self.current_window) == build.ProtocolView:
            self.client.yell("update_status", ("", self.username))
            self.client.yell("commit")
        self.client.close()

def main():
    tk.set_appearance_mode("light")
    app = App()
    app.protocol("WM_DELETE_WINDOW", app.on_closing)
    app.mainloop()
# f5b042

```

הקובץ client:

```

import program_files.rsa as rsa
import threading
import win32ui
import socket
import json
import time

# לקוח TCP socket.socket מהמחלקה שמורשת Client מחלקת
class Client(socket.socket):
    def __init__(self):
        super().__init__()
        self.MSG_LENGTH = 1024 # הודעה של מקסימלי אורך
        self.PORT = 1097 # השרת פורט
        self.IP = "127.0.0.1" # השרת של IP-ה כתובת
        self.lock = threading.Lock() # בתהליכונים התנגשויות למניעת מנעול
        # (לשרת לשליחה) שלהן המקוצר לקוד פקודות שמות בין חיפוי
        self.indexes = {
            "get_employees": "man|00", "add_employee": "man|01",
"update_employee": "man|02",
            "delete_employee": "man|03", "add_protocol": "man|04",
"delete_protocol": "man|05",
            "get_progress": "man|06", "get_employee_names": "gen|13",
"get_content": "gen|00",
            "get_protocols": "gen|01", "add_test": "gen|02",
"get_history": "gen|03",
            "get_len_history": "gen|04", "update_status": "gen|05",
"get_last_version": "gen|06",

```

```

        "get_comments": "gen|07", "get_filtered_history": "gen|08",
        "get_4_pages_filtered": "gen|09",
        "commit": "gen|10", "log_in": "log|00",
        "confirm_admission_code": "log|01", "ask_ai": "plx|08",
        "add_password": "gen|11", "get_protocol_names": "gen|12",
        "get_last_history_id": "gen|14"
    }
    # ההצפנה מפתחות לשמירת משתנים
    self.public = ()
    self.private = ()
    self.server_public = ()

    # לשרת וחיבור לבדיקה פונקציה
    def confirm_connection(self):
        self.connect((self.IP, self.PORT)) # לשרת חיבור ניסיון
        ans = ""
        try:
            ans = self.recv(16).decode() # מהשרת אישור קבלת
        except ConnectionResetError:
            collapse() # בכשל טיפול
        return ans

    # (פענוח מכך ולאחר) מוצפנת תשובה וקבלת (הצפנה עם) לשרת פקודה שליחת
    def yell(self, command, arguments=()):
        self.lock.acquire() # thread lock בשימוש התחלת
        msg = self.indexes[command] # המילון לפי הפקודה קוד קבלת
        for a in arguments:
            msg += "|" + str(a) # להודעה הפרמטרים הוספת
        msg = rsa.encrypt(self.server_public, msg) # עם ההודעה הצפנת
        # השרת של הציבורי המפתח
        msg = ','.join([str(x) for x in msg]) # לרשימת ההודעה המרת
        # מחרוזות
        length = len(msg)
        repetition = length // self.MSG_LENGTH # המקטעים כמות חישוב
        # לשליחה
        answer = ""
        try:
            self.send(str(repetition).zfill(3).encode()) # מספר שליחת
            # תווים 3 באורך המקטעים
            # בחלקים ההודעה שליחת
            for i in range(repetition):
                self.send(msg[i * self.MSG_LENGTH:(i + 1) *
                    self.MSG_LENGTH].encode())
            if length % self.MSG_LENGTH != 0:
                self.send(msg[repetition * self.MSG_LENGTH:].encode())
            # מהמשרת התשובה קבלת
            repetition = self.recv(3).decode()
            repetition = int(repetition)
            for i in range(repetition + 1):
                answer += self.recv(self.MSG_LENGTH).decode()
                time.sleep(0.001) # קצרה המתנה
        except ConnectionResetError:
            collapse()
        # התשובה פיענוח

```

```

        answer = answer.split(",")
        answer = [int(n) for n in answer]
        answer = rsa.decrypt(self.private, answer)
        print("answer", answer)
        self.lock.release() # שחרור lock
        return answer

# השרת של המפתח וקבלת לשרת חדש ציבורי מפתח שליחת
def exchange_key(self, var1, var2):
    self.lock.acquire()
    self.public, self.private = rsa.generate_keypair(var1, var2) #
    # חדש ופרטי ציבורי מפתח יצירת
    print("self public key", self.public)
    try:
        self.send("key".encode()) # מפתחות החלפת פקודת שליחת
        self.send(json.dumps(self.public).encode()) # המפתח שליחת
    # JSON בפורמט הציבורי
    self.server_public = self.recv(self.MSG_LENGTH).decode() #
    # השרת של הציבורי המפתח קבלת
    except ConnectionResetError:
        collapse()
    self.server_public = json.loads(self.server_public) # JSON המרת
    # לאובייקט
    print("server public key", self.server_public)
    self.lock.release()

# התוכנית וסיום הודעה הצגת, חיבור כישלון של במקרה
def collapse():
    win32ui.MessageBox("Connection Failed", "Connection Error", 0)
    exit()

```

הקובץ staff:

```

# Import necessary modules and alias for cleaner usage
from customtkinter import CTkLabel as Label, CTkButton as Button, CTkFrame
as Frame, CTkEntry as Entry, \
    CTkImage as CTkImage
from functools import partial
import customtkinter as tk
from PIL import Image
import win32ui
import json

# Custom OptionMenu class with an identifier
class OptionMenu(tk.CTkOptionMenu):
    def __init__(self, master, n, **kwargs):
        super().__init__(master, **kwargs)
        self.id = n

    def get_id(self):
        return self.id

```

```

# Main class for editing staff members
class EditStaff(Frame):
    def __init__(self, master, json_file):
        super().__init__(master)
        # Load employee data from a client call
        self.employees = json.loads(master.client.yell("get_employees",
        )))

        self.len = 0
        self.json_file = json_file
        self.color = json_file['CTkOptionMenu']['fg_color']

        # State variables
        self.mail_editing = -1
        self.name_editing = False

        # Entry fields for name and email
        self.name_entry = Entry(self, width=250)
        self.num_entry = Entry(self, width=250)
        self.name_entry.bind("<Return>", lambda x: self.click(self.len -
1))
        self.num_entry.bind("<Return>", self.collect)

        # UI components and state tracking
        self.names_lbl = []
        self.buttons = []
        self.menus = []
        self.trash = []
        self.updates = {}
        self.new_employees = {}
        self.names = {}
        self.TITLES = ["Worker", "Lab Manager", "Unit Manager"]

        # Icons and buttons
        self.trash_pic =
CTkImage(light_image=Image.open("icons/trash.png"))
        self.new_but = Button(self, text="",
image=CTkImage(light_image=Image.open("icons/New Test.png")), width=20,
command=self.new)
        self.save_button = Button(self, text="Save", command=self.save,
width=20)
        self.error_msg = Label(self, text="This name is already taken",
text_color="red")

        # Initialize UI
        self.create_editor()

        # Create UI elements for existing employees
        def create_editor(self):
            for i in range(3):
                Label(self, text=("Name:", "Gmail:", "Title:")[i],
text_color="#000000",
fg_color=self.json_file['CTkButton']['fg_color'],
width=260).grid(row=0, column=i)

```

```

        for values in self.employees:
            self.len += 1
            lbl = Label(self, text=values[0], fg_color=self.color,
corner_radius=6, width=250,
text_color=self.json_file['CTkButton']['text_color'])
            lbl.grid(row=self.len, column=0)
            self.names[values[0]] = self.len - 1
            self.names_lbl.append(lbl)
            self.end_line(values)
        self.place_constants()

    # Add email field, role selector, and trash button for a row
    def end_line(self, values=(" ", " ", "Worker")):
        b = Button(self, text=values[1], command=partial(self.click,
self.len - 1), fg_color=self.color, border_width=0,
width=250)
        b.grid(row=self.len, column=1)
        self.buttons.append(b)

        o = OptionMenu(self, self.len - 1, values=self.TITLES, width=250)
        o.configure(command=lambda x: self.mark_edited(o.get_id()))
        o.set(values[2])
        o.grid(row=self.len, column=2)
        self.menus.append(o)

        c = Button(self, text="", image=self.trash_pic, width=20,
command=partial(self.delete, self.len - 1))
        c.grid(row=self.len, column=3, pady=3, padx=1)
        self.trash.append(c)

    # Mark a row as edited
    def mark_edited(self, row):
        self.updates[row] = True

    # Place new and save buttons
    def place_constants(self):
        self.new_buttn.grid(row=self.len + 1, column=1)
        self.save_button.grid(row=self.len + 2, column=1, pady=5)

    # Remove a row from the UI and record it for deletion
    def delete(self, row):
        self.collect()
        ans = win32ui.MessageBox(f"Remove
{self.names_lbl[row].cget('text')} from employees?", "Confirm", 1)
        if ans == 1:
            self.buttons[row].grid_forget()
            self.names_lbl[row].grid_forget()
            self.menus[row].grid_forget()
            self.trash[row].grid_forget()
            self.updates[row] = False
            self.names.pop(self.names_lbl[row].cget("text"))

    # Enter edit mode for email

```

```

def click(self, row):
    if self.collect():
        self.error_msg.grid_forget()
        self.mark_edited(row)
        self.mail_editing = row
        self.num_entry.delete(0, "end")
        text = self.buttons[row].cget("text")
        self.buttons[row].grid_forget()
        self.num_entry.insert(0, text)
        self.num_entry.focus()
        self.num_entry.grid(row=row + 1, column=1)
    else:
        self.error_msg.grid(row=self.len + 3, column=1)

# Apply email change
def replace_mail_editing(self):
    text = self.num_entry.get()
    self.num_entry.grid_forget()
    self.buttons[self.mail_editing].configure(text=text)
    self.buttons[self.mail_editing].grid(row=self.mail_editing + 1,
column=1)
    self.mail_editing = -1
    self.num_entry.delete(0, "end")
    return True

# Apply name change
def replace_name_editing(self):
    name = self.name_entry.get()
    if name not in self.names:
        self.name_entry.grid_forget()
        lbl = Label(self, text=name, fg_color=self.color,
corner_radius=6, width=250,
text_color=self.json_file['CTkButton']['text_color'])
        self.names[name] = self.len - 1
        self.names_lbl.append(lbl)
        lbl.grid(row=self.len, column=0)
        self.name_editing = False
        self.name_entry.delete(0, 'end')
    else:
        return False
    return True

# Add a new employee row
def new(self):
    self.collect()
    self.name_editing = True
    self.mail_editing = self.len
    self.new_employees[self.len] = True
    self.updates[self.len] = True
    self.len += 1
    self.name_entry.grid(row=self.len, column=0)
    self.end_line()
    self.name_entry.focus()

```

```

        self.place_constants()

# Apply pending changes if valid
def collect(self, _=None):
    res = True
    if self.name_editing:
        if not self.replace_name_editing():
            res = False
    if self.mail_editing != -1:
        if not self.replace_mail_editing():
            res = False
    print(res)
    return res

# Save all changes by sending commands to the client
def save(self):
    if self.collect():
        update = []
        add = []
        delete = []

        for key in self.updates.keys():
            if self.updates[key]:
                row = (self.names_lbl[key].cget("text"),
self.buttons[key].cget("text"), self.menus[key].get())
                if key in self.new_employees:
                    add.append(row)
                else:
                    update.append(row)
            else:
                delete.append(self.names_lbl[key].cget("text"))

        print("update", update)
        print("add", add)
        print("delete", delete)

        for value in delete:
            self.master.client.yell("delete_employee", (value,))
        for value in add:
            self.master.client.yell("add_employee", value)
        for value in update:
            self.master.client.yell("update_employee", value)

        self.updates = {}
        self.new_employees = {}
        self.master.client.yell("commit")

```

הקובץ :build

```

from customtkinter import CTkCheckBox as Checkbox, CTkScrollableFrame as
ScrollFrame, CTkFrame as Frame, \
    CTkLabel as Label, CTkEntry as Entry, CTkButton as Button
import program_files.reshape as reshape
import customtkinter as tk
import json

```

```

# Represents a single checkbox line with associated label
class CheckLine(Frame):
    i = 0 # Class-level counter to keep track of number of CheckLine
instances
    checked = 0 # Class-level counter for how many checkboxes are checked
    font = ("Arial", 20)

    def __init__(self, master, **kwargs):
        CheckLine.i += 1
        super().__init__(master)
        self.variable = tk.IntVar(value=0)
        self.checkbox = Checkbutton(
            self, variable=self.variable, offvalue=-1, onvalue=1,
command=self.onclick,
            text=str(CheckLine.i), checkbox_width=30, checkbox_height=30
        )
        self.text = kwargs.get("text")
        self.lbl = Label(self, text=self.text, font=self.font)
        self.lbl.grid(row=0, column=1)
        self.checkbox.grid(row=0, column=2, padx=10)

    def onclick(self):
        # Update class-level counter based on checkbox state
        CheckLine.checked += self.variable.get()

# A checkbox line that also includes a text entry (for comments)
class EntryLine(CheckLine):
    def __init__(self, master, **kwargs):
        super().__init__(master, **kwargs)
        self.entry = Entry(self)
        comments = kwargs["comments"]
        if comments:
            self.entry.insert(0, comments[kwargs["i"]])
            self.entry.configure(state="disabled")
        self.entry.grid(row=0, column=0, padx=15)

    def get_txt(self):
        # Retrieve text from the entry field
        return self.entry.get()

# Represents a labeled line with an optional leading symbol (like a
section header)
class LLabel(Frame):
    font = ("Arial", 20)

    def __init__(self, master, **kwargs):
        super().__init__(master)
        self.lbl = Label(self, text=kwargs.get("text"), font=self.font)
        self.header = Label(self, text=kwargs.get("symbol"),
font=("Arial", 30))

```

```

        self.header.grid(row=0, column=1, padx=(25, 90))
        self.lbl.grid(row=0, column=0)

# Dummy placeholder for a callback function
def filler():
    pass

# The main UI class for viewing and interacting with a protocol (document
with checklists, comments, etc.)
class ProtocolView(ScrollFrame):
    def __init__(self, master, name, number, version, client, employee="",
back=filler, comments=None):
        self.values = [name, number, version, employee]
        self.back = back
        self.client = client
        self.entries = [] # Stores EntryLine instances
        self.comments = comments
        CheckLine.i = 0
        CheckLine.checked = 0

        # Fetch content of the protocol (simulate database/API call)
        text = client.yell("get_content", (name, number, version))
        print(text)
        text = text.split("\n")

        super().__init__(master, width=800, height=600)

        # Parse each line of content and generate the appropriate UI
element
        i = 0
        for word in text:
            pref = word[0:3]
            word = word[3:]
            word = reshape.font_parse(word)

            if pref == ".. ":
                c = CheckLine(self, text=word, index=i)
            elif pref == "!! ":
                c = Label(self, text=word, font=("Arial", 30, 'bold',
'underline'))
            elif pref == "?? ":
                c = EntryLine(self, text=word, comments=self.comments,
i=len(self.entries))
                self.entries.append(c)
            elif pref == "*** ":
                c = LLabel(self, symbol="*", text=word)
            else:
                c = LLabel(self, text=word, symbol=" ")

            c.grid(row=i, column=1, sticky='E', pady=10)
            i += 1

```

```

        # Add submit button only if no existing comments are passed (i.e.,
        first time)
        if comments is None:
            self.submit_button = Button(self, text="Submit",
            command=self.submit, fg_color="gray65")
            self.submit_button.grid(row=i, column=1)

            self.columnconfigure(0, weight=1)

        def able(self):
            # Enable submit button only if all checkboxes are checked
            if CheckLine.checked == CheckLine.i:
                self.submit_button.configure(state="normal",
            fg_color="#ffcf4d")
            elif self.submit_button.cget("state") == "normal":
                self.submit_button.configure(state="disabled",
            fg_color="gray65")

        def submit(self):
            # Gather comments and submit the protocol review
            comments = []
            for entry_line in self.entries:
                comments.append(entry_line.get_txt())
            self.values.append(json.dumps(comments))
            self.client.yell("add_test", self.values)
            self.client.yell("commit")
            self.back()

```

הקובץ creator:

```

from customtkinter import CtkScrollableFrame as ScrollFrame, CtkFrame as
Frame, CtkLabel as Label, CtkEntry as Entry, \
    CtkButton as Button, CtkImage as CtkImage
import program_files.reshape as reshape
import customtkinter as tk
from PIL import Image

# Class for displaying editable protocol information fields (Name, Serial
Number, Version)
class Info(Frame):
    def __init__(self, master, values=("", "", "")):
        super().__init__(master)
        title = ("Name:", "Serial Number:", "Version:")
        self.entries = []
        self.current = 0
        for i in range(3):
            Label(self, text=title[i]).grid(row=0, column=i, padx=5)
            entry = EditableBidirectionalLabel(self, width=150,
            text=values[i])
            entry.bind("<Return>", self.next)
            entry.grid(row=1, column=i, padx=5)
            self.entries.append(entry)

        def next(self, _):

```

```

        # Cycle focus through the entry fields
        self.current += 1
        self.entries[self.current % 3].focus()

    def get_values(self):
        # Returns a tuple of input values
        values = (self.entries[0].get(), self.entries[1].get(),
self.entries[2].get())
        return values

# Scrollable container for dynamically building a protocol
class Wrap(ScrollFrame):
    # Mapping of line prefixes to content types
    translator = {".. ": "checkbox", "?? ": "entry", "## ": "comment", "***
": "comment", "!! ": "title"}
    headers = {"comment": "*** ", "title": "!! ", "checkbox": ".. ",
"entry": "?? "}

    def __init__(self, master, back_command, client):
        super().__init__(master, width=800, height=600)
        self.client = client
        self.len = 2
        self.back = back_command
        self.info = Info(self)
        self.info.grid(row=0, column=0, columnspan=2)
        self.img_bg = "white"
        self.columnconfigure(0, weight=1)
        self.submit = Button(self, text="Submit", font=("Arial", 20,
"bold"))
        self.add_button = Button(self, text="Add Row",
command=self.add_line, font=("Arial", 20, "bold"))
        self.lines = [LineMenu(self, 0)]
        self.error_lbl = Label(self)

    def collect(self):
        # Gathers text from each line and applies the appropriate header
        text = ""
        for line in self.lines[1:]:
            if line is not None:
                text += Wrap.headers[line.get_header()] + line.get_text()
+ "\n"
        return text[:-1] # Remove the trailing newline

    def add_line(self):
        # Adds a new line to the protocol editor
        print(self.len)
        self.add_button.grid_forget()
        line = LineMenu(self, self.len - 1, image_bg=self.img_bg)
        self.lines.append(line)
        line.grid(row=self.len - 1, column=1, pady=5, sticky="E")
        self.len += 1
        self.add_button.grid(row=self.len, column=0, columnspan=2, pady=5)
        self.submit.grid(row=self.len + 1, column=0, columnspan=2, pady=5)

```

```

def remove_line(self, row):
    # Removes a line and re-indexes the remaining lines
    self.lines[row].grid_forget()
    for i in range(row, len(self.lines) - 1):
        self.lines[i] = self.lines[i + 1]
        self.lines[i].change_row_lbl(-1)
        self.lines[i].grid(row=i, column=1, pady=5, sticky="E")
    self.lines = self.lines[:-1]

def move(self, move_row, to_row):
    # Moves a line from one position to another
    if move_row == to_row or to_row < 1:
        return False
    if to_row > len(self.lines) + 1:
        self.lines[move_row].set_row_num(move_row)
        return False
    self.lines[move_row].grid_forget()
    if move_row < to_row:
        for i in range(move_row + 1, to_row + 1):
            self.lines[i].change_row_lbl(-1)
            self.lines[i].grid(row=i - 1, column=1, pady=5)
    else:
        for i in range(to_row, move_row):
            self.lines[i].change_row_lbl(1)
            self.lines[i].grid(row=i + 1, column=1, pady=5)
    line = self.lines.pop(move_row)
    line.set_row_num(to_row)
    line.grid(row=to_row, column=1, pady=5, sticky="E")
    self.lines.insert(to_row, line)

def save(self):
    # Saves the protocol if validated
    text = self.collect()
    values = self.info.get_values()
    verified = verify_new_protocol(self.client, values)
    if verified == "Confirmed":
        self.client.yell("add_protocol", (values[0], values[1],
int(values[2]), text))
        self.client.yell("commit")
        self.back()
    else:
        self.error_lbl.configure(text=verified, text_color="red")
        self.error_lbl.grid(row=self.len + 4, column=1)

# Editor for creating a new protocol
class NewProtocol(Wrap):
    def __init__(self, master, back_command, json_file, client):
        super().__init__(master, back_command, client)
        self.img_bg = json_file["CTkOptionMenu"]["fg_color"]
        self.submit.grid(row=2, column=0, pady=10)
        self.add_button.grid(row=1, column=0)
        self.submit.configure(command=self.save)

```

```

# Editor for modifying an existing protocol
class BuildEditor(Wrap):
    def __init__(self, master, name, number, version, back_command,
json_file, text, client=None):
        super().__init__(master, back_command, client)
        self.img_bg = json_file["CTkOptionMenu"]["fg_color"]
        self.exists = False
        if text == "":
            last_version = int(self.client.yell("get_last_version",
(name,))) + 1
            self.exists = True
            text = self.client.yell("get_content", (name, number,
version))
            self.info = Info(self, values=(name, number, last_version))
            self.build(text)
            self.add_button.grid(row=self.len + 1, column=0, columnspan=2,
pady=5)
            self.submit.configure(command=self.save)
            self.submit.grid(row=self.len + 2, column=0, columnspan=2, pady=5)

    def build(self, text):
        # Builds the editor lines from existing text
        lines = text.split("\n")
        i = 0
        for line in lines:
            line_type = line[:3]
            if line_type in Wrap.translator:
                line = line[3:]
                line = reshape.font_parse(line)
                line_object = LineMenu(self, i + 1, text=line,
image_bg=self.img_bg,
line_type=Wrap.translator[line_type])
                line_object.grid(row=i + 1, column=1, pady=5, sticky="E")
                self.lines.append(line_object)
                self.len += 1
                i += 1

def no_command(_):
    pass

# Editable label component that switches between entry and button view
class EditableBidirectionalLabel(Frame):
    def __init__(self, master, text="", width=10,
enter_command=no_command, font=("Arial", 20)):
        super().__init__(master)
        self.enter_command = enter_command
        self.button = Button(self, command=self.click, text=text,
width=width, font=font)
        self.entry = Entry(self, width=width, font=font)

```

```

        self.entry.insert(0, text)
        self.entry.bind("<Return>", self.enter)
        self.entry.bind("<FocusOut>", self.enter)
        self.button.pack()

    def click(self):
        self.button.pack_forget()
        self.entry.pack()
        self.entry.focus()

    def enter(self, _):
        self.entry.pack_forget()
        t = self.entry.get()
        self.button.configure(text=t)
        self.button.pack()
        self.enter_command(t)

    def set_font(self, font):
        self.button.configure(font=font)
        self.entry.configure(font=font)

    def set(self, value):
        self.entry.delete(0, "end")
        self.entry.insert(0, value)
        self.button.configure(text=value)

    def get(self):
        return self.entry.get()

# A configurable line in the protocol editor
class LineMenu(Frame):
    def __init__(self, master, row, text="", image_bg="#FFFFFF",
line_type="checkbox"):
        super().__init__(master)
        if line_type == "title":
            self.font = ("Arial", 30, "underline", "bold")
        else:
            self.font = ("Arial", 20)
        self.header = line_type
        img = CTkImage(light_image=Image.open(f"icons/{line_type}.png"),
size=(30, 30))
        self.image = Label(self, text="", bg_color=image_bg, image=img)
        self.var = tk.StringVar()
        self.trash_button = Button(self, text="",

image=CTkImage(light_image=Image.open("icons/trash.png"), size=(15, 15)),
width=20,

                                command=self.remove)
        self.options = tk.CTkOptionMenu(self, height=30,
variable=self.var, command=self.clear, width=0,
                                values=["checkbox", "entry",
"comment", "title"], corner_radius=0)
        self.row_number = row

```

```

        self.row_lbl = EditableBidirectionalLabel(self,
text=self.row_number, enter_command=self.change_place, width=40)
        self.content = EditableBidirectionalLabel(self, width=250,
text=text, font=self.font)
        self.place_it()

    def place_it(self):
        self.content.pack(side=tk.LEFT, padx=5)
        self.image.pack(side=tk.LEFT)
        self.options.pack(side=tk.LEFT)
        self.row_lbl.pack(side=tk.LEFT, padx=5)
        self.trash_button.pack(side=tk.LEFT, padx=5)

    def clear(self, line_type):
        self.var.set("")

self.image.configure(image=CTkImage(light_image=Image.open(f"icons/{line_t
ype}.png"), size=(30, 30)))
        if line_type == "title":
            self.content.set_font(("Arial", 30, "bold", "underline"))
        else:
            self.content.set_font(("Arial", 20))
        self.header = line_type

    def remove(self):
        self.master.remove_line(self.row_number)

    def change_row_lbl(self, num):
        self.row_number += num
        self.row_lbl.set(self.row_number)

    def change_place(self, x):
        self.master.move(self.row_number, int(x))

    def set_row_num(self, n):
        self.row_number = n
        self.row_lbl.set(self.row_number)

    def id(self):
        return self.row_number

    def get_header(self):
        return self.header

    def get_text(self):
        return self.content.get()

def verify_new_protocol(client, values):
    if not values or values[0] == "" or values[1] == "":
        return "Add Headers."
    values = list(values)
    if not values[2].isnumeric():
        return "Version must be a number."
    lst = json.loads(client.yell("get_protocols"))

```

```

for protocol in lst:
    print(protocol, values)
    if values == protocol:
        return "This protocol already exists."
    if values[0] == protocol[0] and not values[1] == protocol[1]:
        return "Protocol Number does not match Name"
    if values[1] == protocol[1] and not values[0] == protocol[0]:
        return "Protocol Name does not match Number."
return "Confirmed"

```

הקובץ :rsa_encryption

```

import re
import random

# Euclid's algorithm for determining the greatest common divisor
# Use iteration to make it faster for larger integers
def gcd(a, b):
    while b != 0:
        temp = a
        a = b
        b = temp % a
    return a

# Python program to demonstrate working of extended
# Euclidean Algorithm

# function for extended Euclidean Algorithm
def multiplicative_inverse(e, phi):
    d = 0
    x1 = 0
    x2 = 1
    y1 = 1
    temp_phi = phi

    while e > 0:
        temp1 = temp_phi // e
        temp2 = temp_phi - temp1 * e
        temp_phi = e
        e = temp2

        x = x2 - temp1 * x1
        y = d - temp1 * y1

        x2 = x1
        x1 = x
        d = y1
        y1 = y

    if temp_phi == 1:
        return d + phi

```

```

# check if number is prime
def is_prime(num):
    if num == 2:
        return True
    if num < 2 or num % 2 == 0:
        return False
    for n in range(3, int(num ** 0.5) + 2, 2):
        if num % n == 0:
            return False
    return True

# generate a pair of public and private rsa keys from 2 prime numbers
def generate_keypair(var1, var2):
    p, q = generate_primes(var1, var2)
    # n = pq
    n = p * q

    # Phi is the totient of n
    phi = (p - 1) * (q - 1)

    # print('n=', n, ' phi= ', phi)

    # Choose an integer e such that e and phi(n) are coprime
    e = random.randrange(2, phi)

    # Use Euclid's Algorithm to verify that e and phi(n) are coprime
    g = gcd(e, phi)
    while g != 1:
        e = random.randrange(2, phi)
        g = gcd(e, phi)
    # Use Extended Euclid's Algorithm to generate the private key
    d = multiplicative_inverse(e, phi)

    # Return public and private keypair
    # Public key is (e, n) and private key is (d, n)
    return (e, n), (d, n)

# encrypt message with public key
def encrypt(pk, plaintext):
    # Unpack the key into it's components
    key, n = pk
    # Convert each letter in the plaintext to numbers based on the
    character using a^b mod m
    cipher = [pow(ord(char), key, n) for char in plaintext]
    # Return the array of bytes
    return cipher

# decrypt a message with private key
def decrypt(pk, ciphertext):
    # Unpack the key into its components
    key, n = pk

```

```

# Generate the plaintext based on the ciphertext and key using a^b mod
m
plain = [chr(pow(char, key, n)) for char in ciphertext]
# Return the array of bytes as a string
return ''.join(plain)

# force a number to be within a certain range
def within_range(n):
    while n >= 1000000:
        n = n // 5
    return int(n)

# generates prime numbers from random variables within a certain range
def generate_primes(var1, var2):
    n1 = within_range(random.randrange(10000) * var1 + var1 % 3)
    prime1 = lst[n1]
    n2 = within_range(random.randrange(10000) * var2 + var2 % 3)
    while n1 == n2:
        n2 = within_range(random.randrange(10000) * var2 + var1 % 3)
    prime2 = lst[n2]
    return int(prime1), int(prime2)

# generates two random numbers from values
def generate_randoms(v1, v2):
    d1 = v1 % 10
    d2 = v2 % 10
    n1 = d1 * (v1 % (10 ** d1))
    n2 = d2 * (v2 % (10 ** d2))
    return n1, n2

f = open(r"C:\Users\USER\OneDrive\שולחן העבודה\פרוייקט\העבודה\גמר\primes2.txt",
        "r")
numstxt = f.read()
lst = re.split(" | \n ", numstxt)

if __name__ == '__main__':
    pu, pr = generate_keypair(7, 13)
    print(pu)
    print(pr)

```

הקובץ log:

```

from customtkinter import CTkLabel as Label, CTkButton as Button, CTkEntry
as Entry, CTkFrame as Frame, CTkImage as CTkImage
from PIL import Image
import hashlib
import time
import program_files.rsa_encryption as rsa
import re

```

```

# Login window for existing users
class LoginWindow(Frame):
    def __init__(self, master, client):
        self.times = [time.time()] # Track input timings for random
number generation
        super().__init__(master, width=780, height=600)
        self.client = client

        # components for login
        Label(self, text="Yaelotron", font=("Roberto", 40)).pack(pady=50)
        Label(self, text="Name:").pack()
        self.name = Entry(self)
        self.name.pack()
        self.name.bind("<FocusIn>", self.user_click)
        self.name.bind("<Return>", self.next_entry)
        self.name.focus_set()

        Label(self, text="Password:").pack()
        self.password = Entry(self, show="*")
        self.password.pack()
        self.password.bind("<FocusIn>", self.user_click)
        self.password.bind("<Return>", self.submit)

        # Login, join, and forgot password buttons
        Button(self, text="Log In", command=lambda:
self.submit(0)).pack(pady=10)
        Button(self, text='Join', fg_color="gray50", width=100,
font=("Roberto", 10), command=self.join).pack(pady=2)
        Button(self, text="Forgot Password", fg_color="gray50", width=100,
font=("Roberto", 10), command=self.change_password).pack(pady=2)

        # Error label for incorrect login
        Label(self, text="Wrong name or password", text_color='#ff0004',
font=("Ariel", 15))
        self.rowconfigure(8, weight=1)

    def user_click(self, _):
        # On 3 input events, generate random values for secure key
exchange
        self.times.append(time.time())
        if len(self.times) == 3:
            var1, var2 = rsa.generate_randoms(self.times[1] -
self.times[0], self.times[2] - self.times[1])
            self.client.exchange_key(var1, var2)

    def next_entry(self, _):
        self.password.focus_set()

    def submit(self, _):
        # Submit login info
        name = self.name.get()
        password = self.password.get()
        if name != "" and password != "":

```

```

        self.master.log_in(name, password)

    def join(self):
        # register
        self.user_click("")
        self.master.join(self.times, "Join Yaelotron")

    def change_password(self):
        # Start password reset process
        self.user_click("")
        self.master.join(self.times, "Change Password")

# Registration and password reset page
class JoinPage(Frame):
    def __init__(self, master, times, client, title):
        super().__init__(master)
        self.client = client
        self.times = times
        self.permission = "Worker"
        self.title = Label(self, text=title, font=("Roboto", 30))
        self.title.pack(pady=5)
        self.entry = Entry(self, placeholder_text="Username")
        self.entry.bind("<FocusIn>", self.user_click)
        self.entry.bind("<Return>", self.click)
        self.entry.pack(pady=5)
        self.button = Button(self, text="Submit",
command=self.check_username)
        self.button.pack(pady=5)
        self.error_lbl = Label(self)
        # Password fields
        self.confirm_entry = Entry(self, placeholder_text="password",
show="*")
        self.confirm_entry.bind("<Return>", self.next)
        self.name = ""
        self.code = 0
        # Back and resend buttons
        self.back_button = Button(self.master, text="",
image=CTkImage(light_image=Image.open("icons/back.png")),
command=self.back, width=30)
        self.back_button.place(x=50, y=50)
        self.resend_button = Button(self, text="Resend",
command=self.send_code)

    def back(self):
        # Return to login window
        self.back_button.place_forget()
        self.master.back_to_log()

    def user_click(self, _=""):
        # Handle input timing for key generation
        self.times.append(time.time())
        if len(self.times) == 3:
            var1, var2 = rsa.generate_randoms(self.times[1] -

```

```

self.times[0], self.times[2] - self.times[1])
    self.client.exchange_key(var1, var2)

def click(self, _):
    # Execute current button command
    self.button.cget("command") ()

def next(self, _):
    self.entry.focus()

def send_code(self):
    # Ask server to send verification code
    return self.master.client.yell("log_in", (self.name, "send_mail"))

def check_username(self):
    # Verify if username is valid and send verification code
    self.user_click()
    self.name = self.entry.get()
    self.permission = self.send_code()
    self.error_lbl.pack_forget()
    self.title.configure(text="A gmail with a verification code is
being sent to you.")
    self.entry.configure(placeholder_text="code")
    self.entry.delete(0, "end")
    self.button.configure(command=self.confirm_code)
    self.button.focus()
    self.resend_button.pack(pady=10)

def confirm_code(self):
    # Validate code sent to user's email
    code = self.entry.get()
    ans = self.client.yell("confirm_admission_code", (self.name,
code))
    print("ans", ans)
    if ans == "Confirmed":
        self.resend_button.pack_forget()
        self.create_password()
    else:
        self.error_lbl.configure(text=ans)
        self.error_lbl.pack(pady=5)

def create_password(self):
    # UI for creating and confirming password
    self.title.configure(text="Create Password:")
    self.error_lbl.pack_forget()
    self.entry.configure(placeholder_text="confirm password",
show="*")
    self.entry.delete(0, "end")
    self.entry.pack_forget()
    self.button.pack_forget()
    self.confirm_entry.pack(pady=5)
    self.entry.pack(pady=5)
    self.button.pack(pady=5)
    self.button.focus()

```

```

self.button.configure(command=self.save)

def save(self):
    # Validate and save new password
    password = self.entry.get()
    if re.search("^(?=.*?[A-Z])(?=.*?[a-z])(?=.*?[0-9])(?=.*?[#?!@$%^&*~]).{8,}$", password):
        if password == self.confirm_entry.get():
            self.back_button.place_forget()
            self.error_lbl.pack_forget()
            hashed = hashlib.md5(password.encode()).hexdigest()
            self.master.client.yell("add_password", (self.name, hashed))

            self.destroy()
            self.master.open_main_window(self.name, self.permission)
        else:
            self.error_lbl.configure("Passwords Do Not Match")
            self.error_lbl.pack()
    else:
        # Show error message for weak passwords
        self.error_lbl.configure(
            text="Has minimum 8 characters in length\nAt least one uppercase English letter"
            "\nAt least one lowercase English letter\nAt least one digit\nAt least one special character")
        self.error_lbl.pack()

```

הקובץ :reshape

```

# check if text is in Hebrew
def is_hebrew(word: str):
    if not word:
        return 1
    if 'א' <= word[0] <= 'ת' or 'א' <= word[-1] <= 'ת' or word[0].isnumeric() and word[-1].isnumeric():
        return 1
    return -1

# reshape multiple lines
def multi_parse(paragraph: str):
    par = paragraph.split("\n")
    for i in range(len(par)):
        par[i] = parse(par[i]) + "\n"
    return "".join(par)[:-1]

# reshape a sentence that is to be displayed without changing the default font
def parse(sentence: str):
    words = sentence.split(" ")
    partial_join = []
    current_sequence = []
    hebrew = is_hebrew(words[0])
    for word in words:
        if is_hebrew(word) * hebrew == 1:

```

```

        current_sequence.append(word)
    else:
        hebrew *= -1
        partial_join.append(current_sequence)
        current_sequence = [word]
    partial_join.append(current_sequence)
    for i in range(len(partial_join)):
        if is_hebrew(partial_join[i][0]) == -1:
            partial_join[i].reverse()
        partial_join[i] = " ".join(partial_join[i])
    partial_join.reverse()
    return " ".join(partial_join)

# # reshape a sentence that is to be displayed with a different font
def font_parse(sentence: str):
    sequences = []
    words = sentence.split(" ")
    current = words[0]
    lang = is_hebrew(words[0])
    for i in range(1, len(words)):
        if is_hebrew(words[i]) * lang == 1:
            current += " " + words[i]
        else:
            lang *= -1
            sequences.append(current)
            current = words[i]
    sequences.append(current)
    sequences.reverse()
    return ' '.join(sequences)

```

הקובץ :screens

```

from customtkinter import CTKLabel as Label, CTKButton as Button, CTKImage
as CTKImage, CTKFrame as Frame, \
    CTKScrollableFrame as ScrollFrame, CTKOptionMenu as OptionMenu,
CTKSwitch as Switch, filedialog
from CTKTable import CTKTable as Table
from functools import partial
from PIL import Image
import pandas as pd
import datetime
import win32ui
import json
import re

# Main user interface window shown after login
class PrimaryWindow(Frame):
    icon_size = 70

    def __init__(self, master, name, commands, role):
        super().__init__(master, width=780, height=600)
        self.name = name
        self.commands = commands
        self.role = role

```

```

        # Display greeting based on time of day
        h = int(datetime.datetime.now().hour)
        if h < 13:
            h = "Morning"
        elif h < 18:
            h = "Afternoon"
        else:
            h = "Evening"
        Label(self, text=f"Good {h} {self.name}!", font=("Roboto",
40)).grid(row=0, column=0, columnspan=5, pady=70,
padx=20)
        # Load appropriate UI depending on user's role
        if role.endswith("Manager"):
            self.grid_manager_buttons()
        else:
            self.grid_employee_buttons()

        # Display all manager options
        def grid_manager_buttons(self):
            icons = ["Edit Protocols.png", "Track Progress.png", "New
Test.png", "Edit Staff.png", "View Data.png"]
            for i in range(5):
                Button(self,
image=CTkImage(light_image=Image.open(f"icons/{icons[i]}"),
size=(self.icon_size, self.icon_size)),
width=50, command=self.commands[i],
text="").grid(column=i, row=i % 2 * 2 + 1, padx=10)
                Label(self, text=icons[i][:4], font=("Roboto",
15)).grid(column=i, row=i % 2 * 2 + 2)

        # Display limited options
        def grid_employee_buttons(self):
            Button(self,
image=CTkImage(light_image=Image.open(f"icons/New
Test.png"), size=(self.icon_size, self.icon_size)),
width=50, command=self.commands[2], text="").grid(column=1,
row=2, pady=(30, 0))
            Button(self,
image=CTkImage(light_image=Image.open(f"icons/View
Data.png"), size=(self.icon_size, self.icon_size)),
width=50, command=self.commands[4], text="").grid(column=3,
row=2, pady=(30, 0))
            Label(self, text="New Test", font=("Roboto", 15)).grid(column=1,
row=3)
            Label(self, text="View Data", font=("Roboto", 15)).grid(column=3,
row=3)
            self.columnconfigure((0, 2, 4), weight=1)

# Settings page to toggle color theme and light/dark mode
class Settings(Frame):
    light = True

```

```

current_color = "Yellow"
bg = ['Yellow', 'Blue', 'Pink', 'Green', 'Orange', 'autumn', 'violet']

def __init__(self, master):
    super().__init__(master)
    self.color_picker = OptionMenu(self, values=Settings.bg)
    self.appearance = Switch(self, text="Light", onvalue=0,
offvalue=1)
    self.color_picker.set(Settings.current_color)
    if Settings.light:
        self.appearance.select()
    self.color_picker.configure(command=lambda x:
self.color_clicked(x))
    self.appearance.configure(command=self.appearance_clicked)
    self.color_picker.pack()
    self.appearance.pack()

def appearance_clicked(self):
    Settings.light = not Settings.light
    self.master.change_color_mode(self.appearance.get())

def color_clicked(self, color):
    Settings.current_color = color
    self.master.change_color_theme(color)

# error window if server rejects client due to a DDoS attack
class ErrorWindow(Frame):
    def __init__(self, master, command):
        super().__init__(master)
        Label(self, text="Connection Failed").pack(pady=20)
        Button(self, text="Retry", command=command).pack(pady=20)

# Scrollable view to browse/edit/delete test protocols
class BrowseProtocolsWindow(ScrollFrame):
    def __init__(self, master, edit, client, command, new_command=None):
        super().__init__(master, width=600, height=600)
        self.client = client
        protocols = json.loads(master.client.yell("get_protocols", ()))
        length = 0
        self.buttons = []
        self.trash = []
        for name, number, version in protocols:
            # Protocol button
            b = Button(self, text=name + " v" + str(version), width=550,
                        command=partial(command, name, number, version))
            self.buttons.append(b)
            b.grid(row=length, column=0, sticky="EW", pady=5, padx=5)
            if edit:
                # Delete button if in edit mode
                b = Button(self, text="",
image=CTkImage(light_image=Image.open("icons/trash.png")),
                        command=partial(self.delete, length, name,

```

```

version), width=10)
        self.trash.append(b)
        b.grid(row=length, column=1)
        length += 1
    if edit:
        Button(self, text="",
image=CTkImage(light_image=Image.open("icons/plus.png")),
command=new_command).grid(
        row=length, column=0, pady=5)

    def delete(self, index, name, version):
        ans = win32ui.MessageBox(f"Delete {name} Version {version}?",
"Confirm", 1)
        if ans == 1:
            self.client.yell("delete_protocol", (name, version))
            self.buttons[index].grid_forget()
            self.trash[index].grid_forget()
            self.client.yell("commit")

# View and refresh table showing employee progress on protocols
class ViewProgress(Frame):
    def __init__(self, master, client, json_file):
        self.client = client
        values = [("Employee:", "Working On:")]
        values.extend(json.loads(self.client.yell("get_progress", ())))
        super().__init__(master)
        self.table = Table(self, values=values,
                           colors=[json_file["CTk"]["fg_color"],
json_file["CTkFrame"]["fg_color"]])
        self.table.pack(padx=10, pady=10)
        Button(self, text="Reload", command=self.reload).pack()

    def reload(self):
        values = [("Employee:", "Working On:")]
        values.extend(json.loads(self.client.yell("get_progress", ())))
        self.table.configure(values=values)

# get rid of redundant empty lines
def sort_file(t):
    i = 0
    t = t[t.find("\n"):]
    while re.match("[0-9Nan \n]", t[i]):
        i += 1
    return i

def sort_file_back(t):
    length = len(t)
    i = 0
    t = t[:t.rfind("\n")]
    while re.match("[0-9Nan \n]", t[-i]):
        i += 1

```

```

        return length - i

# Window to allow creation of new protocol either from scratch or existing
file
class NewProtocol(Frame):
    def __init__(self, master, new_command, ai_command):
        super().__init__(master)
        self.ai_command = ai_command
        Button(self, text="Create New Protocol", font=("Roberto", 20,
"bold"), command=new_command).pack(pady=20,
padx=20)
        Button(self, text="Create New Protocol From Existing File",
font=("Roberto", 20, "bold"),
            command=self.ai_chosen).pack(pady=20, padx=20)

    def ai_chosen(self):
        filename = filedialog.askopenfilename(filetypes=[("Excel files",
"*.xlsx"), ("Word Files", "*.docx")])
        if filename != "":
            sheet_names = pd.ExcelFile(filename).sheet_names
            for n in sheet_names:
                Button(self, text=n, command=partial(self.get_sheet,
filename, n)).pack(pady=10)

    def get_sheet(self, filename, sheet_name):
        fd = pd.read_excel(filename, sheet_name)
        pd.set_option('display.max_columns', None)
        df = fd.head(60)
        t = str(df)
        t = t[sort_file(t):sort_file_back(t)]
        self.ai_command(t)

# Label widget that animates a GIF
class GIFLabel(Label):
    def __init__(self, master, image_path, **kwargs):
        self._gif_image = Image.open(image_path)
        kwargs.setdefault("width", self._gif_image.width)
        kwargs.setdefault("height", self._gif_image.height)
        kwargs.setdefault("text", "")
        self._duration = kwargs.pop("duration", None) or
self._gif_image.info["duration"]
        super().__init__(master, **kwargs)
        self._frames = []
        for i in range(self._gif_image.n_frames):
            self._gif_image.seek(i)
            self._frames.append(CTkImage(self._gif_image.copy(),
size=(self["width"], self["height"])))
            self._animate()

    def _animate(self, idx=0):
        self.configure(image=self._frames[idx])

```

```

        self.after(self._duration, self._animate, (idx + 1) %
len(self._frames))

```

הקובץ `:view_data`

```

from customtkinter import CTKLabel as Label, CTKFrame as Frame,
CTKOptionMenu as OptionMenu, CTKTabview as Tabview, \
    CTKButton as Button, CTKEntry as Entry
from CTKTable import CTKTable as Table
import program_files.build as build
import customtkinter as tk
import win32ui
import json
import re

# The View History page.
class ViewData(Frame):
    def __init__(self, master, client):
        super().__init__(master)
        self.client = client # communication object
        self.has_next = True # are there currently more pages available
to load
        self.data_pages = Data(self, self.get_data) # TabView object that
contains the tables of data
        self.data_pages.grid(row=0, column=0, columnspan=4)
        self.filters = [] # current active filter objects
        self.last_id = json.loads(client.yell("get_last_history_id")) #
last id number in the history table
        self.apply_button = Button(self, text="apply", command=self.apply)
# button to apply filters
        self.add_button = Button(self, text="Add Filter",
command=self.add_filter) # button to add filter
        self.add_button.grid(row=1, column=0)
        self.apply_button.grid(row=2, column=0)
        self.build_tables()

        # make initial pages show
    def build_tables(self):
        data, n = self.get_data(True)
        self.data_pages.build_pages(data, n)

        # create new Filter object
    def add_filter(self):
        if len(self.filters) < 3:
            f = Filter(self, self.client)
            self.filters.append(f)
            f.grid(row=1, rowspan=2, column=len(self.filters))

        # is_new: True if the tables being built are because the screen just
opened or a new filter was applied
        # False if they're being built due to "Next" button
        # gets all info from filters and gets data from server
        # returns data and number of pages
    def get_data(self, is_new):
        if self.has_next or is_new:

```

```

        filters = []
        values = []
        for i in range(len(self.filters)):
            filter_name = self.filters[i].get_filter()
            filter_value = self.filters[i].get_value()
            if filter_name != "No Filter" and filter_value is not
None:
                filters.append(filter_name)
                values.append(filter_value)
        # data = db_worker.get_4_pages_filtered(self.last_id, filters,
values)

        data = json.loads(
            self.client.yell("get_4_pages_filtered", (self.last_id,
json.dumps(filters), json.dumps(values))))
        if len(data) == 0:
            return None, 0
        self.last_id = data[-1][0]
        n = len(data) // 12
        if len(data) % 12 != 0:
            n += 1
        if n < 4:
            self.has_next = False
        return data, n
    return None, 0

# reload the Data object according to new filters
def apply(self):
    self.last_id = json.loads(self.client.yell("get_last_history_id"))
    self.has_next = True
    self.data_pages.clear()
    self.build_tables()

# TabView object that always has on 4 current pages, with "Previous" and
"Next" buttons to navigate
class Data(Tabview):
    def __init__(self, master, get_data_command):
        super().__init__(master, command=self.click)
        self.NUM = 4 # number of pages available on screen at any moment
        self.get_data_command = get_data_command # get_data command from
ViewData class
        self.built_pages = [] # list of CTkTable objects built for each
page in order
        self.current_first = 1 # keeps the current first page with a
button available on the screen
        self.current_num_of_pages = self.NUM # how many pages on screen
currently have a table on them (are not empty)
        # create all buttons for the TabView object
        self.add("Previous")
        for i in range(self.NUM):
            self.add(f"page{i + 1}")
            Label(self.tab(f"page{i + 1}"))
        self.add("Next")
        self.set("page1")

```

```

    # data: 2d list of history values from database. num_of_pages: amount
    of pages to create with data
    # creates all tables and places them on their pages
    def build_pages(self, data, num_of_pages):
        self.current_num_of_pages = num_of_pages
        for t in self.built_pages[self.current_first - self.NUM -
1:self.current_first]:
            t.pack_forget()
            if self.current_num_of_pages != 0:
                for i in range(self.current_first, self.current_first +
num_of_pages - 1):
                    t = Table(self.tab(f"page{i}"),
                                values=data[(i - self.current_first) * 12: (i -
self.current_first + 1) * 12],
                                command=self.open_protocol_view)
                    t.pack()
                    self.built_pages.append(t)
                t = Table(self.tab(f"page{self.current_first + num_of_pages -
1}"), values=data[(num_of_pages - 1) * 12:],
                            command=self.open_protocol_view)
                t.pack()
                self.built_pages.append(t)

    # cell: information about the cell that was pressed in the table
    # opens TopLevel screen with the protocol that was clicked, with the
    comments that were written on it.
    def open_protocol_view(self, cell):
        # table = self.tab(self.get()).wininfo_children()[0]
        table = self.built_pages[int(self.get()[4:]) - 1]
        row = cell["row"]
        row_id = table.get(row=row, column=0)
        protocol_id = table.get(row=row, column=1)
        protocol = table.get(row=row, column=2)
        version = table.get(row=row, column=3)
        comments = self.master.client.yell("get_comments", (row_id,))
        if comments != "None":
            comments = json.loads(comments)
        else:
            comments = []
        w1 = tk.CTkToplevel(self)
        w1.geometry("600x600")
        w1.after(100, w1.lift)
        print(type(comments))
        if protocol_exists(self.master.client, [protocol, protocol_id,
version]):
            p = build.ProtocolView(w1, protocol, protocol_id, version,
self.master.client, comments=comments)
            p.pack()
        else:
            Label(w1, text=f"This Protocol Has Been
Deleted.\n\nComments:\n{comments}").pack()

    # when TabView segmented button is clicked

```

```

def click(self):
    if self.get() == "Next":
        self.next()
    elif self.get() == "Previous":
        self.previous()

# if the tables have already been built, place them
# if they haven't, build them
def next(self):
    if len(self.built_pages) > self.current_first + self.NUM - 1:
        self.place_next()
    else:
        self.build_next()
    self.set(f"page{self.current_first}")

# get the data and build pages if there's data
def build_next(self):
    data, n = self.get_data_command(False)
    if data is not None:
        for i in range(self.NUM):
            self.rename(f"page{self.current_first + i}",
f"page{self.current_first + i + 4}")
            self.current_first = self.current_first + self.NUM
            self.build_pages(data, n)

# place the next built tables
def place_next(self):
    for i in range(self.NUM):
        self.built_pages[i + self.current_first - 1].pack_forget()
    for i in range(self.current_first + self.NUM - 1,
self.current_first + 2 * self.NUM - 1):
        self.rename(f"page{i - self.NUM + 1}", f"page{i + 1}")
    i = self.current_first + self.NUM - 1
    while i < self.current_first + 2 * self.NUM - 1 and i <
len(self.built_pages):
        self.built_pages[i].pack()
        i += 1
    self.current_num_of_pages = i - self.current_first - self.NUM + 1
    self.current_first = self.current_first + self.NUM

# place the previous tables if there are any
def previous(self):
    n = self.current_num_of_pages
    if self.current_first > 1:
        for i in range(n):
            self.built_pages[i + self.current_first - 1].pack_forget()
        for i in range(self.NUM):
            self.rename(f"page{self.current_first + i}",
f"page{self.current_first + i - self.NUM}")
            self.built_pages[self.current_first + i - 1 -
self.NUM].pack()
            self.current_first = self.current_first - self.NUM
            self.move(self.NUM + 1, "Next")
        self.current_num_of_pages = self.NUM

```

```

        self.set(f"page{self.current_first}")

        # delete all tables, to prepare for new ones with new applied filters
        def clear(self):
            print("first", self.current_first, "num",
self.current_num_of_pages)
            i = 0
            while i < self.NUM and len(self.built_pages) >= self.current_first
+ i:
                self.built_pages[self.current_first + i - 1].pack_forget()
                i += 1
            # for i in range(self.current_first, self.current_first +
self.current_num_of_pages):
                if self.current_first != 1:
                    for i in range(self.current_first, self.current_first +
self.NUM):
                        self.rename(f"page{i}", f"page{i - self.current_first +
1}")

            self.current_first = 1
            self.set("page1")
            self.built_pages = []

# a single filter object has a category and value
class Filter(Frame):
    def __init__(self, master, client):
        super().__init__(master)
        self.client = client
        self.values = {"Employee": []}
        self.date = False
        self.filter_menu = OptionMenu(self, values=["No Filter",
"Employee", "Protocol", "Date Range"],
                                     command=self.first_filter_by)

        self.filter_menu.pack(pady=5)
        self.secondary_filter = OptionMenu(self)
        self.from_lbl = Label(self, text="From:", font=("Roberto", 15),
height=15)
        self.to_lbl = Label(self, text="To:", font=("Roberto", 15),
height=15)
        self.start_date_entry = Entry(self, placeholder_text="YYYY-MM-DD")
        self.end_date_entry = Entry(self, placeholder_text="YYYY-MM-DD")

        # arg: value chosen in the OptionMenu
        # what to do when the filter's category is chosen
        # loads possible values into the next OptionMenu or places Entries
        def first_filter_by(self, arg):
            if arg == "No Filter":
                self.secondary_filter.pack_forget()
            if self.date:
                self.exit_date_range()
            if arg == "Employee":
                employees = self.client.yell("get_employee_names")
                employees = json.loads(employees)
                employees = [x[0] for x in employees]

```

```

        self.secondary_filter.configure(values=employees)
        self.secondary_filter.set(employees[0])
        self.secondary_filter.pack(pady=5)
    elif arg == "Protocol":
        protocols = self.client.yell("get_protocol_names")
        protocols = json.loads(protocols)
        protocols = [x[0] for x in protocols]
        self.secondary_filter.configure(values=protocols)
        self.secondary_filter.set(protocols[0])
        self.secondary_filter.pack(pady=5)
    elif arg == "Date Range":
        self.date = True
        self.from_lbl.pack()
        self.start_date_entry.pack()
        self.to_lbl.pack()
        self.end_date_entry.pack()

# remove date range entries and labels
def exit_date_range(self):
    self.date = False
    self.from_lbl.pack_forget()
    self.start_date_entry.pack_forget()
    self.to_lbl.pack_forget()
    self.end_date_entry.pack_forget()

# returns the filter category name
def get_filter(self):
    return self.filter_menu.get()

# return the value of the filter, if it is valid, in the way it should
# go in the sqlite query
# note that the server verifies the data to prevent sql injection
def get_value(self):
    if not self.date:
        return f"'{self.secondary_filter.get()}'"
    date_from = self.start_date_entry.get()
    date_to = self.end_date_entry.get()
    time_format = "[0-9]{4}(-[0-1][0-9](-[0-3][0-9])?)?" # 'YYYY-MM-
DD' or 'YYYY-MM' or 'YYYY'
    if re.fullmatch(time_format, date_from) and
re.fullmatch(time_format, date_to):
        ans = f"'{date_from}'" + " and " + f"'{date_to}'"
        print("CONFIRM")
        return ans
    win32ui.MessageBox("Make sure you're using YYYY-MM-DD format!",
"ok", 1)
    return None

# check if a certain protocol currently exists
def protocol_exists(client, values):
    protocols = json.loads(client.yell("get_protocols"))
    for p in protocols:
        print(p)

```

```

        if p == values:
            return True
    return False

```

הקובץ server:

```

import program_files.db_worker as db_worker
from email.message import EmailMessage
import program_files.rsa as rsa
from dotenv import load_dotenv
from google import genai
import threading
import hashlib
import smtplib
import socket
import select
import random
import time
import json
import os

# Load environment variables from .env file
load_dotenv("hey.env")

# Database functions - wrap db_worker functions
def get_user(name):
    return json.dumps(db_worker.get_user(name[0]))

def get_employees(_):
    return json.dumps(db_worker.get_employees())

def get_content(values):
    name, number, version = values
    return db_worker.get_content(name, number, version)

def get_protocols(_):
    return json.dumps(db_worker.get_protocols())

def add_employee(values):
    employees = db_worker.get_employees()
    if values[0] not in [x[0] for x in employees]:
        return db_worker.add_employee(values)
    else:
        return "User Already In System"

def update_employee(values):
    return db_worker.update_employee(values)

```

```
def delete_employee(values):
    name = values[0]
    return db_worker.delete_employee(name)

def exists(values):
    name = values[0]
    return str(db_worker.exists(name))

def get_protocols_id(_):
    return json.dumps(db_worker.get_protocols())

def add_protocol(values):
    name, number, version, contents = values
    return db_worker.add_protocol(name, number, version, contents)

def delete_protocol(values):
    name, version = values
    return db_worker.delete_protocol(name, version)

def add_test(values):
    return db_worker.add_test(values)

def get_history(values):
    page = values[0]
    return json.dumps(db_worker.get_history(page))

def get_len_history(_):
    return json.dumps(db_worker.get_len_history())

def can_join_user(values):
    name = values[0]
    user = db_worker.can_join_user(name)
    if user is None:
        return "User not in system."
    if user[1] is not None:
        return "User already joined."
    if user[4] == "":
        return "Gmail address not in system."
    return "Confirmed" + str(user[4])

def add_password(values):
    name, password = values
    return db_worker.add_password(name, password)
```

```
def update_status(values):
    return db_worker.update_status(values)

def get_progress(_):
    return json.dumps(db_worker.get_progress())

def get_employee_names(_):
    n = json.dumps(db_worker.get_employee_names())
    return n

def get_last_version(name):
    return json.dumps(db_worker.get_last_version(name[0]))

def get_comments(row_id):
    x = db_worker.get_comments(row_id[0])
    if x is None:
        return "None"
    return x

def get_last_history_id(_):
    return json.dumps(db_worker.get_last_history_id())

def get_protocol_names(_):
    return json.dumps(db_worker.get_protocol_names())

def get_filtered_history(values):
    return json.dumps(db_worker.get_filtered_history(values[0], values[1],
values[2]))

menu = {"Employee": "employee=", "Protocol": "protocol_name=", "No
Filter": None, "Date Range": "date between"}

def get_4_pages_filtered(values):
    start_id = values[0]
    filters = json.loads(values[1])
    values = json.loads(values[2])
    filters = [menu[x] for x in filters]
    return json.dumps(db_worker.get_4_pages_filtered(start_id, filters,
values))

def commit(_):
    return db_worker.commit()
```

```

def filler(_, _1):
    pass

MSG_LENGTH = 1024
PORT = 1097
IP = "127.0.0.1"

# Server communication
class Comm(socket.socket):
    def __init__(self):
        super().__init__(socket.AF_INET, socket.SOCK_STREAM)
        # variables to create random numbers with
        self.difference_between_last_joins = 0
        self.last_time_when_client_joined = time.time()
        self.when_each_ip_was_last_connected = {}
        self.each_ip_difference_from_last_join = {}
        # keep each client's address
        self.client_addresses = {}
        self.bind((IP, PORT))
        self.listen()
        self.client_permissions = {}
        self.client_codes = {}
        # self.gen_requests = [get_user, get_employees, get_content,
        get_protocols, add_employee, update_employee,
        # delete_employee, exists, get_protocols_id,
        add_protocol, delete_protocol, add_test,
        # get_history, get_len_history,
        can_join_user, add_password, update_status, get_progress,
        # get_employee_names, get_last_version,
        get_comments, get_filtered_history,
        # get_4_pages_filtered, commit]
        self.manager_requests = [get_employees, add_employee,
        update_employee, delete_employee, add_protocol,
        delete_protocol, get_progress]
        self.general_requests = [get_content, get_protocols, add_test,
        get_history, get_len_history, update_status,
        get_last_version, get_comments,
        get_filtered_history, get_4_pages_filtered, commit,
        add_password, get_protocol_names,
        get_employee_names, get_last_history_id]
        self.complex_requests = [self.ask_]
        self.log_options = [self.log_in, self.verify_admission_code,
        filler]
        self.rabbit_hole = {"man": self.manager_requests, "gen":
        self.general_requests, "log": self.log_options,
        "plx": self.complex_requests}

        self.clients = []
        self.messages_out = []
        self.client_keys = {}
        # self.public, self.private = rsa.generate_keypair(13, 67)
        # print("self public key", self.public)

```

```

    @staticmethod
    def send_message(current_client, msg):
        length = len(msg)
        repetition = length // MSG_LENGTH
        current_client.send(f"#{repetition}".encode())
        for i in range(repetition):
            current_client.send(msg[i * MSG_LENGTH:(i + 1) *
MSG_LENGTH].encode())
        if length % 1024 != 0:
            current_client.send(msg[repetition * MSG_LENGTH:].encode())

    def run(self):
        while True:
            read_list, write_list, x_list = select.select([self] +
self.clients, self.clients, [])
            for current_socket in read_list:
                # accept a client trying to connect if there's available
room
                if current_socket is self:
                    if len(self.clients) <= 20:
                        client, address = self.accept()
                        print("new client joined")
                        self.clients.append(client)
                        self.client_addresses[client] = address[0]
                        client.send("Confirmed".encode())
                        t = time.time()
                        self.difference_between_last_joins = t -
self.last_time_when_client_joined
                        self.last_time_when_client_joined = t
                        if address[0] in
self.when_each_ip_was_last_connected:
self.each_ip_difference_from_last_join[address[0]] = t - \
self.when_each_ip_was_last_connected[
address[0]]
                    else:
self.each_ip_difference_from_last_join[address[0]] = t
                        self.when_each_ip_was_last_connected[address[0]] =
t
                        # reject client if there are too many clients
connected
                    else:
                        sock, add = self.accept()
                        sock.send("Deny".encode())
                        sock.close()

                else:
                    # if client closed, shut it down
                    print("data from client:")
                    try:
                        data = current_socket.recv(3).decode()

```

```

        except ConnectionResetError:
            data = ""
        if data == "":
            self.close_connection(current_socket)

        # exchange encryption keys
        elif data == "key":
            var1, var2 =
rsa.generate_randoms(self.difference_between_last_joins,
self.each_ip_difference_from_last_join[
self.client_addresses[current_socket]])
            public, private = rsa.generate_keypair(var1, var2)
            print("self public key", public)
            client_key = current_socket.recv(1024).decode()
            client_key = json.loads(client_key)
            print("client public key", client_key)
            self.client_keys[current_socket] = (client_key,
private)

            current_socket.send(json.dumps(public).encode())

        # send encrypted messages to clients according to
their permissions
        elif current_socket in self.client_keys:
            # print("repetition", data)
            repetition = int(data)
            msg_got = ""
            for i in range(repetition + 1):
                msg_got = msg_got +
current_socket.recv(MSG_LENGTH).decode()
                time.sleep(0.001)
                msg_got = msg_got.split(",")
                msg_got = [int(n) for n in msg_got]
                msg_got =
rsa.decrypt(self.client_keys[current_socket][1], msg_got)
                print("MSG GOT", msg_got)
                msg_got = msg_got.split("|")
                dic = self.rabbit_hole[msg_got[0]]
                print(dic)
                if msg_got[0] == "plx":
                    self.ask_(current_socket, msg_got[2])
                else:
                    if current_socket in self.client_permissions:
                        if msg_got[0] == "gen":
                            command = dic[int(msg_got[1])]
                            print(command.__name__)
                            msg_back = command(msg_got[2:])
                        elif msg_got[0] == "man" and
self.client_permissions[current_socket].endswith(
                    "Manager"):
                            command = dic[int(msg_got[1])]
                            print(command.__name__)
                            msg_back = command(msg_got[2:])

```

```

        else:
            msg_back = "Denied"
        elif msg_got[0] == "log":
            command =
self.log_options[int(msg_got[1])]
            print(command.__name__)
            msg_back = command(msg_got[2:],
current_socket)

        else:
            msg_back = "Denied"
            self.messages_out.append((current_socket,
msg_back))

        # if current_socket in self.client_permissions:
        #     command = self.gen_requests[int(msg_got[0])]
        #     print(command.__name__)
        #     msg_back = command(msg_got[1:])
        # else:
        #     command = self.log_options[int(msg_got[0])]
        #     print(command.__name__)
        #     msg_back = command(msg_got[1:],
current_socket)

        else:
            self.close_connection(current_socket)
        # send back messages to clients
        for message in self.messages_out:
            current_socket, data = message
            if current_socket in write_list:
                data =
rsa.encrypt(self.client_keys[current_socket][0], data)
                data = ",".join([str(x) for x in data])
                length = len(data)
                repetition = length // MSG_LENGTH
                current_socket.send(str(repetition).zfill(3).encode())
                for i in range(repetition):
                    current_socket.send(data[i * MSG_LENGTH:(i + 1) *
MSG_LENGTH].encode())
                if length % 1024 != 0:
                    current_socket.send(data[repetition *
MSG_LENGTH:].encode())
                self.messages_out.remove(message)

        # close a certain client and remove them from lists
        def close_connection(self, current_socket):
            commit(0)
            current_socket.close()
            self.clients.remove(current_socket)
            if current_socket in self.client_keys:
                self.client_keys.pop(current_socket)
            if current_socket in self.client_permissions:
                self.client_permissions.pop(current_socket)
            print("Closed")

        # log in user of send them a code

```

```

def log_in(self, values, current_socket):
    user = json.loads(get_user(values))
    if user is None:
        return "Not In System"
    if values[1] == "send_mail":
        # if user[1] is not None:
        #     return "User Already In System"
        t = time.time()
        m = threading.Thread(target=self.send_mail2,
args=(current_socket, user[4], t, user[2]))
        m.start()
        # self.send_mail(current_socket, user[4], t, user[2])
        return user[2]
    if user[1] == values[1]:
        self.client_permissions[current_socket] = user[2]
        return user[2]
    return "Denied"

# send mail with verification code to user
def send_mail2(self, current_socket, address, send_time, role):
    print(address)
    msg = EmailMessage()
    code = random.randrange(1000, 10000)
    self.client_codes[current_socket] =
(hashlib.md5(str(code).encode()).hexdigest(), send_time, role)
    msg.set_content(f'Your Confirmation Code is: {code}')
    msg['Subject'] = "Join Yaelotron"
    msg['To'] = address
    s = smtplib.SMTP_SSL('smtp.gmail.com', 465)
    s.login("dnhqwrg@gmail.com", os.environ.get("GMAIL_PASSWORD"))
    s.send_message(msg)
    s.quit()

# verify the code from the user with the code sent to them
def verify_admission_code(self, values, current_socket):
    code = hashlib.md5(values[1].encode()).hexdigest()
    codes = self.client_codes[current_socket]
    if time.time() - codes[1] <= 60:
        if codes[0] == code:
            self.client_permissions[current_socket] = codes[2]
            self.client_codes.pop(current_socket)
            add_password(values)
            return "Confirmed"
        else:
            return "Denied"
    return "Timed Out"

# start thread to ask gemini
def ask_(self, current_socket, text):
    t = threading.Thread(target=self.ask_gemini, args=(current_socket,
text))
    t.start()

# talks to gemini and adds response to messages back list

```

```

def ask_gemini(self, current_socket, text):
    load_dotenv("hey.env")

    instructions = open("examples/instructions", "r").read()

    client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY"))

    response = client.models.generate_content(
        model="gemini-2.0-flash", contents=[instructions, text]
    )
    self.messages_out.append((current_socket, response.text))
    return response.text

if __name__ == '__main__':
    c = Comm()
    c.run()

```

הקובץ db_worker:

```

import sqlite3

# Establish a connection to the SQLite database and create a cursor for
# executing SQL commands
con = sqlite3.connect("base.db")
cur = con.cursor()

# Retrieve a user by name from the employees table
def get_user(name):
    user = cur.execute("SELECT * FROM employees WHERE name = ?",
        (name,)).fetchall()
    if not user:
        return None
    else:
        return user[0]

# Get all employees with their name, Gmail, and level
def get_employees():
    return cur.execute("SELECT name, gmail, level FROM
employees").fetchall()

# Get the content of a specific protocol based on name, id number, and
# version
def get_content(name, number, version):
    return cur.execute("SELECT content FROM protocols WHERE id = ? AND
version = ? AND name = ?",
        (number, version, name)).fetchall()[0][0]

# Get all protocols with their name, id, and version
def get_protocols():
    return cur.execute("SELECT name, id, version FROM
protocols").fetchall()

```

```

# Add a new employee to the database
def add_employee(values):
    cur.execute("INSERT INTO employees (name, gmail, level) VALUES (?, ?, ?)", values)
    return "Confirmed"

# Update an existing employee's Gmail and level based on their name
def update_employee(values):
    cur.execute("UPDATE employees SET gmail = ?, level = ? WHERE name = ?", (values[1], values[2], values[0]))
    return "Confirmed"

# Delete an employee by name
def delete_employee(name):
    cur.execute("DELETE FROM employees WHERE name = ?", (name,))
    return "Confirmed"

# Check if a user exists by name
def exists(name):
    if cur.execute("SELECT * FROM employees WHERE name = ?", (name,)).fetchall():
        return True
    return False

# Get all protocol records with name, id, and version
def get_protocols_id():
    return cur.execute("SELECT name, id, version FROM protocols").fetchall()

# Add a new protocol to the database
def add_protocol(name, number, version, contents):
    cur.execute("INSERT INTO protocols (id, version, name, content) VALUES (?, ?, ?, ?)",
                (number, version, name, contents))
    return "Confirmed"

# Delete a specific version of a protocol by name
def delete_protocol(name, version):
    cur.execute("DELETE FROM protocols WHERE name = ? AND version = ?", (name, version))
    return "Confirmed"

# Add a test history record to the history table
def add_test(values):
    cur.execute(

```

```

        "INSERT INTO history (protocol_name, protocol_id,
protocol_version, employee, comment) VALUES (?, ?, ?, ?, ?)",
        values)
    return "Confirmed"

# Get a page of history records, 15 per page, ordered by most recent
def get_history(page):
    return cur.execute(
        "SELECT id, protocol_id, protocol_name, protocol_version,
employee, date FROM history ORDER BY id DESC LIMIT 15 OFFSET ?",
        (int(page) * 15,)).fetchall()

# Get a filtered page of history based on a specific column and value
def get_filtered_history(page, column_filter, value):
    return cur.execute(
        f"SELECT id, protocol_id, protocol_name, protocol_version,
employee, date FROM history WHERE {column_filter} = ? ORDER BY id DESC
LIMIT 15 OFFSET ?",
        (value, int(page) * 15)).fetchall()

# Get the highest id value from the history table
def get_len_history():
    return cur.execute("SELECT id FROM history ORDER BY id DESC LIMIT
1").fetchall()[0][0]

# Retrieve an employee by name, used to determine if they can join
def can_join_user(name):
    user = cur.execute("SELECT * FROM employees WHERE name = ?",
(name,)).fetchall()
    if not user:
        return None
    return user[0]

# Set a password for an employee
def add_password(name, password):
    cur.execute("UPDATE employees SET password = ? WHERE name = ?",
(password, name))
    return "Confirmed"

# Update the status of an employee
def update_status(values):
    cur.execute("UPDATE employees SET status = ? WHERE name = ?", values)
    return "Confirmed"

# Get a list of employee names and their current status
def get_progress():
    return cur.execute("SELECT name, status FROM employees").fetchall()

```

```

# Get the highest version number of a given protocol name
def get_last_version(name):
    return cur.execute("SELECT MAX(version) FROM protocols WHERE name = ?", (name,)).fetchall()[0][0]

# Get the names of all employees
def get_employee_names():
    return cur.execute("SELECT name FROM employees").fetchall()

# Get comments from a specific history entry by its ID
def get_comments(row_id):
    return cur.execute("SELECT comment FROM history WHERE id = ?", (row_id,)).fetchall()[0][0]

# Get the ID of the most recent history entry
def get_last_history_id():
    return cur.execute("SELECT MAX(id) FROM history").fetchall()[0][0]

# Get a list of all unique protocol names
def get_protocol_names():
    return cur.execute("SELECT DISTINCT name FROM protocols").fetchall()

# Get up to 4 pages (48 entries) of filtered history records starting from a given ID
def get_4_pages_filtered(start_id, filters, values):
    query = "SELECT id, protocol_id, protocol_name, protocol_version, employee, date FROM history WHERE id < ?"
    for i in range(len(filters)):
        query += f" AND {filters[i]} {values[i]} "
    query += "ORDER BY id DESC LIMIT 48"
    print(query, start_id)
    return cur.execute(query, (start_id,)).fetchall()

# Commit the current transaction to the database
def commit():
    con.commit()
    return "Confirmed"

# Close the connection to the database
def close():
    con.close()
    return "Confirmed"

```